FedMed

Complete Source Code Documentation

A federated cloud platform for privacy-preserving medical image intelligence

Total files: 68

Generated automatically from project source

Table of Contents

Root

README.md
Makefile
docker-compose.yml
requirements-dev.txt
pytest.ini
.gitignore

Common

common/pyproject.toml
common/fedmed/__init__.py
common/fedmed/config.py
common/fedmed/data.py
common/fedmed/model.py
common/fedmed/metrics.py
common/fedmed/privacy.py
common/fedmed/storage.py

Hospital Node

hospital_node/Dockerfile
hospital_node/requirements.txt
hospital_node/client.py

Fl Server

fl_server/Dockerfile
fl_server/requirements.txt
fl_server/server.py
fl_server/strategy.py

Inference Api

inference_api/Dockerfile
inference_api/Dockerfile.local
inference_api/requirements.txt
inference_api/app.py

Dashboard

dashboard/index.html
dashboard/package.json
dashboard/vite.config.js
dashboard/src/main.jsx
dashboard/src/theme.css
dashboard/src/api.js
dashboard/src/App.jsx
dashboard/src/components/AccuracyChart.jsx
dashboard/src/components/HospitalPanel.jsx
dashboard/src/components/PredictPanel.jsx
dashboard/src/components/PrivacyPanel.jsx

Scripts

scripts/partition_data.py
scripts/run_local.py
scripts/run_experiments.py
scripts/train_centralized.py
scripts/aws_common.sh
scripts/gen_certs.sh
scripts/push_images.sh
scripts/start_run.sh
scripts/stop_run.sh
scripts/deploy_web.sh
scripts/upload_data.sh
scripts/teardown.sh

Tests

tests/test_model_and_data.py
tests/test_inference_api.py
tests/test_partitioning.py
tests/test_privacy_and_storage.py

Infra

infra/versions.tf
infra/variables.tf
infra/main.tf
infra/ecr.tf
infra/ecs.tf
infra/storage.tf
infra/lambda.tf
infra/ssm.tf
infra/web.tf
infra/outputs.tf
infra/terraform.tfvars.example

Docs

docs/architecture.md
docs/threat-model.md
docs/cost.md
docs/runbook.md
docs/demo-script.md

README.md

````
# FedMed

**A federated cloud platform for privacy-preserving medical image intelligence.**

Three hospitals train one pneumonia classifier on their combined experience
without a single X-ray leaving the site that owns it. Built on AWS Fargate,
Lambda and S3, sized to run a full demo for about one and a half cents.

Cloud computing course project. Team of three.

---

## What it does

Each hospital runs its own container with its own private data bucket. A central
aggregator sends out the current model, each site trains on its own patients, and
the aggregator averages the returned weights. Images never appear in a request.
The resulting model is served through a public API and a live dashboard.

| | |
|---|---|
| **Federated learning** | Flower, FedAvg over three sites, ten rounds |
| **Dataset** | PneumoniaMNIST at 64 pixels, split with Dirichlet(0.5) so the sites are genuinely unlike each other |
| **Privacy** | Data locality by construction, server-side clipping and Gaussian noise, pinned TLS, hash-only inferen
| **Compute** | ECS Fargate one-off tasks, no cluster nodes, no idle cost |
| **Serving** | FastAPI on Lambda behind an HTTP API Gateway |
| **Dashboard** | React on an S3 static website |
| **Infrastructure** | Terraform, targeted at an AWS Academy Learner Lab |

## Results

Measured by `scripts/run_experiments.py`, ten rounds or epochs throughout.

| Configuration | Final accuracy | Best round | vs pooled |
|---|---|---|---|
| All data pooled on one server | 89.4% | 89.7% | baseline |
| Federated, even split | 85.4% | 87.9% | -4.0 pts |
| Federated, realistic uneven split | 80.4% | 87.8% | -9.0 pts |
| Federated non-IID, light privacy noise | 87.0% | 87.0% | -2.4 pts |
| Federated non-IID, heavy privacy noise | 67.9% | 67.9% | -21.4 pts |

The four-point gap between pooled and evenly-split federated training is the cost
of the architecture itself. The further five points is the cost of the hospitals
being genuinely different from one another.

**On the privacy budget:** the accountant reports an epsilon in the thousands at
the default noise level, which is not a meaningful formal guarantee. With three
sites all participating in every round there is no subsampling amplification to
collect, and noise strong enough to buy a tight budget costs twenty to thirty
accuracy points. The platform reports its exact parameters and the resulting
number rather than calling itself private. See
[docs/threat-model.md](docs/threat-model.md).

---

## Quick start

```bash
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements-dev.txt   # Windows
````
````

```
# source .venv/bin/activate && pip install -r requirements-dev.txt  # macOS, Linux

python scripts/partition_data.py     # download and split the dataset
python -m pytest                     # 43 tests
python scripts/run_local.py --rounds 10 --dp
```

Ten rounds finish in about thirty seconds on a laptop CPU. Then the same thing in
containers:

```bash
docker compose up --build --abort-on-container-exit --exit-code-from fl-server
```

Without those two flags the command never returns: the inference API is a server
that does not exit, so compose stays attached long after the rounds finish.

Deploying to AWS is docs/runbook.md, start to finish.

Layout

```
common/fedmed/      shared library: model, data, storage, metrics, privacy accounting
hospital_node/      the client that trains on one site's data and never sends images
fl_server/          the aggregator: FedAvg, differential privacy, checkpointing
inference_api/      FastAPI application, packaged for Lambda
dashboard/          React console, hand-drawn SVG charts, no charting dependency
infra/              Terraform for the whole stack
scripts/            partition, run, sweep, deploy, teardown
tests/              43 tests, including one asserting predictions persist nothing
docs/               architecture, threat model, cost, runbook, demo script
```

Documentation

docs/architecture.md	How it fits together, and why each choice was made
docs/threat-model.md	What is protected, what is not, and why
docs/cost.md	Under three dollars, and what the alternatives would have cost
docs/runbook.md	Deployment, operation, and what to do when it breaks
docs/demo-script.md	The eight-minute presentation

Common commands

```bash
make help              # every target
make local             # federation as plain processes
make compose           # federation as containers
make experiments       # the evaluation table and figures
make test

./scripts/push_images.sh    # build and push to ECR
./scripts/upload_data.sh    # one shard per bucket
./scripts/start_run.sh      # one training run on Fargate
./scripts/stop_run.sh       # stop everything. Run this before closing the lab
./scripts/deploy_web.sh     # publish the dashboard
```

```
## Two things to know before running it on AWS

**Stop your tasks before closing the lab.** An AWS Academy session expires after
four hours, but a Fargate task started inside it keeps running and keeps billing
after the console has logged you out. `./scripts/stop_run.sh` is the last command
of every session.

**Apply Terraform in two phases.** The Lambda function needs an image that only
exists once the registries have been created and pushed to.
`terraform apply -target=aws_ecr_repository.this` first, then push, then a full
apply. The runbook walks through it.

---

## Not a medical device

A teaching demonstration built on a public, de-identified research dataset. No
output should inform a clinical decision.

```

Makefile

```
# FedMed task runner.
#
# Every target is a thin wrapper around a script, so nothing here is required.
# If make is not installed (common on Windows), run the script in the recipe
# directly and you get exactly the same result.

SHELL := /bin/bash
PYTHON ?= python
VENV ?= .venv

.DEFAULT_GOAL := help
.PHONY: help setup data certs test lint local compose compose-down experiments \
        tf-init tf-repos tf-apply tf-output push upload web run stop teardown clean

help: ## Show this list
    @grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) \
        | awk 'BEGIN {FS = ":.*?## "}; {printf "  \033[36m%-16s\033[0m %s\n", $$1, $$2}'

# ------------------------------------------------------------------- local

setup: ## Create the virtualenv and install everything
    $(PYTHON) -m venv $(VENV)
    $(VENV)/Scripts/python.exe -m pip install --upgrade pip || $(VENV)/bin/python -m pip install --upgrade pip
    $(VENV)/Scripts/python.exe -m pip install -r requirements-dev.txt || $(VENV)/bin/python -m pip install -r require

data: ## Download PneumoniaMNIST and split it across three hospitals
    $(PYTHON) scripts/partition_data.py

certs: ## Mint the self-signed CA and aggregator certificate
    ./scripts/gen_certs.sh

test: ## Run the test suite
    $(PYTHON) -m pytest

local: ## Run the federation as plain processes, no Docker
    $(PYTHON) scripts/run_local.py --rounds 10 --dp

compose: ## Run the federation as containers, the way Fargate will
    # The two flags stop the stack when the aggregator finishes. Without them
    # compose stays attached to the inference API, which never exits, and the
    # terminal looks frozen long after training is done.
    docker compose up --build --abort-on-container-exit --exit-code-from fl-server

compose-down: ## Stop the compose stack and remove its containers
    docker compose down -v

experiments: ## Produce the evaluation table and figures for the report
    $(PYTHON) scripts/run_experiments.py

# ------------------------------------------------------------------- aws

tf-init: ## Initialise Terraform
    terraform -chdir=infra init

tf-repos: ## Phase one: create only the ECR repositories, so images can be pushed
    terraform -chdir=infra apply -target=aws_ecr_repository.this

tf-apply: ## Phase two: create everything else
    terraform -chdir=infra apply

```

```
tf-output: ## Show the stack's outputs
    terraform -chdir=infra output

push: ## Build the three images and push them to ECR
    ./scripts/push_images.sh

upload: ## Copy each hospital's shard into its own bucket
    ./scripts/upload_data.sh

web: ## Build the dashboard against the deployed API and publish it
    ./scripts/deploy_web.sh

run: ## Start one federated training run on Fargate
    ./scripts/start_run.sh

stop: ## Stop every running task. Run this before you close the lab
    ./scripts/stop_run.sh

teardown: ## Destroy the whole stack
    ./scripts/teardown.sh

# ------------------------------------------------------------------ chores

clean: ## Remove generated artifacts, keeping the datasets
    rm -rf artifacts dashboard/dist .fedmed-run

```

docker-compose.yml

```
# The whole platform on one laptop.
#
#   python scripts/partition_data.py     # once, to create the shards
#   docker compose up --build
#
# This mirrors the Fargate deployment closely enough to be a real rehearsal: the
# same four images, the same environment variables, the same startup race between
# hospitals and aggregator. The only differences are that storage is a mounted
# volume instead of S3, and Docker's DNS stands in for AWS Cloud Map.

services:
  fl-server:
    build:
      context: .
      dockerfile: fl_server/Dockerfile
    image: fedmed-server:local
    container_name: fedmed-fl-server
    environment:
      RUN_ID: compose-run
      NUM_ROUNDS: "10"
      NUM_HOSPITALS: "3"
      BIND_ADDRESS: 0.0.0.0:8080
      STORAGE_BACKEND: local
      LOCAL_ROOT: /workspace/artifacts
      DP_ENABLED: "true"
      DP_NOISE_MULTIPLIER: "0.05"
      DP_CLIP_NORM: "1.0"
      TLS_ENABLED: "false"
      LOG_LEVEL: INFO
    volumes:
      - ./artifacts:/workspace/artifacts
    ports:
      - "8080:8080"

  # The three hospitals are identical except for HOSPITAL_ID and the single
  # directory each one is allowed to see. Note the :ro mount and that no hospital
  # has the artifacts volume: a node can neither write the global model nor read
  # another site's images.
  hospital-a:
    build:
      context: .
      dockerfile: hospital_node/Dockerfile
    image: fedmed-hospital:local
    container_name: fedmed-hospital-a
    depends_on: [fl-server]
    environment:
      HOSPITAL_ID: a
      SERVER_ADDRESS: fl-server:8080
      SHARD_PATH: /workspace/data/shard.npz
      STORAGE_BACKEND: local
      LOCAL_EPOCHS: "1"
      BATCH_SIZE: "64"
      LEARNING_RATE: "0.001"
      TLS_ENABLED: "false"
    volumes:
      - ./data/hospital_a:/workspace/data:ro

  hospital-b:
    # The same build block as hospital-a on purpose. Compose deduplicates by
    # image tag, so this builds once; declaring it here means the service does
```

```
    # not try to pull a tag that only exists because another service built it.
    build:
      context: .
      dockerfile: hospital_node/Dockerfile
    image: fedmed-hospital:local
    container_name: fedmed-hospital-b
    depends_on: [hospital-a]
    environment:
      HOSPITAL_ID: b
      SERVER_ADDRESS: fl-server:8080
      SHARD_PATH: /workspace/data/shard.npz
      STORAGE_BACKEND: local
      LOCAL_EPOCHS: "1"
      BATCH_SIZE: "64"
      LEARNING_RATE: "0.001"
      TLS_ENABLED: "false"
    volumes:
      - ./data/hospital_b:/workspace/data:ro

  hospital-c:
    # The same build block as hospital-a on purpose. Compose deduplicates by
    # image tag, so this builds once; declaring it here means the service does
    # not try to pull a tag that only exists because another service built it.
    build:
      context: .
      dockerfile: hospital_node/Dockerfile
    image: fedmed-hospital:local
    container_name: fedmed-hospital-c
    depends_on: [hospital-a]
    environment:
      HOSPITAL_ID: c
      SERVER_ADDRESS: fl-server:8080
      SHARD_PATH: /workspace/data/shard.npz
      STORAGE_BACKEND: local
      LOCAL_EPOCHS: "1"
      BATCH_SIZE: "64"
      LEARNING_RATE: "0.001"
      TLS_ENABLED: "false"
    volumes:
      - ./data/hospital_c:/workspace/data:ro

  inference-api:
    build:
      context: .
      dockerfile: inference_api/Dockerfile.local
    # Tagged -dev so it can never collide with the Lambda image, which is built
    # from inference_api/Dockerfile and would otherwise claim the same name. The
    # two images share application code but have completely different entry
    # points, and starting the Lambda runtime here would look like a hang.
    image: fedmed-api:local-dev
    container_name: fedmed-api
    environment:
      STORAGE_BACKEND: local
      LOCAL_ROOT: /workspace/artifacts
      MODEL_CACHE_SECONDS: "5"
      CORS_ORIGINS: "*"
    volumes:
      - ./artifacts:/workspace/artifacts:ro
    ports:
      - "8000:8000"

```

requirements-dev.txt

```
# Everything needed to run the federation locally and produce the report figures.
-e ./common
numpy==1.26.4
torch==2.5.1
flwr==1.15.2
medmnist==3.0.2
boto3==1.35.68
fastapi==0.115.5
uvicorn==0.32.1
mangum==0.19.0
pillow==11.0.0
python-multipart==0.0.20
matplotlib==3.9.2
pytest==8.3.4
httpx==0.28.1

```

pytest.ini

```
[pytest]
testpaths = tests
addopts = -q --strict-markers
filterwarnings =
    ignore::DeprecationWarning:flwr.*

```

.gitignore

```
# Datasets and generated artifacts. The shards are reproducible from
# scripts/partition_data.py, so there is no reason to carry them in git.
data/
data_iid/
artifacts/
*.npz

# Certificate material. Committing a private key is the classic own goal.
certs/
*.pem
*.key
*.crt
*.srl

# Terraform. State can hold secrets, and the provider plugins are large.
infra/.terraform/
infra/*.tfstate
infra/*.tfstate.*
infra/*.tfplan
# The provider lock file IS committed on purpose, so every team member
# resolves the same AWS provider version.
infra/terraform.tfvars

# Run bookkeeping written by scripts/start_run.sh
.fedmed-run

# Python
.venv/
__pycache__/
*.py[cod]
*.egg-info/
.pytest_cache/

# Node
dashboard/node_modules/
dashboard/dist/
dashboard/public/partition_summary.json
dashboard/public/centralized_baseline.json

# Editors and OS
.vscode/
.idea/
.DS_Store
Thumbs.db

```

common/pyproject.toml

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "fedmed"
version = "0.1.0"
description = "Shared library for the FedMed federated medical imaging platform"
requires-python = ">=3.10"
dependencies = []

[tool.setuptools.packages.find]
where = ["."]
include = ["fedmed*"]

```

common/fedmed/__init__.py

```
"""Shared library for the FedMed federated medical-imaging platform."""

__version__ = "0.1.0"

```

common/fedmed/config.py

```python
"""Environment-driven configuration shared by every FedMed component.

Every knob the demo needs is an environment variable so the same container image
runs unchanged locally (docker compose) and on AWS Fargate.
"""

from __future__ import annotations

import os
from dataclasses import dataclass, field


def _env(name: str, default: str) -> str:
    value = os.environ.get(name)
    return default if value is None or value == "" else value


def _env_int(name: str, default: int) -> int:
    return int(_env(name, str(default)))


def _env_float(name: str, default: float) -> float:
    return float(_env(name, str(default)))


def _env_bool(name: str, default: bool) -> bool:
    return _env(name, "true" if default else "false").strip().lower() in {
        "1",
        "true",
        "yes",
        "on",
    }


@dataclass
class TrainingConfig:
    """Hyper-parameters for local training on a hospital node."""

    local_epochs: int = field(default_factory=lambda: _env_int("LOCAL_EPOCHS", 1))
    batch_size: int = field(default_factory=lambda: _env_int("BATCH_SIZE", 64))
    learning_rate: float = field(default_factory=lambda: _env_float("LEARNING_RATE", 1e-3))
    seed: int = field(default_factory=lambda: _env_int("SEED", 42))


@dataclass
class FederationConfig:
    """Topology and round settings for the federation."""

    server_address: str = field(
        default_factory=lambda: _env("SERVER_ADDRESS", "fl-server:8080")
    )
    bind_address: str = field(default_factory=lambda: _env("BIND_ADDRESS", "0.0.0.0:8080"))
    num_rounds: int = field(default_factory=lambda: _env_int("NUM_ROUNDS", 10))
    num_hospitals: int = field(default_factory=lambda: _env_int("NUM_HOSPITALS", 3))
    run_id: str = field(default_factory=lambda: _env("RUN_ID", "local-run"))


@dataclass
class PrivacyConfig:
    """Differential-privacy knobs applied server side by the Flower strategy."""
```

```
61
62          enabled: bool = field(default_factory=lambda: _env_bool("DP_ENABLED", True))
63          noise_multiplier: float = field(
64              default_factory=lambda: _env_float("DP_NOISE_MULTIPLIER", 0.05)
65          )
66          clipping_norm: float = field(default_factory=lambda: _env_float("DP_CLIP_NORM", 1.0))
67          delta: float = field(default_factory=lambda: _env_float("DP_DELTA", 1e-5))
68
69
70      @dataclass
71      class StorageConfig:
72          """Where model checkpoints, metrics and data shards live."""
73
74          backend: str = field(default_factory=lambda: _env("STORAGE_BACKEND", "local"))
75          local_root: str = field(default_factory=lambda: _env("LOCAL_ROOT", "/workspace/artifacts"))
76          models_bucket: str = field(default_factory=lambda: _env("MODELS_BUCKET", ""))
77          data_bucket: str = field(default_factory=lambda: _env("DATA_BUCKET", ""))
78          rounds_table: str = field(default_factory=lambda: _env("ROUNDS_TABLE", "fedmed-rounds"))
79          audit_table: str = field(default_factory=lambda: _env("AUDIT_TABLE", "fedmed-audit"))
80          aws_region: str = field(default_factory=lambda: _env("AWS_REGION", "us-east-1"))
81          cloudwatch_namespace: str = field(
82              default_factory=lambda: _env("CLOUDWATCH_NAMESPACE", "FedMed")
83          )
84          push_cloudwatch: bool = field(default_factory=lambda: _env_bool("PUSH_CLOUDWATCH", False))
85
86
87      @dataclass
88      class TLSConfig:
89          """Paths to the self-signed certificate material for the Flower gRPC channel."""
90
91          enabled: bool = field(default_factory=lambda: _env_bool("TLS_ENABLED", False))
92          ca_path: str = field(default_factory=lambda: _env("TLS_CA_PATH", "/certs/ca.crt"))
93          cert_path: str = field(default_factory=lambda: _env("TLS_CERT_PATH", "/certs/server.crt"))
94          key_path: str = field(default_factory=lambda: _env("TLS_KEY_PATH", "/certs/server.key"))
95
```

common/fedmed/data.py

```
"""Dataset loading, local training and evaluation helpers.

A hospital shard is a single ``.npz`` file holding ``images`` (uint8, N x 64 x 64)
and ``labels`` (int64, N). Keeping shards as plain NumPy archives means a
hospital container needs no database and no dataset library at runtime, and it
makes the "this file never leaves the node" claim easy to inspect.
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Dict, Tuple

import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

from .model import CLASS_NAMES, IMAGE_SIZE

# MedMNIST chest X-rays are 8-bit grayscale. Normalising to roughly zero mean and
# unit variance keeps the small network's gradients well scaled.
PIXEL_MEAN = 0.5
PIXEL_STD = 0.5


@dataclass
class Shard:
    """One hospital's private slice of the dataset (or the server's test set)."""

    images: np.ndarray  # uint8, (N, H, W)
    labels: np.ndarray  # int64, (N,)
    name: str = "shard"

    def __len__(self) -> int:
        return int(self.images.shape[0])

    @property
    def class_counts(self) -> Dict[str, int]:
        counts = np.bincount(self.labels.astype(np.int64), minlength=len(CLASS_NAMES))
        return {name: int(count) for name, count in zip(CLASS_NAMES, counts)}


def load_shard(path: str, name: str = "shard") -> Shard:
    """Read a shard from disk and sanity-check its shape."""
    with np.load(path) as archive:
        images = archive["images"]
        labels = archive["labels"]
    images = np.asarray(images)
    labels = np.asarray(labels).reshape(-1).astype(np.int64)
    if images.ndim == 4 and images.shape[-1] == 1:  # (N, H, W, 1) -> (N, H, W)
        images = images[..., 0]
    if images.ndim != 3:
        raise ValueError(f"{path}: expected images with shape (N, H, W), got {images.shape}")
    if images.shape[1] != IMAGE_SIZE or images.shape[2] != IMAGE_SIZE:
        raise ValueError(
            f"{path}: expected {IMAGE_SIZE}x{IMAGE_SIZE} images, got "
            f"{images.shape[1]}x{images.shape[2]}"
        )
    if images.shape[0] != labels.shape[0]:
```

```
            raise ValueError(f"{path}: {images.shape[0]} images but {labels.shape[0]} labels")
        return Shard(images=images, labels=labels, name=name)


    def to_tensors(shard: Shard) -> Tuple[torch.Tensor, torch.Tensor]:
        """Convert a shard to normalised float tensors of shape (N, 1, H, W)."""
        images = torch.from_numpy(shard.images.astype(np.float32) / 255.0).unsqueeze(1)
        images = (images - PIXEL_MEAN) / PIXEL_STD
        labels = torch.from_numpy(shard.labels.astype(np.int64))
        return images, labels


    def make_loader(shard: Shard, batch_size: int, shuffle: bool) -> DataLoader:
        images, labels = to_tensors(shard)
        dataset = TensorDataset(images, labels)
        return DataLoader(
            dataset,
            batch_size=batch_size,
            shuffle=shuffle,
            drop_last=False,
            num_workers=0,  # Fargate tasks get one vCPU; workers would only add overhead.
        )


    def preprocess_image(array: np.ndarray) -> torch.Tensor:
        """Turn one raw grayscale image into a (1, 1, H, W) batch for inference."""
        if array.ndim == 3:
            array = array[..., 0] if array.shape[-1] == 1 else array.mean(axis=-1)
        tensor = torch.from_numpy(array.astype(np.float32) / 255.0).unsqueeze(0).unsqueeze(0)
        return (tensor - PIXEL_MEAN) / PIXEL_STD


    def train_local(
        model: nn.Module,
        shard: Shard,
        epochs: int,
        batch_size: int,
        learning_rate: float,
    ) -> Dict[str, float]:
        """Run local SGD on a hospital's own data and report training statistics.

        Only the numbers returned here plus the updated weights ever leave the node.
        """
        device = torch.device("cpu")
        model.to(device).train()
        loader = make_loader(shard, batch_size=batch_size, shuffle=True)
        criterion = nn.CrossEntropyLoss()
        optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

        total_loss = 0.0
        total_correct = 0
        total_seen = 0
        for _ in range(epochs):
            for images, labels in loader:
                optimizer.zero_grad()
                logits = model(images)
                loss = criterion(logits, labels)
                loss.backward()
                optimizer.step()

                total_loss += float(loss.item()) * labels.size(0)
                total_correct += int((logits.argmax(dim=1) == labels).sum().item())
                total_seen += int(labels.size(0))
```

```
124
125         return {
126             "train_loss": total_loss / max(total_seen, 1),
127             "train_accuracy": total_correct / max(total_seen, 1),
128             "examples": float(len(shard)),
129             "epochs": float(epochs),
130         }
131
132
133     @torch.no_grad()
134     def evaluate(model: nn.Module, shard: Shard, batch_size: int = 128) -> Dict[str, float]:
135         """Compute loss, accuracy and per-class recall on a held-out shard."""
136         model.eval()
137         loader = make_loader(shard, batch_size=batch_size, shuffle=False)
138         criterion = nn.CrossEntropyLoss(reduction="sum")
139
140         total_loss = 0.0
141         total_correct = 0
142         total_seen = 0
143         per_class_correct = np.zeros(len(CLASS_NAMES), dtype=np.int64)
144         per_class_total = np.zeros(len(CLASS_NAMES), dtype=np.int64)
145
146         for images, labels in loader:
147             logits = model(images)
148             total_loss += float(criterion(logits, labels).item())
149             predictions = logits.argmax(dim=1)
150             total_correct += int((predictions == labels).sum().item())
151             total_seen += int(labels.size(0))
152             for class_index in range(len(CLASS_NAMES)):
153                 mask = labels == class_index
154                 per_class_total[class_index] += int(mask.sum().item())
155                 per_class_correct[class_index] += int((predictions[mask] == class_index).sum().item())
156
157         metrics = {
158             "loss": total_loss / max(total_seen, 1),
159             "accuracy": total_correct / max(total_seen, 1),
160             "examples": float(total_seen),
161         }
162         for class_index, class_name in enumerate(CLASS_NAMES):
163             denominator = max(int(per_class_total[class_index]), 1)
164             metrics[f"recall_{class_name}"] = float(per_class_correct[class_index]) / denominator
165         return metrics
166
```

common/fedmed/model.py

```
"""The classification model shared by every hospital and the aggregator.

A deliberately small CNN: it has to converge on a single Fargate vCPU in seconds
per epoch so a live classroom demo never stalls, while still being a real
convolutional network rather than a toy linear model.
"""

from __future__ import annotations

from collections import OrderedDict
from typing import List

import numpy as np
import torch
import torch.nn as nn

IMAGE_SIZE = 64
NUM_CLASSES = 2
CLASS_NAMES = ["normal", "pneumonia"]


class PneumoniaCNN(nn.Module):
    """Three convolution blocks then a small classifier head.

    GroupNorm is used instead of BatchNorm on purpose. BatchNorm carries running
    mean/variance buffers that are computed on each hospital's own class
    distribution; averaging those across non-IID hospitals is known to degrade
    federated accuracy, and the integer ``num_batches_tracked`` buffer does not
    survive a weighted average cleanly. GroupNorm is stateless, so the parameter
    list Flower exchanges contains only learnable weights.
    """

    def __init__(self, num_classes: int = NUM_CLASSES) -> None:
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.GroupNorm(4, 16),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),  # 64 -> 32
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.GroupNorm(8, 32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),  # 32 -> 16
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.GroupNorm(8, 64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),  # 16 -> 8
        )
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Dropout(0.2),
            nn.Linear(64, num_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.pool(x)
        return self.classifier(x)

```

```
61
62      def build_model(seed: int | None = None) -> PneumoniaCNN:
63          """Create a model with a deterministic initialisation.
64
65          Every hospital must start from identical weights, otherwise the first round
66          of averaging mixes unrelated random networks.
67          """
68          if seed is not None:
69              torch.manual_seed(seed)
70          return PneumoniaCNN()
71
72
73      def get_weights(model: nn.Module) -> List[np.ndarray]:
74          """Flower speaks lists of NumPy arrays, not torch state dicts."""
75          return [value.detach().cpu().numpy() for value in model.state_dict().values()]
76
77
78      def set_weights(model: nn.Module, weights: List[np.ndarray]) -> None:
79          """Load a Flower parameter list back into a torch model."""
80          keys = list(model.state_dict().keys())
81          if len(keys) != len(weights):
82              raise ValueError(
83                  f"parameter count mismatch: model has {len(keys)}, received {len(weights)}"
84              )
85          state_dict = OrderedDict(
86              (key, torch.tensor(np.asarray(value))) for key, value in zip(keys, weights)
87          )
88          model.load_state_dict(state_dict, strict=True)
89
90
91      def count_parameters(model: nn.Module) -> int:
92          return sum(p.numel() for p in model.parameters())
93
94
95      __all__ = [
96          "CLASS_NAMES",
97          "IMAGE_SIZE",
98          "NUM_CLASSES",
99          "PneumoniaCNN",
100         "build_model",
101         "count_parameters",
102         "get_weights",
103         "set_weights",
104     ]
105
```

common/fedmed/metrics.py

```
"""Per-round metric sinks: a local JSON file, DynamoDB, and CloudWatch.

The dashboard reads whatever the sink wrote, so the same React app works against
a docker compose run and against the deployed API.
"""

from __future__ import annotations

import json
import logging
import os
import time
from abc import ABC, abstractmethod
from decimal import Decimal
from typing import Any, Dict, List, Optional

from .config import StorageConfig

LOGGER = logging.getLogger(__name__)


def _to_dynamo(value: Any) -> Any:
    """DynamoDB rejects Python floats, so numbers go in as Decimal."""
    if isinstance(value, bool):
        return value
    if isinstance(value, float):
        return Decimal(str(round(value, 6)))
    if isinstance(value, dict):
        return {key: _to_dynamo(item) for key, item in value.items()}
    if isinstance(value, (list, tuple)):
        return [_to_dynamo(item) for item in value]
    return value


class MetricsSink(ABC):
    """Record what happened in one federated round."""

    @abstractmethod
    def record_round(self, run_id: str, server_round: int, payload: Dict[str, Any]) -> None:
        ...

    def close(self) -> None:  # pragma: no cover - default is a no-op
        return None


class LocalMetrics(MetricsSink):
    """Append rounds to a JSON file the dashboard can load directly."""

    def __init__(self, path: str) -> None:
        self.path = path
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        self.rounds: List[Dict[str, Any]] = []

    def record_round(self, run_id: str, server_round: int, payload: Dict[str, Any]) -> None:
        entry = {"run_id": run_id, "round": server_round, "timestamp": int(time.time())}
        entry.update(payload)
        self.rounds.append(entry)
        with open(self.path, "w", encoding="utf-8") as handle:
            json.dump({"run_id": run_id, "rounds": self.rounds}, handle, indent=2)
        LOGGER.info("round %s metrics written to %s", server_round, self.path)
```

```

class AwsMetrics(MetricsSink):
    """Write one item per round to DynamoDB plus a custom CloudWatch metric."""

    def __init__(
        self,
        table_name: str,
        region: str,
        namespace: str = "FedMed",
        push_cloudwatch: bool = True,
    ) -> None:
        import boto3  # lazy import keeps local runs SDK-free

        self.table = boto3.resource("dynamodb", region_name=region).Table(table_name)
        self.namespace = namespace
        self.cloudwatch = (
            boto3.client("cloudwatch", region_name=region) if push_cloudwatch else None
        )

    def record_round(self, run_id: str, server_round: int, payload: Dict[str, Any]) -> None:
        item = {"run_id": run_id, "round": server_round, "timestamp": int(time.time())}
        item.update(payload)
        self.table.put_item(Item=_to_dynamo(item))

        if self.cloudwatch is None:
            return
        metric_data = [
            {
                "MetricName": name,
                "Dimensions": [{"Name": "RunId", "Value": run_id}],
                "Value": float(value),
                "Unit": "None",
            }
            for name, value in payload.items()
            if isinstance(value, (int, float)) and not isinstance(value, bool)
        ]
        if metric_data:
            self.cloudwatch.put_metric_data(
                Namespace=self.namespace, MetricData=metric_data[:20]
            )


class NullMetrics(MetricsSink):
    """Used when a sink fails to start. Training must never die over telemetry."""

    def record_round(self, run_id: str, server_round: int, payload: Dict[str, Any]) -> None:
        LOGGER.warning("metrics sink unavailable, dropping round %s", server_round)


def build_metrics_sink(config: Optional[StorageConfig] = None) -> MetricsSink:
    config = config or StorageConfig()
    if config.backend.lower() == "s3":
        try:
            return AwsMetrics(
                table_name=config.rounds_table,
                region=config.aws_region,
                namespace=config.cloudwatch_namespace,
                push_cloudwatch=config.push_cloudwatch,
            )
        except Exception as error:  # pragma: no cover - depends on live AWS
            LOGGER.error("could not create the AWS metrics sink: %s", error)
            return NullMetrics()
```

```
        return LocalMetrics(os.path.join(config.local_root, "metrics.json"))


__all__ = ["AwsMetrics", "LocalMetrics", "MetricsSink", "NullMetrics", "build_metrics_sink"]

```

common/fedmed/privacy.py

```python
"""Differential-privacy accounting for the report and the dashboard.

Flower applies the mechanism itself: clip every client update to a fixed L2 norm,
then add Gaussian noise to the aggregate. What it does not give you is the
privacy budget, so this module converts a noise multiplier and a round count into
an epsilon at a chosen delta using Renyi differential privacy.
"""

from __future__ import annotations

import math
from dataclasses import asdict, dataclass
from typing import Dict, List, Optional

# The RDP order grid used by the Opacus and TF-Privacy accountants.
_INTEGER_ORDERS: List[int] = list(range(2, 64)) + [128, 256, 512]


@dataclass
class PrivacyReport:
    """A privacy budget statement that can be quoted directly in the report."""

    mechanism: str
    noise_multiplier: float
    clipping_norm: float
    sampling_rate: float
    rounds: int
    delta: float
    epsilon: Optional[float]

    def as_dict(self) -> Dict[str, object]:
        return asdict(self)


def _log_add(logx: float, logy: float) -> float:
    """Numerically stable log(exp(logx) + exp(logy))."""
    if logx == -math.inf:
        return logy
    if logy == -math.inf:
        return logx
    lower, higher = min(logx, logy), max(logx, logy)
    return higher + math.log1p(math.exp(lower - higher))


def _log_comb(n: int, k: int) -> float:
    return math.lgamma(n + 1) - math.lgamma(k + 1) - math.lgamma(n - k + 1)


def _rdp_sampled_gaussian(q: float, noise_multiplier: float, alpha: int) -> float:
    """RDP of the sampled Gaussian mechanism at an integer order.

    Binomial expansion from Mironov, Talwar and Zhang, 2019.
    """
    if q >= 1.0:
        # Full participation: the plain Gaussian mechanism.
        return alpha / (2.0 * noise_multiplier**2)

    log_terms = -math.inf
    for i in range(alpha + 1):
        log_term = (
```

```
                _log_comb(alpha, i)
                + i * math.log(q)
                + (alpha - i) * math.log1p(-q)
                + (i * i - i) / (2.0 * noise_multiplier**2)
            )
            log_terms = _log_add(log_terms, log_term)
        return float(log_terms) / (alpha - 1)


    def compute_epsilon(
        noise_multiplier: float,
        rounds: int,
        sampling_rate: float = 1.0,
        delta: float = 1e-5,
    ) -> Optional[float]:
        """Convert a noise multiplier and round count into an (epsilon, delta) budget.

        Returns ``None`` when noise is switched off, because no finite budget exists.
        """
        if noise_multiplier <= 0 or rounds <= 0:
            return None
        sampling_rate = min(max(sampling_rate, 1e-12), 1.0)

        best = math.inf
        for alpha in _INTEGER_ORDERS:
            rdp = rounds * _rdp_sampled_gaussian(sampling_rate, noise_multiplier, alpha)
            # Canonne, Kamath and Steinke, 2020: a tight RDP to (eps, delta) conversion.
            epsilon = (
                rdp
                + math.log1p(-1.0 / alpha)
                - (math.log(delta) + math.log(alpha)) / (alpha - 1)
            )
            best = min(best, epsilon)
        if not math.isfinite(best):
            return None
        return max(best, 0.0)


    def build_report(
        noise_multiplier: float,
        clipping_norm: float,
        rounds: int,
        num_clients: int,
        clients_per_round: Optional[int] = None,
        delta: float = 1e-5,
        enabled: bool = True,
    ) -> PrivacyReport:
        """Assemble the privacy statement shown on the dashboard."""
        if not enabled or noise_multiplier <= 0:
            return PrivacyReport(
                mechanism="none",
                noise_multiplier=0.0,
                clipping_norm=clipping_norm,
                sampling_rate=1.0,
                rounds=rounds,
                delta=delta,
                epsilon=None,
            )
        sampled = clients_per_round or num_clients
        sampling_rate = sampled / max(num_clients, 1)
        return PrivacyReport(
            mechanism="server-side fixed clipping with Gaussian noise",
            noise_multiplier=noise_multiplier,
```

```
            clipping_norm=clipping_norm,
            sampling_rate=sampling_rate,
            rounds=rounds,
            delta=delta,
            epsilon=compute_epsilon(noise_multiplier, rounds, sampling_rate, delta),
        )


    __all__ = ["PrivacyReport", "build_report", "compute_epsilon"]

```

common/fedmed/storage.py

```
    """Model checkpoint storage with interchangeable local and S3 backends.

    The same code path runs under docker compose (writes to a mounted volume) and on
    Fargate (writes to a versioned S3 bucket). Switching is a single environment
    variable, ``STORAGE_BACKEND``.
    """

    from __future__ import annotations

    import io
    import json
    import logging
    import os
    from abc import ABC, abstractmethod
    from typing import Any, Dict, List, Optional

    import numpy as np

    from .config import StorageConfig

    LOGGER = logging.getLogger(__name__)

    LATEST_KEY = "global/latest.npz"
    SUMMARY_KEY = "global/summary.json"


    def _serialise(weights: List[np.ndarray], metadata: Dict[str, Any]) -> bytes:
        """Pack a parameter list plus metadata into one compressed ``.npz`` blob."""
        buffer = io.BytesIO()
        arrays = {f"arr_{index}": array for index, array in enumerate(weights)}
        arrays["__metadata__"] = np.frombuffer(
            json.dumps(metadata).encode("utf-8"), dtype=np.uint8
        )
        np.savez_compressed(buffer, **arrays)
        return buffer.getvalue()


    def _deserialise(blob: bytes) -> tuple[List[np.ndarray], Dict[str, Any]]:
        with np.load(io.BytesIO(blob), allow_pickle=False) as archive:
            metadata: Dict[str, Any] = {}
            if "__metadata__" in archive.files:
                metadata = json.loads(bytes(archive["__metadata__"]).decode("utf-8"))
            indices = sorted(
                (int(name.split("_")[1]) for name in archive.files if name.startswith("arr_")),
            )
            weights = [np.array(archive[f"arr_{index}"]) for index in indices]
        return weights, metadata


    class Storage(ABC):
        """Persist and retrieve global model checkpoints."""

        @abstractmethod
        def save_checkpoint(
            self, server_round: int, weights: List[np.ndarray], metadata: Dict[str, Any]
        ) -> str:
            """Write one round's global model and return a human-readable location."""

        @abstractmethod
        def load_latest(self) -> tuple[List[np.ndarray], Dict[str, Any]]:
```

```
            """Read back the most recent global model."""

        @abstractmethod
        def write_json(self, key: str, payload: Dict[str, Any]) -> str:
            """Write a small JSON document (run summaries, partition manifests)."""

        @abstractmethod
        def read_bytes(self, key: str) -> bytes:
            """Read an arbitrary object, used to fetch a hospital's data shard."""


    class LocalStorage(Storage):
        """Filesystem backend used by docker compose and the unit tests."""

        def __init__(self, root: str) -> None:
            self.root = root
            try:
                os.makedirs(os.path.join(root, "global"), exist_ok=True)
            except OSError as error:
                # The inference API mounts the artifacts volume read-only, because
                # it has no business writing checkpoints. Failing to create a
                # directory it will never write to must not stop it from serving.
                LOGGER.debug("could not create %s: %s", root, error)

        def _path(self, key: str) -> str:
            path = os.path.join(self.root, key)
            os.makedirs(os.path.dirname(path), exist_ok=True)
            return path

        def save_checkpoint(
            self, server_round: int, weights: List[np.ndarray], metadata: Dict[str, Any]
        ) -> str:
            blob = _serialise(weights, metadata)
            round_key = f"global/round_{server_round:03d}.npz"
            for key in (round_key, LATEST_KEY):
                with open(self._path(key), "wb") as handle:
                    handle.write(blob)
            return os.path.join(self.root, round_key)

        def load_latest(self) -> tuple[List[np.ndarray], Dict[str, Any]]:
            with open(self._path(LATEST_KEY), "rb") as handle:
                return _deserialise(handle.read())

        def write_json(self, key: str, payload: Dict[str, Any]) -> str:
            path = self._path(key)
            with open(path, "w", encoding="utf-8") as handle:
                json.dump(payload, handle, indent=2)
            return path

        def read_bytes(self, key: str) -> bytes:
            with open(self._path(key), "rb") as handle:
                return handle.read()


    class S3Storage(Storage):
        """S3 backend. Bucket versioning gives a free audit trail of every round."""

        def __init__(self, bucket: str, region: str, prefix: str = "") -> None:
            if not bucket:
                raise ValueError("S3Storage requires a bucket name (set MODELS_BUCKET)")
            import boto3  # imported lazily so local runs need no AWS SDK

            self.bucket = bucket
```

```
            self.prefix = prefix.strip("/")
            self.client = boto3.client("s3", region_name=region)

        def _key(self, key: str) -> str:
            return f"{self.prefix}/{key}" if self.prefix else key

        def save_checkpoint(
            self, server_round: int, weights: List[np.ndarray], metadata: Dict[str, Any]
        ) -> str:
            blob = _serialise(weights, metadata)
            round_key = f"global/round_{server_round:03d}.npz"
            for key in (round_key, LATEST_KEY):
                self.client.put_object(
                    Bucket=self.bucket,
                    Key=self._key(key),
                    Body=blob,
                    ContentType="application/octet-stream",
                    ServerSideEncryption="AES256",
                )
            return f"s3://{self.bucket}/{self._key(round_key)}"

        def load_latest(self) -> tuple[List[np.ndarray], Dict[str, Any]]:
            response = self.client.get_object(Bucket=self.bucket, Key=self._key(LATEST_KEY))
            return _deserialise(response["Body"].read())

        def write_json(self, key: str, payload: Dict[str, Any]) -> str:
            self.client.put_object(
                Bucket=self.bucket,
                Key=self._key(key),
                Body=json.dumps(payload, indent=2).encode("utf-8"),
                ContentType="application/json",
                ServerSideEncryption="AES256",
            )
            return f"s3://{self.bucket}/{self._key(key)}"

        def read_bytes(self, key: str) -> bytes:
            response = self.client.get_object(Bucket=self.bucket, Key=self._key(key))
            return response["Body"].read()


    def build_storage(config: Optional[StorageConfig] = None, bucket: Optional[str] = None) -> Storage:
        """Pick a backend from configuration.

        ``bucket`` overrides the models bucket, which is how a hospital node points
        the same class at its own private data bucket.
        """
        config = config or StorageConfig()
        if config.backend.lower() == "s3":
            return S3Storage(bucket or config.models_bucket, config.aws_region)
        root = config.local_root
        LOGGER.info("Using local storage at %s", root)
        return LocalStorage(root)


    __all__ = [
        "LATEST_KEY",
        "LocalStorage",
        "S3Storage",
        "Storage",
        "SUMMARY_KEY",
        "build_storage",
    ]

```

hospital_node/Dockerfile

```
# Hospital node image.
#
# One image, three running copies, distinguished only by HOSPITAL_ID. No dataset
# is baked in: a node fetches its own shard at startup from its own bucket, so
# the image itself carries no patient data and is safe to share between sites.
#
#     docker build -f hospital_node/Dockerfile -t fedmed-hospital .
FROM python:3.11-slim

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

COPY hospital_node/requirements.txt ./requirements.txt
RUN pip install --no-cache-dir \
        --index-url https://download.pytorch.org/whl/cpu \
        --extra-index-url https://pypi.org/simple \
        -r requirements.txt

COPY common /app/common
RUN pip install --no-cache-dir /app/common

COPY hospital_node/client.py /app/

# Shards land here at runtime: a mounted volume locally, ephemeral task storage
# on Fargate. Nothing under this path survives the container.
RUN mkdir -p /workspace/data

RUN useradd --create-home --uid 10001 fedmed && chown -R fedmed /app /workspace
USER fedmed

CMD ["python", "/app/client.py"]

```

hospital_node/requirements.txt

```
numpy==1.26.4
torch==2.5.1+cpu
flwr==1.15.2
boto3==1.35.68

```

hospital_node/client.py

```
"""A hospital node.

Three copies of this container run in the federation, distinguished only by
``HOSPITAL_ID``. Each one reads a single shard that lives in its own private
bucket, trains on it locally, and returns weights plus scalar metrics.

Nothing in this file has a code path that sends pixels anywhere. The gRPC
messages Flower emits contain the model parameter list and the metrics dictionary
built in ``fit`` and ``evaluate``, and that is the whole outbound surface.
"""

from __future__ import annotations

import logging
import os
import sys
import time
from typing import Dict, List, Tuple

import flwr as fl
import numpy as np
from flwr.common import Config, NDArrays, Scalar

from fedmed import data as data_utils
from fedmed.config import FederationConfig, StorageConfig, TLSConfig, TrainingConfig
from fedmed.model import build_model, get_weights, set_weights
from fedmed.storage import build_storage

HOSPITAL_ID = os.environ.get("HOSPITAL_ID", "unknown")

logging.basicConfig(
    level=os.environ.get("LOG_LEVEL", "INFO"),
    format=f"%(asctime)s %(levelname)s [hospital-{HOSPITAL_ID}] %(message)s",
)
LOGGER = logging.getLogger("fedmed.hospital")

VALIDATION_FRACTION = 0.15


def fetch_shard(storage_config: StorageConfig) -> str:
    """Return a local path to this hospital's shard, downloading it if needed.

    On Fargate the shard is pulled once from the hospital's own S3 bucket into
    the task's ephemeral storage, which disappears when the task stops.
    """
    local_path = os.environ.get("SHARD_PATH", f"/workspace/data/hospital_{HOSPITAL_ID}/shard.npz")
    if storage_config.backend.lower() != "s3":
        return local_path

    key = os.environ.get("SHARD_KEY", "shard.npz")
    storage = build_storage(storage_config, bucket=storage_config.data_bucket)
    LOGGER.info("downloading %s from s3://%s", key, storage_config.data_bucket)
    blob = storage.read_bytes(key)
    os.makedirs(os.path.dirname(local_path), exist_ok=True)
    with open(local_path, "wb") as handle:
        handle.write(blob)
    LOGGER.info("shard cached at %s (%.1f MB)", local_path, len(blob) / 1e6)
    return local_path


```

```
def split_train_validation(
    shard: data_utils.Shard, seed: int
) -> Tuple[data_utils.Shard, data_utils.Shard]:
    """Hold back a slice of the local data so the hospital can score the global model.

    This is what makes the federated evaluation round meaningful: each hospital
    judges the incoming global model on patients the model has never trained on.
    """
    rng = np.random.default_rng(seed)
    order = rng.permutation(len(shard))
    cut = max(int(len(shard) * (1 - VALIDATION_FRACTION)), 1)
    train_index, validation_index = order[:cut], order[cut:]
    if validation_index.size == 0:  # tiny shard, reuse the tail of training
        validation_index = train_index[-1:]
    return (
        data_utils.Shard(shard.images[train_index], shard.labels[train_index], "train"),
        data_utils.Shard(shard.images[validation_index], shard.labels[validation_index], "val"),
    )


class HospitalClient(fl.client.NumPyClient):
    """Implements the three calls Flower makes on a participant."""

    def __init__(
        self,
        train_shard: data_utils.Shard,
        validation_shard: data_utils.Shard,
        training: TrainingConfig,
    ) -> None:
        self.train_shard = train_shard
        self.validation_shard = validation_shard
        self.training = training
        self.model = build_model(seed=training.seed)

    def get_properties(self, config: Config) -> Dict[str, Scalar]:
        return {"hospital_id": HOSPITAL_ID, "examples": len(self.train_shard)}

    def get_parameters(self, config: Config) -> NDArrays:
        return get_weights(self.model)

    def fit(
        self, parameters: NDArrays, config: Config
    ) -> Tuple[NDArrays, int, Dict[str, Scalar]]:
        """Start from the global model, train locally, hand back new weights."""
        set_weights(self.model, list(parameters))
        started_at = time.time()
        stats = data_utils.train_local(
            self.model,
            self.train_shard,
            epochs=self.training.local_epochs,
            batch_size=self.training.batch_size,
            learning_rate=self.training.learning_rate,
        )
        elapsed = time.time() - started_at
        LOGGER.info(
            "local training done: loss %.4f accuracy %.4f on %s images in %.1fs",
            stats["train_loss"],
            stats["train_accuracy"],
            len(self.train_shard),
            elapsed,
        )
        metrics: Dict[str, Scalar] = {
            "hospital_id": HOSPITAL_ID,
```

```
                "train_loss": stats["train_loss"],
                "train_accuracy": stats["train_accuracy"],
                "local_seconds": elapsed,
            }
            return get_weights(self.model), len(self.train_shard), metrics

        def evaluate(
            self, parameters: NDArrays, config: Config
        ) -> Tuple[float, int, Dict[str, Scalar]]:
            """Score the incoming global model on this hospital's held-out patients."""
            set_weights(self.model, list(parameters))
            results = data_utils.evaluate(self.model, self.validation_shard)
            LOGGER.info(
                "global model on local validation: accuracy %.4f loss %.4f",
                results["accuracy"],
                results["loss"],
            )
            return (
                results["loss"],
                len(self.validation_shard),
                {"hospital_id": HOSPITAL_ID, "accuracy": results["accuracy"]},
            )


    def connect_with_retry(
        client: fl.client.Client, address: str, root_certificates: bytes | None
    ) -> None:
        """Retry the initial connection.

        Fargate starts all four tasks at once, so a hospital routinely comes up
        before the aggregator is listening. Without this the demo fails on a race.
        """
        attempts = int(os.environ.get("CONNECT_ATTEMPTS", "30"))
        delay = float(os.environ.get("CONNECT_DELAY_SECONDS", "5"))
        for attempt in range(1, attempts + 1):
            try:
                LOGGER.info("connecting to %s (attempt %s/%s)", address, attempt, attempts)
                fl.client.start_client(
                    server_address=address,
                    client=client,
                    root_certificates=root_certificates,
                )
                return
            except Exception as error:  # noqa: BLE001 - any transport error is worth a retry
                if attempt == attempts:
                    raise
                LOGGER.warning("server not ready (%s); retrying in %ss", error, delay)
                time.sleep(delay)


    def main() -> int:
        federation = FederationConfig()
        training = TrainingConfig()
        storage_config = StorageConfig()
        tls = TLSConfig()

        shard_path = fetch_shard(storage_config)
        shard = data_utils.load_shard(shard_path, name=f"hospital-{HOSPITAL_ID}")
        train_shard, validation_shard = split_train_validation(shard, training.seed)
        LOGGER.info(
            "local dataset: %s images (%s train / %s validation), class balance %s",
            len(shard),
            len(train_shard),
```

```
            len(validation_shard),
            shard.class_counts,
        )

        root_certificates = None
        if tls.enabled:
            with open(tls.ca_path, "rb") as handle:
                root_certificates = handle.read()
            LOGGER.info("TLS enabled, pinning CA at %s", tls.ca_path)

        client = HospitalClient(train_shard, validation_shard, training).to_client()
        connect_with_retry(client, federation.server_address, root_certificates)
        LOGGER.info("federation finished, shutting down; local data was never transmitted")
        return 0


    if __name__ == "__main__":
        sys.exit(main())

```

fl_server/Dockerfile

```
# Aggregation server image.
#
# Build from the repository root so the shared `common/` package is in context:
#     docker build -f fl_server/Dockerfile -t fedmed-server .
FROM python:3.11-slim

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

# The CPU-only PyTorch wheels are roughly a tenth the size of the default CUDA
# ones, which matters for Fargate cold starts and for ECR storage cost.
COPY fl_server/requirements.txt ./requirements.txt
RUN pip install --no-cache-dir \
        --index-url https://download.pytorch.org/whl/cpu \
        --extra-index-url https://pypi.org/simple \
        -r requirements.txt

COPY common /app/common
RUN pip install --no-cache-dir /app/common

COPY fl_server/server.py fl_server/strategy.py /app/

# The global test split is not private hospital data, so it ships in the image.
# That keeps the aggregator free of any read permission on a hospital bucket,
# which is the property the whole design rests on.
COPY data/server /workspace/data/server

# Fargate maps this port through the task ENI; hospitals reach it via Cloud Map.
EXPOSE 8080

# Refuse to run as root, so a compromised aggregator cannot rewrite its own image.
RUN useradd --create-home --uid 10001 fedmed && chown -R fedmed /app /workspace
USER fedmed

CMD ["python", "/app/server.py"]

```

fl_server/requirements.txt

```
numpy==1.26.4
torch==2.5.1+cpu
flwr==1.15.2
boto3==1.35.68

```

fl_server/server.py

```
"""FedMed aggregation server.

Runs as a single Fargate task. It never reads a hospital's data bucket: the only
dataset it touches is the global test split that ships inside its own image, and
the only thing it receives from hospitals is a list of weight arrays.

The process exits as soon as the configured number of rounds completes, which is
what keeps a demo run down to a few cents of Fargate time.
"""

from __future__ import annotations

import argparse
import json
import logging
import os
import sys
import time
from typing import Any, Dict, List, Optional

import flwr as fl
from flwr.common import ndarrays_to_parameters
from flwr.server import ServerConfig

from fedmed import data as data_utils
from fedmed import privacy as privacy_utils
from fedmed.config import FederationConfig, PrivacyConfig, StorageConfig, TLSConfig
from fedmed.metrics import build_metrics_sink
from fedmed.model import build_model, count_parameters, get_weights
from fedmed.storage import build_storage

from strategy import RoundRecorder, build_strategy, make_evaluate_fn

logging.basicConfig(
    level=os.environ.get("LOG_LEVEL", "INFO"),
    format="%(asctime)s %(levelname)s [server] %(message)s",
)
LOGGER = logging.getLogger("fedmed.server")

DEFAULT_TEST_SHARD = "/workspace/data/server/test.npz"


def load_certificates(tls: TLSConfig) -> Optional[tuple]:
    """Read the self-signed CA, server certificate and key for the gRPC channel."""
    if not tls.enabled:
        LOGGER.warning("TLS is DISABLED; the gRPC channel is in plaintext")
        return None
    paths = [tls.ca_path, tls.cert_path, tls.key_path]
    missing = [path for path in paths if not os.path.exists(path)]
    if missing:
        raise FileNotFoundError(
            "TLS_ENABLED is set but these files are missing: " + ", ".join(missing)
        )
    LOGGER.info("TLS enabled using %s", tls.cert_path)
    return tuple(open(path, "rb").read() for path in paths)


def main() -> int:
    parser = argparse.ArgumentParser(description="FedMed federated aggregation server")
    parser.add_argument(
```

```
            "--test-shard",
            default=os.environ.get("TEST_SHARD", DEFAULT_TEST_SHARD),
            help="Path to the global held-out test set used for centralised evaluation",
        )
        args = parser.parse_args()

        federation = FederationConfig()
        privacy_config = PrivacyConfig()
        storage_config = StorageConfig()
        tls = TLSConfig()

        run_id = federation.run_id or f"run-{int(time.time())}"
        LOGGER.info("run_id=%s rounds=%s hospitals=%s", run_id, federation.num_rounds, federation.num_hospitals)

        test_shard = data_utils.load_shard(args.test_shard, name="server-test")
        LOGGER.info(
            "global test set: %s images, class balance %s",
            len(test_shard),
            test_shard.class_counts,
        )

        model = build_model(seed=42)
        LOGGER.info("model has %s trainable parameters", f"{count_parameters(model):,}")
        initial_parameters = ndarrays_to_parameters(get_weights(model))

        storage = build_storage(storage_config)
        sink = build_metrics_sink(storage_config)
        recorder = RoundRecorder()
        history: List[Dict[str, Any]] = []

        privacy_report = privacy_utils.build_report(
            noise_multiplier=privacy_config.noise_multiplier,
            clipping_norm=privacy_config.clipping_norm,
            rounds=federation.num_rounds,
            num_clients=federation.num_hospitals,
            delta=privacy_config.delta,
            enabled=privacy_config.enabled,
        )
        LOGGER.info("privacy budget: %s", json.dumps(privacy_report.as_dict()))

        evaluate_fn = make_evaluate_fn(
            test_shard=test_shard,
            storage=storage,
            sink=sink,
            recorder=recorder,
            run_id=run_id,
            privacy=privacy_report.as_dict(),
            history=history,
        )

        strategy = build_strategy(
            initial_parameters=initial_parameters,
            evaluate_fn=evaluate_fn,
            recorder=recorder,
            num_hospitals=federation.num_hospitals,
            dp_enabled=privacy_config.enabled,
            noise_multiplier=privacy_config.noise_multiplier,
            clipping_norm=privacy_config.clipping_norm,
        )

        started_at = time.time()
        LOGGER.info("waiting for %s hospitals on %s", federation.num_hospitals, federation.bind_address)
        fl.server.start_server(
```

```
            server_address=federation.bind_address,
            config=ServerConfig(num_rounds=federation.num_rounds),
            strategy=strategy,
            certificates=load_certificates(tls),
        )
        elapsed = round(time.time() - started_at, 2)

        summary = {
            "run_id": run_id,
            "rounds": federation.num_rounds,
            "hospitals": federation.num_hospitals,
            "wall_seconds": elapsed,
            "privacy": privacy_report.as_dict(),
            "final_accuracy": history[-1]["accuracy"] if history else None,
            "best_accuracy": max((entry["accuracy"] for entry in history), default=None),
            "history": history,
            "finished_at": int(time.time()),
        }
        location = storage.write_json("global/summary.json", summary)
        LOGGER.info("run finished in %ss, summary at %s", elapsed, location)
        LOGGER.info("final accuracy: %s", summary["final_accuracy"])
        return 0


    if __name__ == "__main__":
        sys.exit(main())

```

fl_server/strategy.py

```
    """The aggregation strategy: FedAvg, optional differential privacy, persistence.

    Two things are worth knowing about the ordering here.

    First, the per-round checkpoint and the metrics record are written from the
    server-side ``evaluate_fn`` rather than from ``aggregate_fit``. Flower calls the
    strategy's ``evaluate`` with the parameters that come *out* of aggregation, so
    when differential privacy is switched on this is the only hook that sees the
    noised model. Saving from inside ``aggregate_fit`` would persist the clean
    average and quietly defeat the privacy layer.

    Second, ``DifferentialPrivacyServerSideFixedClipping`` wraps a strategy rather
    than replacing it, so FedAvg keeps doing the weighted averaging and the wrapper
    only adds clipping and noise around it.
    """

    from __future__ import annotations

    import logging
    import time
    from typing import Any, Callable, Dict, List, Optional, Tuple

    import numpy as np
    from flwr.common import Metrics, NDArrays, Scalar
    from flwr.server.strategy import FedAvg, Strategy

    from fedmed import data as data_utils
    from fedmed.metrics import MetricsSink
    from fedmed.model import build_model, set_weights
    from fedmed.storage import Storage

    LOGGER = logging.getLogger(__name__)


    class RoundRecorder:
        """Collects what each hospital reported during the current round.

        Flower hands client training metrics to ``fit_metrics_aggregation_fn`` and
        then throws them away. The dashboard wants them per hospital, so they are
        parked here until ``evaluate_fn`` assembles the round record.
        """

        def __init__(self) -> None:
            self.hospitals: List[Dict[str, Any]] = []
            self.round_started_at: float = time.time()

        def reset(self) -> None:
            self.hospitals = []
            self.round_started_at = time.time()

        def capture(self, results: List[Tuple[int, Metrics]]) -> Metrics:
            """Store per-hospital metrics and return the sample-weighted average."""
            self.hospitals = [
                {
                    "hospital_id": str(metrics.get("hospital_id", f"unknown-{index}")),
                    "examples": int(num_examples),
                    "train_loss": float(metrics.get("train_loss", 0.0)),
                    "train_accuracy": float(metrics.get("train_accuracy", 0.0)),
                    "local_seconds": float(metrics.get("local_seconds", 0.0)),
                }
```

```
                for index, (num_examples, metrics) in enumerate(results)
            ]
            total = sum(item["examples"] for item in self.hospitals) or 1
            return {
                "train_loss": sum(i["train_loss"] * i["examples"] for i in self.hospitals) / total,
                "train_accuracy": sum(
                    i["train_accuracy"] * i["examples"] for i in self.hospitals
                )
                / total,
                "participating_hospitals": len(self.hospitals),
                "total_examples": total,
            }


    def make_evaluate_fn(
        test_shard: data_utils.Shard,
        storage: Storage,
        sink: MetricsSink,
        recorder: RoundRecorder,
        run_id: str,
        privacy: Dict[str, Any],
        history: List[Dict[str, Any]],
    ) -> Callable[[int, NDArrays, Dict[str, Scalar]], Optional[Tuple[float, Dict[str, Scalar]]]]:
        """Build the centralised evaluation hook.

        The server holds a global test set that belongs to no hospital, which is what
        makes the round-by-round accuracy curve comparable across runs.
        """

        model = build_model()

        def evaluate_fn(
            server_round: int, parameters: NDArrays, config: Dict[str, Scalar]
        ) -> Optional[Tuple[float, Dict[str, Scalar]]]:
            set_weights(model, list(parameters))
            results = data_utils.evaluate(model, test_shard)

            if server_round == 0:
                # Round 0 is the evaluation of the initial random model. It is a
                # useful baseline for the chart but there is nothing to checkpoint.
                LOGGER.info("initial model accuracy %.4f", results["accuracy"])
                return results["loss"], {"accuracy": results["accuracy"]}

            metadata = {
                "run_id": run_id,
                "round": server_round,
                "accuracy": results["accuracy"],
                "loss": results["loss"],
                "privacy": privacy,
                "saved_at": int(time.time()),
            }
            location = storage.save_checkpoint(server_round, list(parameters), metadata)

            record: Dict[str, Any] = {
                "accuracy": results["accuracy"],
                "loss": results["loss"],
                "recall_normal": results["recall_normal"],
                "recall_pneumonia": results["recall_pneumonia"],
                "test_examples": int(results["examples"]),
                "round_seconds": round(time.time() - recorder.round_started_at, 2),
                "hospitals": recorder.hospitals,
                "checkpoint": location,
                "dp_enabled": bool(privacy.get("mechanism") != "none"),
```

```
            }
            sink.record_round(run_id, server_round, record)
            history.append({"round": server_round, **record})
            recorder.reset()

            LOGGER.info(
                "round %s | accuracy %.4f | loss %.4f | checkpoint %s",
                server_round,
                results["accuracy"],
                results["loss"],
                location,
            )
            return results["loss"], {"accuracy": results["accuracy"]}

        return evaluate_fn


    def build_strategy(
        initial_parameters,
        evaluate_fn,
        recorder: RoundRecorder,
        num_hospitals: int,
        dp_enabled: bool,
        noise_multiplier: float,
        clipping_norm: float,
    ) -> Strategy:
        """FedAvg over every hospital, optionally wrapped in the DP mechanism."""

        strategy: Strategy = FedAvg(
            fraction_fit=1.0,
            fraction_evaluate=1.0,
            min_fit_clients=num_hospitals,
            min_evaluate_clients=num_hospitals,
            min_available_clients=num_hospitals,
            initial_parameters=initial_parameters,
            evaluate_fn=evaluate_fn,
            fit_metrics_aggregation_fn=recorder.capture,
            evaluate_metrics_aggregation_fn=_weighted_average,
        )

        if not dp_enabled or noise_multiplier <= 0:
            LOGGER.warning("differential privacy is DISABLED for this run")
            return strategy

        from flwr.server.strategy import DifferentialPrivacyServerSideFixedClipping

        LOGGER.info(
            "differential privacy on: clip=%.3f noise_multiplier=%.4f",
            clipping_norm,
            noise_multiplier,
        )
        return DifferentialPrivacyServerSideFixedClipping(
            strategy,
            noise_multiplier=noise_multiplier,
            clipping_norm=clipping_norm,
            num_sampled_clients=num_hospitals,
        )


    def _weighted_average(results: List[Tuple[int, Metrics]]) -> Metrics:
        """Sample-weighted mean of the hospitals' local evaluation metrics."""
        total = sum(num_examples for num_examples, _ in results) or 1
        accuracy = sum(
```

```
            float(metrics.get("accuracy", 0.0)) * num_examples for num_examples, metrics in results
        )
        return {"accuracy": accuracy / total}


def parameters_l2_norm(parameters: NDArrays) -> float:
    """Handy for the report: how large is the update the server just averaged."""
    return float(np.sqrt(sum(float(np.sum(np.square(array))) for array in parameters)))


__all__ = [
    "RoundRecorder",
    "build_strategy",
    "make_evaluate_fn",
    "parameters_l2_norm",
]

```

inference_api/Dockerfile

```
# Inference API image, built on the AWS Lambda Python base so the same artifact
# runs as a Lambda function and as a local container.
#
#     docker build -f inference_api/Dockerfile -t fedmed-api .
#     docker run -p 9000:8080 fedmed-api          # Lambda runtime emulator
FROM public.ecr.aws/lambda/python:3.11

ENV PIP_NO_CACHE_DIR=1

COPY inference_api/requirements.txt ${LAMBDA_TASK_ROOT}/requirements.txt
RUN pip install --no-cache-dir \
        --index-url https://download.pytorch.org/whl/cpu \
        --extra-index-url https://pypi.org/simple \
        -r ${LAMBDA_TASK_ROOT}/requirements.txt

COPY common ${LAMBDA_TASK_ROOT}/common
RUN pip install --no-cache-dir ${LAMBDA_TASK_ROOT}/common

COPY inference_api/app.py ${LAMBDA_TASK_ROOT}/app.py

# Mangum translates the API Gateway payload into an ASGI scope for FastAPI.
CMD ["app.handler"]

```

inference_api/Dockerfile.local

```
# Local development image for the inference API.
#
# The production image builds on the Lambda base and is invoked through Mangum.
# For docker compose we want a plain uvicorn server on port 8000 so the browser
# can talk to it directly, so the two Dockerfiles differ only in their entry
# point. The application code is identical.
FROM python:3.11-slim

ENV PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

COPY inference_api/requirements.txt ./requirements.txt
RUN pip install --no-cache-dir \
        --index-url https://download.pytorch.org/whl/cpu \
        --extra-index-url https://pypi.org/simple \
        -r requirements.txt \
    && pip install --no-cache-dir uvicorn==0.32.1

COPY common /app/common
RUN pip install --no-cache-dir /app/common

COPY inference_api/app.py /app/app.py

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

```

inference_api/requirements.txt

```
numpy==1.26.4
torch==2.5.1+cpu
fastapi==0.115.5
mangum==0.19.0
pillow==11.0.0
python-multipart==0.0.20
boto3==1.35.68

```

inference_api/app.py

```
"""Inference and reporting API for the federated model.

Deployed as a Lambda container image behind an HTTP API Gateway, and runnable as
a plain uvicorn app for local development. It is the only public entry point to
the platform.

Privacy behaviour worth stating in the report: an uploaded X-ray is decoded in
memory, classified, and discarded. Nothing writes the image, its filename, or any
pixel data to disk or to a bucket. The audit trail records a SHA-256 digest of
the bytes, which proves a given image was submitted if someone later presents the
same file, but cannot be turned back into the image.
"""

from __future__ import annotations

import hashlib
import io
import logging
import os
import time
from typing import Any, Dict, List, Optional

import numpy as np
import torch
from fastapi import FastAPI, File, HTTPException, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from PIL import Image

from fedmed import data as data_utils
from fedmed.config import StorageConfig
from fedmed.model import CLASS_NAMES, IMAGE_SIZE, build_model, set_weights
from fedmed.storage import build_storage

logging.basicConfig(level=os.environ.get("LOG_LEVEL", "INFO"))
LOGGER = logging.getLogger("fedmed.api")

MODEL_CACHE_SECONDS = int(os.environ.get("MODEL_CACHE_SECONDS", "60"))
MAX_UPLOAD_BYTES = int(os.environ.get("MAX_UPLOAD_BYTES", str(8 * 1024 * 1024)))
AUDIT_ENABLED = os.environ.get("AUDIT_ENABLED", "true").lower() in {"1", "true", "yes"}

app = FastAPI(
    title="FedMed Inference API",
    description="Serves the model produced by the federated training rounds",
    version="0.1.0",
)

# The dashboard is a static site on a different origin, so it needs CORS. The
# allowed origin list is configurable and defaults to the permissive value only
# because this is a teaching deployment with no patient data behind it.
app.add_middleware(
    CORSMiddleware,
    allow_origins=os.environ.get("CORS_ORIGINS", "*").split(","),
    allow_credentials=False,
    allow_methods=["GET", "POST", "OPTIONS"],
    allow_headers=["*"],
)


class ModelHolder:
    """Keeps one copy of the global model warm inside the Lambda container.
```

```

    Re-reading the checkpoint on every request would add hundreds of milliseconds
    and a needless S3 call, but caching forever would mean a finished training run
    never reaches the API. A short time-to-live gets both.
    """

    def __init__(self) -> None:
        self.model = None
        self.metadata: Dict[str, Any] = {}
        self.loaded_at: float = 0.0

    def refresh(self, force: bool = False) -> None:
        if not force and self.model is not None:
            if time.time() - self.loaded_at < MODEL_CACHE_SECONDS:
                return
        storage = build_storage(StorageConfig())
        weights, metadata = storage.load_latest()
        model = build_model()
        set_weights(model, weights)
        model.eval()
        self.model = model
        self.metadata = metadata
        self.loaded_at = time.time()
        LOGGER.info("loaded global model from round %s", metadata.get("round"))

    def get(self):
        self.refresh()
        if self.model is None:
            raise HTTPException(
                status_code=503,
                detail="No global model is available yet. Run a training round first.",
            )
        return self.model


HOLDER = ModelHolder()


def decode_image(raw: bytes) -> np.ndarray:
    """Turn an uploaded file into a 64x64 grayscale array.

    Accepts anything Pillow can open, so a demo can use a PNG exported from the
    dataset or a JPEG chest film without any client-side preparation.
    """
    try:
        image = Image.open(io.BytesIO(raw))
        image = image.convert("L").resize((IMAGE_SIZE, IMAGE_SIZE), Image.BILINEAR)
    except Exception as error:  # noqa: BLE001 - any decode failure is a client error
        raise HTTPException(status_code=400, detail=f"Unreadable image: {error}") from error
    return np.asarray(image, dtype=np.uint8)


def write_audit_entry(digest: str, prediction: Dict[str, Any]) -> None:
    """Record that a prediction happened, without recording what was in it."""
    config = StorageConfig()
    if not AUDIT_ENABLED or config.backend.lower() != "s3":
        return
    try:
        import boto3

        table = boto3.resource("dynamodb", region_name=config.aws_region).Table(
            config.audit_table
        )
```

```
            table.put_item(
                Item={
                    "request_hash": digest,
                    "timestamp": int(time.time()),
                    "predicted_label": prediction["label"],
                    "model_round": int(prediction.get("model_round") or 0),
                }
            )
        except Exception as error:  # pragma: no cover - telemetry must not break serving
            LOGGER.warning("audit write failed: %s", error)


    @app.get("/health")
    def health() -> Dict[str, Any]:
        """Liveness plus whether a trained model has been published yet."""
        try:
            HOLDER.refresh()
            model_round = HOLDER.metadata.get("round")
        except Exception as error:  # noqa: BLE001 - health must answer even when S3 is empty
            LOGGER.info("health check could not load a model: %s", error)
            model_round = None
        return {
            "status": "ok",
            "model_available": model_round is not None,
            "model_round": model_round,
            "storage_backend": StorageConfig().backend,
        }


    @app.get("/model")
    def model_info() -> Dict[str, Any]:
        """Describe the model currently being served, including its privacy budget."""
        HOLDER.refresh()
        if HOLDER.model is None:
            raise HTTPException(status_code=503, detail="No global model available yet")
        return {
            "run_id": HOLDER.metadata.get("run_id"),
            "round": HOLDER.metadata.get("round"),
            "accuracy": HOLDER.metadata.get("accuracy"),
            "loss": HOLDER.metadata.get("loss"),
            "privacy": HOLDER.metadata.get("privacy", {}),
            "classes": CLASS_NAMES,
            "input_size": IMAGE_SIZE,
            "loaded_at": int(HOLDER.loaded_at),
        }


    @app.get("/rounds")
    def rounds(run_id: Optional[str] = None, limit: int = 100) -> Dict[str, Any]:
        """Return the per-round training history that drives the dashboard chart."""
        config = StorageConfig()
        if config.backend.lower() == "s3":
            return {"rounds": _rounds_from_dynamo(config, run_id, limit)}
        return {"rounds": _rounds_from_file(config, limit)}


    def _rounds_from_dynamo(
        config: StorageConfig, run_id: Optional[str], limit: int
    ) -> List[Dict[str, Any]]:
        import boto3
        from boto3.dynamodb.conditions import Key

        table = boto3.resource("dynamodb", region_name=config.aws_region).Table(
```

```
            config.rounds_table
        )
        if run_id:
            response = table.query(
                KeyConditionExpression=Key("run_id").eq(run_id), Limit=limit
            )
        else:
            # No run specified: scan, then keep the newest run only. The table holds
            # a handful of demo runs, so a scan is cheaper than a secondary index.
            response = table.scan(Limit=limit)
        items = _decimals_to_floats(response.get("Items", []))
        if not items:
            return []
        if not run_id:
            newest = max(items, key=lambda item: item.get("timestamp", 0))["run_id"]
            items = [item for item in items if item["run_id"] == newest]
        return sorted(items, key=lambda item: item["round"])


    def _rounds_from_file(config: StorageConfig, limit: int) -> List[Dict[str, Any]]:
        import json

        path = os.path.join(config.local_root, "metrics.json")
        if not os.path.exists(path):
            return []
        with open(path, encoding="utf-8") as handle:
            return json.load(handle).get("rounds", [])[:limit]


    def _decimals_to_floats(value: Any) -> Any:
        """DynamoDB returns Decimal, which the JSON encoder refuses to serialise."""
        from decimal import Decimal

        if isinstance(value, Decimal):
            return int(value) if value == value.to_integral_value() else float(value)
        if isinstance(value, list):
            return [_decimals_to_floats(item) for item in value]
        if isinstance(value, dict):
            return {key: _decimals_to_floats(item) for key, item in value.items()}
        return value


    @app.post("/predict")
    async def predict(file: UploadFile = File(...)) -> Dict[str, Any]:
        """Classify one chest X-ray with the current global model."""
        raw = await file.read()
        if not raw:
            raise HTTPException(status_code=400, detail="Empty upload")
        if len(raw) > MAX_UPLOAD_BYTES:
            raise HTTPException(
                status_code=413, detail=f"Image exceeds {MAX_UPLOAD_BYTES} bytes"
            )

        model = HOLDER.get()
        array = decode_image(raw)
        tensor = data_utils.preprocess_image(array)

        with torch.no_grad():
            probabilities = torch.softmax(model(tensor), dim=1)[0]
        index = int(torch.argmax(probabilities).item())

        prediction = {
            "label": CLASS_NAMES[index],
```

```
            "confidence": round(float(probabilities[index].item()), 4),
            "probabilities": {
                name: round(float(probabilities[i].item()), 4)
                for i, name in enumerate(CLASS_NAMES)
            },
            "model_round": HOLDER.metadata.get("round"),
            "model_run_id": HOLDER.metadata.get("run_id"),
        }

        digest = hashlib.sha256(raw).hexdigest()
        write_audit_entry(digest, prediction)
        # The digest goes back to the caller so a clinician can quote it in a ticket
        # without attaching the image itself.
        prediction["request_hash"] = digest
        return prediction


    # Lambda entry point. API Gateway payloads are translated by Mangum.
    try:
        from mangum import Mangum

        handler = Mangum(app, lifespan="off")
    except ImportError:  # pragma: no cover - only needed inside the Lambda image
        handler = None


    if __name__ == "__main__":
        import uvicorn

        uvicorn.run(app, host="0.0.0.0", port=int(os.environ.get("PORT", "8000")))

```

dashboard/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="color-scheme" content="light dark" />
    <title>FedMed Federation Console</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

```

dashboard/package.json

```
{
  "name": "fedmed-dashboard",
  "private": true,
  "version": "0.1.0",
  "type": "module",
  "description": "Operator view of the FedMed federated training platform",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "18.3.1",
    "react-dom": "18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "4.3.4",
    "vite": "5.4.11"
  }
}

```

dashboard/vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

// The bundle is served from an S3 website endpoint at the bucket root, so the
// default absolute base path is correct. VITE_API_URL is baked in at build time
// by scripts/deploy_web.sh.
export default defineConfig({
  plugins: [react()],
  build: {
    outDir: "dist",
    sourcemap: false,
    chunkSizeWarningLimit: 700,
  },
  server: {
    port: 5173,
  },
});

```

dashboard/src/main.jsx

```
import React from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
import "./theme.css";

createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);

```

dashboard/src/theme.css

```
/* Design tokens.
 *
 * Both themes are defined explicitly rather than by inverting one. The dark
 * series colours are the same three hues re-stepped for the dark surface, and
 * the set was checked with the palette validator: worst adjacent colour-vision
 * separation is 9.2 in light and 9.4 in dark, against a target of 8.
 *
 * Aqua sits below 3:1 against the light surface, so every series carries a
 * visible direct label rather than relying on its colour.
 */

:root {
  color-scheme: light;

  --surface-1: #fcfcfb;
  --page-plane: #f9f9f7;
  --text-primary: #0b0b0b;
  --text-secondary: #52514e;
  --text-muted: #898781;
  --gridline: #e1e0d9;
  --baseline: #c3c2b7;
  --border: rgba(11, 11, 11, 0.1);

  --series-1: #2a78d6; /* global model */
  --series-2: #eb6834; /* hospital A */
  --series-3: #1baf7a; /* hospital B */
  --series-4: #e87ba4; /* hospital C */

  --status-good: #0ca30c;
  --status-warning: #fab219;
  --status-critical: #d03b3b;

  --shadow: 0 1px 2px rgba(11, 11, 11, 0.05);
  --radius: 10px;
  --font: system-ui, -apple-system, "Segoe UI", sans-serif;
  --mono: ui-monospace, SFMono-Regular, Menlo, Consolas, monospace;
}

@media (prefers-color-scheme: dark) {
  :root:where(:not([data-theme="light"])) {
    color-scheme: dark;

    --surface-1: #1a1a19;
    --page-plane: #0d0d0d;
    --text-primary: #ffffff;
    --text-secondary: #c3c2b7;
    --text-muted: #898781;
    --gridline: #2c2c2a;
    --baseline: #383835;
    --border: rgba(255, 255, 255, 0.1);

    --series-1: #3987e5;
    --series-2: #d95926;
    --series-3: #199e70;
    --series-4: #d55181;

    --shadow: none;
  }
}

```

```
:root[data-theme="dark"] {
  color-scheme: dark;

  --surface-1: #1a1a19;
  --page-plane: #0d0d0d;
  --text-primary: #ffffff;
  --text-secondary: #c3c2b7;
  --text-muted: #898781;
  --gridline: #2c2c2a;
  --baseline: #383835;
  --border: rgba(255, 255, 255, 0.1);

  --series-1: #3987e5;
  --series-2: #d95926;
  --series-3: #199e70;
  --series-4: #d55181;

  --shadow: none;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: var(--page-plane);
  color: var(--text-primary);
  font-family: var(--font);
  font-size: 14px;
  line-height: 1.5;
  -webkit-font-smoothing: antialiased;
}

.app {
  max-width: 1180px;
  margin: 0 auto;
  padding: 28px 20px 64px;
}

/* ---------------------------------------------------------------- header */

.masthead {
  display: flex;
  flex-wrap: wrap;
  align-items: flex-end;
  justify-content: space-between;
  gap: 16px;
  margin-bottom: 24px;
}

.masthead h1 {
  margin: 0;
  font-size: 22px;
  font-weight: 650;
  letter-spacing: -0.01em;
}

.masthead p {
  margin: 4px 0 0;
  color: var(--text-secondary);
  max-width: 60ch;
}
```

```
124
125     .controls {
126       display: flex;
127       align-items: center;
128       gap: 10px;
129     }
130
131     select,
132     button {
133       font: inherit;
134       color: var(--text-primary);
135       background: var(--surface-1);
136       border: 1px solid var(--border);
137       border-radius: 8px;
138       padding: 7px 11px;
139       cursor: pointer;
140     }
141
142     button:hover:not(:disabled),
143     select:hover {
144       border-color: var(--baseline);
145     }
146
147     button:disabled {
148       cursor: not-allowed;
149       opacity: 0.55;
150     }
151
152     .pill {
153       display: inline-flex;
154       align-items: center;
155       gap: 6px;
156       padding: 3px 9px;
157       border-radius: 999px;
158       border: 1px solid var(--border);
159       font-size: 12px;
160       color: var(--text-secondary);
161       white-space: nowrap;
162     }
163
164     .pill .dot {
165       width: 7px;
166       height: 7px;
167       border-radius: 50%;
168       background: var(--text-muted);
169     }
170
171     .pill.good .dot {
172       background: var(--status-good);
173     }
174
175     .pill.warn .dot {
176       background: var(--status-warning);
177     }
178
179     /* ---------------------------------------------------------------- cards */
180
181     .card {
182       background: var(--surface-1);
183       border: 1px solid var(--border);
184       border-radius: var(--radius);
185       box-shadow: var(--shadow);
186       padding: 18px 20px;
```

```
  }

  .card h2 {
    margin: 0 0 2px;
    font-size: 15px;
    font-weight: 620;
  }

  .card .subtitle {
    margin: 0 0 16px;
    color: var(--text-secondary);
    font-size: 13px;
  }

  .grid {
    display: grid;
    gap: 16px;
    margin-bottom: 16px;
  }

  .grid.tiles {
    grid-template-columns: repeat(auto-fit, minmax(190px, 1fr));
  }

  .grid.split {
    grid-template-columns: minmax(0, 1.65fr) minmax(0, 1fr);
  }

  .grid.hospitals {
    grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  }

  @media (max-width: 900px) {
    .grid.split {
      grid-template-columns: minmax(0, 1fr);
    }
  }

  /* ------------------------------------------------------------ stat tiles */

  .tile .label {
    color: var(--text-secondary);
    font-size: 12px;
    text-transform: uppercase;
    letter-spacing: 0.04em;
  }

  .tile .value {
    font-size: 30px;
    font-weight: 640;
    letter-spacing: -0.02em;
    margin-top: 4px;
  }

  .tile .note {
    color: var(--text-muted);
    font-size: 12px;
    margin-top: 2px;
  }

  /* ---------------------------------------------------------------- charts */

  .chart-wrap {
```

```
    position: relative;
    width: 100%;
    overflow-x: auto;
  }

  .legend {
    display: flex;
    flex-wrap: wrap;
    gap: 14px;
    margin-top: 12px;
    font-size: 12px;
    color: var(--text-secondary);
  }

  .legend .swatch {
    display: inline-block;
    width: 10px;
    height: 10px;
    border-radius: 3px;
    margin-right: 6px;
    vertical-align: -1px;
  }

  .tooltip {
    position: absolute;
    pointer-events: none;
    background: var(--surface-1);
    border: 1px solid var(--border);
    border-radius: 8px;
    padding: 8px 10px;
    font-size: 12px;
    box-shadow: 0 4px 14px rgba(11, 11, 11, 0.12);
    min-width: 132px;
    z-index: 5;
  }

  .tooltip .head {
    font-weight: 620;
    margin-bottom: 4px;
  }

  .tooltip .row {
    display: flex;
    justify-content: space-between;
    gap: 12px;
    font-variant-numeric: tabular-nums;
  }

  /* ------------------------------------------------------------ hospitals */

  .hospital .name {
    font-weight: 620;
    display: flex;
    align-items: center;
    gap: 8px;
  }

  .hospital .marker {
    width: 9px;
    height: 9px;
    border-radius: 3px;
  }

```

```
.hospital dl {
  margin: 12px 0 0;
  display: grid;
  grid-template-columns: auto auto;
  gap: 4px 12px;
  font-size: 13px;
}

.hospital dt {
  color: var(--text-secondary);
}

.hospital dd {
  margin: 0;
  text-align: right;
  font-variant-numeric: tabular-nums;
}

/* ---------------------------------------------------------------- predict */

.dropzone {
  border: 1px dashed var(--baseline);
  border-radius: var(--radius);
  padding: 22px;
  text-align: center;
  color: var(--text-secondary);
  cursor: pointer;
  transition: border-color 0.15s ease;
}

.dropzone:hover,
.dropzone.active {
  border-color: var(--series-1);
  color: var(--text-primary);
}

.preview {
  display: flex;
  gap: 16px;
  align-items: flex-start;
  margin-top: 16px;
}

.preview img {
  width: 128px;
  height: 128px;
  object-fit: cover;
  border-radius: 8px;
  border: 1px solid var(--border);
  background: var(--page-plane);
}

.result .verdict {
  font-size: 20px;
  font-weight: 640;
}

.prob-row {
  display: flex;
  align-items: center;
  gap: 10px;
  margin-top: 8px;
  font-size: 13px;
```

```
}

.prob-row .track {
  flex: 1;
  height: 8px;
  border-radius: 4px;
  background: var(--gridline);
  overflow: hidden;
}

.prob-row .fill {
  height: 100%;
  border-radius: 4px;
}

.mono {
  font-family: var(--mono);
  font-size: 12px;
  color: var(--text-muted);
  word-break: break-all;
}

/* ---------------------------------------------------------------- tables */

table {
  width: 100%;
  border-collapse: collapse;
  font-size: 13px;
  font-variant-numeric: tabular-nums;
}

th,
td {
  text-align: right;
  padding: 6px 8px;
  border-bottom: 1px solid var(--gridline);
}

th:first-child,
td:first-child {
  text-align: left;
}

th {
  color: var(--text-secondary);
  font-weight: 560;
}

details summary {
  cursor: pointer;
  color: var(--text-secondary);
  font-size: 13px;
  margin-top: 14px;
}

.callout {
  border-left: 3px solid var(--status-warning);
  padding: 10px 14px;
  background: var(--page-plane);
  border-radius: 0 8px 8px 0;
  color: var(--text-secondary);
  font-size: 13px;
  margin-top: 14px;
```

```
  }

  .error {
    border-left: 3px solid var(--status-critical);
    padding: 10px 14px;
    background: var(--page-plane);
    border-radius: 0 8px 8px 0;
    font-size: 13px;
  }

  footer {
    margin-top: 28px;
    color: var(--text-muted);
    font-size: 12px;
  }

```

dashboard/src/api.js

```
/**
 * Thin client for the FedMed inference API.
 *
 * The base URL is baked in at build time by scripts/deploy_web.sh. During local
 * development it falls back to the uvicorn container that docker compose starts
 * on port 8000, so `npm run dev` works against a compose run with no config.
 */

const BASE = (import.meta.env.VITE_API_URL || "http://localhost:8000").replace(/\/+$/, "");

export const apiBase = BASE;

async function request(path, options = {}) {
  const response = await fetch(`${BASE}${path}`, options);
  if (!response.ok) {
    // FastAPI reports problems as {"detail": "..."}; fall back to the status
    // line when the failure came from somewhere else, such as API Gateway.
    let detail = `${response.status} ${response.statusText}`;
    try {
      const body = await response.json();
      if (body && body.detail) detail = body.detail;
    } catch {
      /* response had no JSON body */
    }
    throw new Error(detail);
  }
  return response.json();
}

export function fetchHealth() {
  return request("/health");
}

export function fetchModel() {
  return request("/model");
}

export function fetchRounds(runId) {
  const query = runId ? `?run_id=${encodeURIComponent(runId)}` : "";
  return request(`/rounds${query}`);
}

export function predict(file) {
  const form = new FormData();
  form.append("file", file);
  return request("/predict", { method: "POST", body: form });
}

/**
 * The per-hospital class balance comes from the partition manifest rather than
 * the API, because it describes how the data was split before any training
 * happened. scripts/deploy_web.sh copies it into the bundle.
 */
export async function fetchPartition() {
  const response = await fetch("partition_summary.json", { cache: "no-cache" });
  if (!response.ok) return null;
  try {
    return await response.json();
  } catch {
    return null;
```

```
  }
}

/**
 * The centralised baseline, produced by scripts/train_centralized.py. It is the
 * accuracy a single model reaches when it is allowed to see all three shards at
 * once, so the chart can draw the line federated training is measured against.
 */
export async function fetchBaseline() {
  const response = await fetch("centralized_baseline.json", { cache: "no-cache" });
  if (!response.ok) return null;
  try {
    const payload = await response.json();
    return typeof payload.final_accuracy === "number" ? payload : null;
  } catch {
    return null;
  }
}

```

dashboard/src/App.jsx

```jsx
import { useCallback, useEffect, useMemo, useState } from "react";
import AccuracyChart from "./components/AccuracyChart";
import HospitalPanel from "./components/HospitalPanel";
import PrivacyPanel from "./components/PrivacyPanel";
import PredictPanel from "./components/PredictPanel";
import { apiBase, fetchBaseline, fetchHealth, fetchModel, fetchPartition, fetchRounds } from "./api";

const POLL_INTERVAL_MS = 5000;

function StatTile({ label, value, note }) {
  return (
    <div className="card tile">
      <div className="label">{label}</div>
      <div className="value">{value}</div>
      {note ? <div className="note">{note}</div> : null}
    </div>
  );
}

/**
 * Pull the per-hospital training accuracy for one site out of a round record.
 * Returns null rather than zero when a site did not report, so the chart draws
 * a gap instead of a plunge to the floor.
 */
function hospitalAccuracy(round, hospitalId) {
  const entry = (round.hospitals || []).find(
    (item) => String(item.hospital_id) === String(hospitalId),
  );
  return entry ? Number(entry.train_accuracy) : null;
}

export default function App() {
  const [rounds, setRounds] = useState([]);
  const [model, setModel] = useState(null);
  const [health, setHealth] = useState(null);
  const [partition, setPartition] = useState(null);
  const [baseline, setBaseline] = useState(null);
  const [error, setError] = useState(null);
  const [live, setLive] = useState(true);
  const [loading, setLoading] = useState(true);

  const refresh = useCallback(async () => {
    try {
      const [roundsPayload, healthPayload] = await Promise.all([fetchRounds(), fetchHealth()]);
      setRounds(roundsPayload.rounds || []);
      setHealth(healthPayload);
      setError(null);

      // /model returns 503 until the first checkpoint exists, which is normal
      // during the opening seconds of a demo and is not worth surfacing.
      if (healthPayload.model_available) {
        try {
          setModel(await fetchModel());
        } catch {
          /* the checkpoint is being written right now */
        }
      }
    } catch (caught) {
      setError(caught.message);
    } finally {
```

```
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    refresh();
    fetchPartition().then(setPartition);
    fetchBaseline().then(setBaseline);
  }, [refresh]);

  // Poll while a run is in flight. Five seconds is comfortably inside a round,
  // so the chart grows visibly during the demo without hammering the API.
  useEffect(() => {
    if (!live) return undefined;
    const timer = setInterval(refresh, POLL_INTERVAL_MS);
    return () => clearInterval(timer);
  }, [live, refresh]);

  const series = useMemo(() => {
    const hospitalIds = (partition?.hospitals || []).map((hospital) => hospital.hospital_id);
    const fallbackIds = rounds.length
      ? [...new Set((rounds[rounds.length - 1].hospitals || []).map((entry) => entry.hospital_id))]
      : [];
    const ids = (hospitalIds.length ? hospitalIds : fallbackIds).slice(0, 3);

    return [
      {
        key: "global",
        label: "global",
        legend: "global model on the held-out test set",
        color: "var(--series-1)",
        emphasis: true,
        accessor: (round) => Number(round.accuracy),
      },
      ...ids.map((id, index) => ({
        key: `hospital-${id}`,
        label: String(id).toUpperCase(),
        legend: `hospital ${String(id).toUpperCase()} local training accuracy`,
        color: `var(--series-${index + 2})`,
        accessor: (round) => hospitalAccuracy(round, id),
      })),
    ];
  }, [partition, rounds]);

  const latest = rounds.length ? rounds[rounds.length - 1] : null;
  const best = rounds.length ? Math.max(...rounds.map((round) => Number(round.accuracy))) : null;
  const totalImages = (partition?.hospitals || []).reduce(
    (sum, hospital) => sum + hospital.examples,
    0,
  );

  return (
    <div className="app">
      <header className="masthead">
        <div>
          <h1>FedMed federation console</h1>
          <p>
            Three hospitals train one pneumonia classifier without exchanging a single image. This
            page reads the aggregator's round history and serves the model it produced.
          </p>
        </div>

        <div className="controls">
```

```
            <span className={`pill ${health?.model_available ? "good" : "warn"}`}>
              <span className="dot" />
              {health?.model_available ? `model from round ${health.model_round}` : "no model yet"}
            </span>
            <button onClick={() => setLive((value) => !value)}>
              {live ? "Pause updates" : "Resume updates"}
            </button>
            <button onClick={refresh}>Refresh</button>
          </div>
        </header>

        {error ? (
          <div className="error" style={{ marginBottom: 16 }}>
            Could not reach the API at {apiBase}. {error}
          </div>
        ) : null}

        {loading ? <p className="subtitle">Loading...</p> : null}

        <div className="grid tiles">
          <StatTile
            label="Global accuracy"
            value={latest ? `${(Number(latest.accuracy) * 100).toFixed(1)}%` : "--"}
            note={latest ? `after round ${latest.round}` : "waiting for round 1"}
          />
          <StatTile
            label="Best so far"
            value={best !== null ? `${(best * 100).toFixed(1)}%` : "--"}
            note={`${rounds.length} rounds recorded`}
          />
          <StatTile
            label="Images kept local"
            value={totalImages ? totalImages.toLocaleString() : "--"}
            note="never transmitted, in any round"
          />
          <StatTile
            label="Privacy budget"
            value={
              model?.privacy?.epsilon != null
                ? `e = ${Number(model.privacy.epsilon).toFixed(1)}`
                : model?.privacy?.mechanism === "none"
                  ? "off"
                  : "--"
            }
            note={model?.privacy ? `at delta = ${model.privacy.delta}` : ""}
          />
        </div>

        <div className="grid split">
          <div className="card">
            <h2>Accuracy by round</h2>
            <p className="subtitle">
              The blue line is the aggregated model measured on a test set no hospital owns. The
              other lines are each site's accuracy on its own patients. The dashed line is what a
              single model reaches when it is allowed to pool all three datasets, which is the
              accuracy this platform gives up in exchange for keeping the images apart.
            </p>
            <AccuracyChart rounds={rounds} series={series} baseline={baseline?.final_accuracy ?? null} />

            <details>
              <summary>Show the numbers as a table</summary>
              <table style={{ marginTop: 10 }}>
                <thead>
```

```
                  <tr>
                    <th>Round</th>
                    <th>Global accuracy</th>
                    <th>Loss</th>
                    <th>Recall pneumonia</th>
                    <th>Seconds</th>
                  </tr>
                </thead>
                <tbody>
                  {rounds.map((round) => (
                    <tr key={round.round}>
                      <td>{round.round}</td>
                      <td>{(Number(round.accuracy) * 100).toFixed(2)}%</td>
                      <td>{Number(round.loss).toFixed(4)}</td>
                      <td>
                        {round.recall_pneumonia != null
                          ? `${(Number(round.recall_pneumonia) * 100).toFixed(1)}%`
                          : "--"}
                      </td>
                      <td>{round.round_seconds ?? "--"}</td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </details>
          </div>

          <PrivacyPanel model={model} />
        </div>

        <HospitalPanel partition={partition} latestRound={latest} />

        <div style={{ marginTop: 16 }}>
          <PredictPanel modelRound={model?.round} />
        </div>

        <footer>
          API {apiBase} · run {latest?.run_id || model?.run_id || "none"} · updates{" "}
          {live ? `every ${POLL_INTERVAL_MS / 1000}s` : "paused"}
        </footer>
      </div>
    );
  }

```

dashboard/src/components/AccuracyChart.jsx

```jsx
import { useEffect, useLayoutEffect, useMemo, useRef, useState } from "react";

/**
 * Accuracy across federated rounds.
 *
 * Hand-drawn SVG rather than a charting library. The chart has four series and
 * one axis; a library would add half a megabyte to the bundle and then have to
 * be fought to get thin marks, a crosshair tooltip and direct labels.
 *
 * Every series is labelled at its right-hand end as well as in the legend, so
 * identity never rests on colour alone. That is also what lets the aqua series
 * pass review: it sits below 3:1 against the light surface.
 */

const PADDING = { top: 18, right: 92, bottom: 34, left: 46 };
const HEIGHT = 300;

function buildPath(points) {
  return points.map((point, index) => `${index === 0 ? "M" : "L"}${point.x},${point.y}`).join(" ");
}

export default function AccuracyChart({ rounds, series, baseline }) {
  const baselineValue = typeof baseline === "number" ? baseline : null;
  const wrapRef = useRef(null);
  const [hover, setHover] = useState(null);
  const [width, setWidth] = useState(720);


  const geometry = useMemo(() => {
    if (!rounds.length) return null;

    const innerWidth = width - PADDING.left - PADDING.right;
    const innerHeight = HEIGHT - PADDING.top - PADDING.bottom;
    const maxRound = Math.max(...rounds.map((entry) => entry.round));
    const minRound = Math.min(...rounds.map((entry) => entry.round));

    // A fixed 0.4 to 1.0 window. Auto-scaling to the data would make a two point
    // wobble look like a breakthrough, which is exactly the wrong impression
    // when the point of the chart is that federated training converges.
    const yMin = 0.4;
    const yMax = 1.0;

    const xFor = (round) =>
      PADDING.left +
      (maxRound === minRound ? innerWidth / 2 : ((round - minRound) / (maxRound - minRound)) * innerWidth);
    const yFor = (value) =>
      PADDING.top + innerHeight - ((Math.min(Math.max(value, yMin), yMax) - yMin) / (yMax - yMin)) * innerHeight;

    const lines = series
      .map((definition) => {
        const points = rounds
          .map((entry) => {
            const value = definition.accessor(entry);
            return value === null || value === undefined || Number.isNaN(value)
              ? null
              : { x: xFor(entry.round), y: yFor(value), value, round: entry.round };
          })
          .filter(Boolean);
        return { ...definition, points };
      })
```

```
      .filter((line) => line.points.length > 0);

    const ticks = [0.4, 0.55, 0.7, 0.85, 1.0];
    const roundTicks = rounds.map((entry) => entry.round);

    // Direct labels are the reason this chart does not depend on colour alone,
    // so they have to stay readable when two series finish at the same accuracy.
    // Lay them out by pushing each one down until it clears the previous label,
    // then shift the whole stack up if it has run past the bottom of the plot.
    const labelCandidates = lines
      .map((line) => ({
        key: line.key,
        label: line.label,
        emphasis: line.emphasis,
        anchor: line.points[line.points.length - 1].y,
      }))
      .concat(
        baselineValue === null || baselineValue === undefined
          ? []
          : [{ key: "__baseline__", label: `centralised ${(baselineValue * 100).toFixed(1)}%`, anchor: yFor(baselineV
      )
      .sort((a, b) => a.anchor - b.anchor);

    const MIN_GAP = 14;
    let previous = -Infinity;
    for (const candidate of labelCandidates) {
      candidate.y = Math.max(candidate.anchor, previous + MIN_GAP);
      previous = candidate.y;
    }
    const overflow = previous - (PADDING.top + innerHeight);
    if (overflow > 0) {
      for (const candidate of labelCandidates) candidate.y -= overflow;
    }
    const labelPositions = Object.fromEntries(labelCandidates.map((item) => [item.key, item]));

    return { innerWidth, innerHeight, xFor, yFor, lines, ticks, roundTicks, minRound, maxRound, labelPositions };
  }, [rounds, series, width, baselineValue]);

  const hasGeometry = geometry !== null;

  // Track the container width so the chart reflows with the layout. A
  // ResizeObserver rather than a window resize listener, because the card also
  // changes width when the grid switches to a single column. Below 420px the
  // chart stops shrinking and its own container scrolls instead, so the axis
  // labels never collapse into each other.
  // Measure once synchronously before the browser paints. Waiting for the
  // observer's first callback leaves one frame where the chart is drawn at the
  // fallback width, which shows up as a visible jump on load.
  useLayoutEffect(() => {
    const node = wrapRef.current;
    if (node?.clientWidth) setWidth(Math.max(Math.floor(node.clientWidth), 420));
  }, [hasGeometry]);

  useEffect(() => {
    const node = wrapRef.current;
    if (!node || typeof ResizeObserver === "undefined") return undefined;

    const observer = new ResizeObserver((entries) => {
      const measured = entries[0]?.contentRect?.width;
      if (measured) setWidth(Math.max(Math.floor(measured), 420));
    });
    observer.observe(node);
    return () => observer.disconnect();
```

```
    // hasGeometry is in the dependency list because the wrapper element does not
    // exist during the empty state, so the observer has nothing to attach to
    // until the first round arrives.
  }, [hasGeometry]);

  if (!geometry) {
    return <p className="subtitle">No rounds yet. Start a training run and this fills in.</p>;
  }

  const handleMove = (event) => {
    const bounds = event.currentTarget.getBoundingClientRect();
    const x = event.clientX - bounds.left;
    let closest = null;
    for (const entry of rounds) {
      const distance = Math.abs(geometry.xFor(entry.round) - x);
      if (!closest || distance < closest.distance) closest = { entry, distance };
    }
    if (closest && closest.distance < 60) {
      setHover({ round: closest.entry.round, x: geometry.xFor(closest.entry.round) });
    } else {
      setHover(null);
    }
  };

  const hoveredRound = hover ? rounds.find((entry) => entry.round === hover.round) : null;

  return (
    <div className="chart-wrap" ref={wrapRef}>
      <svg
        width={width}
        height={HEIGHT}
        role="img"
        aria-label="Model accuracy by federated round"
        onMouseMove={handleMove}
        onMouseLeave={() => setHover(null)}
        style={{ display: "block" }}
      >
        {/* Recessive gridlines: hairlines behind the data, never competing with it. */}
        {geometry.ticks.map((tick) => (
          <g key={tick}>
            <line
              x1={PADDING.left}
              x2={width - PADDING.right}
              y1={geometry.yFor(tick)}
              y2={geometry.yFor(tick)}
              stroke="var(--gridline)"
              strokeWidth="1"
            />
            <text
              x={PADDING.left - 9}
              y={geometry.yFor(tick) + 4}
              textAnchor="end"
              fontSize="11"
              fill="var(--text-muted)"
            >
              {Math.round(tick * 100)}%
            </text>
          </g>
        ))}

        {/* The centralised baseline: what pooling every hospital's images would
            achieve. The gap between it and the blue line is the cost of privacy. */}
        {baselineValue !== null ? (
```

```
              <g>
                <line
                  x1={PADDING.left}
                  x2={width - PADDING.right}
                  y1={geometry.yFor(baselineValue)}
                  y2={geometry.yFor(baselineValue)}
                  stroke="var(--text-muted)"
                  strokeWidth="1"
                  strokeDasharray="4 4"
                />
                <text
                  x={width - PADDING.right + 7}
                  y={(geometry.labelPositions.__baseline__?.y ?? geometry.yFor(baselineValue)) + 4}
                  fontSize="11"
                  fill="var(--text-muted)"
                >
                  {(baselineValue * 100).toFixed(1)}% pooled
                </text>
              </g>
            ) : null}

            <line
              x1={PADDING.left}
              x2={width - PADDING.right}
              y1={HEIGHT - PADDING.bottom}
              y2={HEIGHT - PADDING.bottom}
              stroke="var(--baseline)"
              strokeWidth="1"
            />

            {geometry.roundTicks.map((round) => (
              <text
                key={round}
                x={geometry.xFor(round)}
                y={HEIGHT - PADDING.bottom + 17}
                textAnchor="middle"
                fontSize="11"
                fill="var(--text-muted)"
              >
                {round}
              </text>
            ))}
            <text
              x={PADDING.left + geometry.innerWidth / 2}
              y={HEIGHT - 2}
              textAnchor="middle"
              fontSize="11"
              fill="var(--text-secondary)"
            >
              federated round
            </text>

            {hover ? (
              <line
                x1={hover.x}
                x2={hover.x}
                y1={PADDING.top}
                y2={HEIGHT - PADDING.bottom}
                stroke="var(--baseline)"
                strokeWidth="1"
              />
            ) : null}

```

```
            {geometry.lines.map((line) => {
              const last = line.points[line.points.length - 1];
              return (
                <g key={line.key}>
                  <path
                    d={buildPath(line.points)}
                    fill="none"
                    stroke={line.color}
                    strokeWidth={line.emphasis ? 2.5 : 2}
                    strokeLinejoin="round"
                    strokeLinecap="round"
                    opacity={line.emphasis ? 1 : 0.85}
                  />
                  {/* A ring in the surface colour keeps overlapping markers legible. */}
                  {line.points.map((point) => (
                    <circle
                      key={`${line.key}-${point.round}`}
                      cx={point.x}
                      cy={point.y}
                      r={hover && hover.round === point.round ? 5 : 3.5}
                      fill={line.color}
                      stroke="var(--surface-1)"
                      strokeWidth="2"
                    />
                  ))}
                  {(() => {
                    const placed = geometry.labelPositions[line.key];
                    const labelY = placed ? placed.y : last.y;
                    return (
                      <>
                        {Math.abs(labelY - last.y) > 2 ? (
                          <line
                            x1={last.x + 3}
                            y1={last.y}
                            x2={last.x + 9}
                            y2={labelY}
                            stroke={line.color}
                            strokeWidth="1"
                            opacity="0.6"
                          />
                        ) : null}
                        <text
                          x={last.x + 12}
                          y={labelY + 4}
                          fontSize="11"
                          fill="var(--text-secondary)"
                          fontWeight={line.emphasis ? 600 : 400}
                        >
                          {line.label}
                        </text>
                      </>
                    );
                  })()}
                </g>
              );
            })}
          </svg>

          {hoveredRound ? (
            <div
              className="tooltip"
              style={{
                left: Math.min(Math.max(hover.x - 66, 0), width - 150),
```

```
              top: 8,
            }}
          >
            <div className="head">Round {hoveredRound.round}</div>
            {geometry.lines.map((line) => {
              const value = line.accessor(hoveredRound);
              if (value === null || value === undefined || Number.isNaN(value)) return null;
              return (
                <div className="row" key={line.key}>
                  <span>
                    <span className="swatch" style={{ background: line.color }} />
                    {line.label}
                  </span>
                  <span>{(value * 100).toFixed(1)}%</span>
                </div>
              );
            })}
          </div>
        ) : null}

        <div className="legend">
          {geometry.lines.map((line) => (
            <span key={line.key}>
              <span className="swatch" style={{ background: line.color }} />
              {line.legend || line.label}
            </span>
          ))}
          {baselineValue !== null ? (
            <span>
              <span
                className="swatch"
                style={{ background: "transparent", borderTop: "2px dashed var(--text-muted)", borderRadius: 0 }}
              />
              centralised baseline, all data pooled
            </span>
          ) : null}
        </div>
      </div>
    );
  }

```

dashboard/src/components/HospitalPanel.jsx

```jsx
/**
 * One card per participating site.
 *
 * The class-balance bar is the point of this panel. It shows at a glance that
 * the three hospitals hold very different populations, which is what makes the
 * federated problem hard and what a pooled dataset would hide.
 */

const SERIES_COLORS = ["var(--series-2)", "var(--series-3)", "var(--series-4)"];

function ClassBalanceBar({ counts, total }) {
  const normal = counts.normal || 0;
  const pneumonia = counts.pneumonia || 0;
  const denominator = total || normal + pneumonia || 1;
  const normalShare = (normal / denominator) * 100;

  return (
    <div style={{ marginTop: 12 }}>
      <div
        style={{
          display: "flex",
          height: 10,
          borderRadius: 5,
          overflow: "hidden",
          background: "var(--gridline)",
          // A 2px gap between adjacent fills, drawn in the surface colour.
          gap: 2,
        }}
        role="img"
        aria-label={`${normal} normal and ${pneumonia} pneumonia images`}
      >
        <div style={{ width: `${normalShare}%`, background: "var(--series-1)" }} />
        <div style={{ flex: 1, background: "var(--series-2)" }} />
      </div>
      <div
        style={{
          display: "flex",
          justifyContent: "space-between",
          fontSize: 12,
          color: "var(--text-secondary)",
          marginTop: 5,
        }}
      >
        <span>{normal.toLocaleString()} normal</span>
        <span>{pneumonia.toLocaleString()} pneumonia</span>
      </div>
    </div>
  );
}

export default function HospitalPanel({ partition, latestRound }) {
  const hospitals = partition?.hospitals || [];

  if (!hospitals.length) {
    return (
      <div className="card">
        <h2>Participating hospitals</h2>
        <p className="subtitle">
          The partition manifest was not published with this build, so only the counts reported
          during training are available.
```

```
            </p>
            <div className="grid hospitals">
              {(latestRound?.hospitals || []).map((entry, index) => (
                <div className="card hospital" key={entry.hospital_id}>
                  <div className="name">
                    <span className="marker" style={{ background: SERIES_COLORS[index % 3] }} />
                    Hospital {String(entry.hospital_id).toUpperCase()}
                  </div>
                  <dl>
                    <dt>Training images</dt>
                    <dd>{Number(entry.examples).toLocaleString()}</dd>
                    <dt>Local accuracy</dt>
                    <dd>{(Number(entry.train_accuracy) * 100).toFixed(1)}%</dd>
                  </dl>
                </div>
              ))}
            </div>
          </div>
        );
      }

      return (
        <div className="card">
          <h2>Participating hospitals</h2>
          <p className="subtitle">
            Split with {partition.strategy}. Each site keeps its own images and sends only weights.
          </p>

          <div className="grid hospitals">
            {hospitals.map((hospital, index) => {
              const live = (latestRound?.hospitals || []).find(
                (entry) => String(entry.hospital_id) === String(hospital.hospital_id),
              );
              return (
                <div className="card hospital" key={hospital.hospital_id}>
                  <div className="name">
                    <span className="marker" style={{ background: SERIES_COLORS[index % 3] }} />
                    Hospital {String(hospital.hospital_id).toUpperCase()}
                  </div>

                  <ClassBalanceBar counts={hospital.class_counts} total={hospital.examples} />

                  <dl>
                    <dt>Images held</dt>
                    <dd>{hospital.examples.toLocaleString()}</dd>
                    <dt>Pneumonia share</dt>
                    <dd>{(hospital.pneumonia_ratio * 100).toFixed(1)}%</dd>
                    <dt>Shard size</dt>
                    <dd>{hospital.size_mb} MB</dd>
                    {live ? (
                      <>
                        <dt>Last local accuracy</dt>
                        <dd>{(Number(live.train_accuracy) * 100).toFixed(1)}%</dd>
                        <dt>Last round time</dt>
                        <dd>{Number(live.local_seconds).toFixed(1)}s</dd>
                      </>
                    ) : null}
                  </dl>
                </div>
              );
            })}
          </div>

```

```
          <div className="callout">
            Hospital shares differ by an order of magnitude and their pneumonia rates differ by more
            than sixty points. Averaging models across sites this unbalanced is the whole difficulty
            of federated learning, and it is why the global curve lags a pooled baseline.
          </div>
        </div>
      );
    }

```

dashboard/src/components/PredictPanel.jsx

```jsx
import { useRef, useState } from "react";
import { predict } from "../api";

/**
 * Upload one chest X-ray and get the federated model's verdict.
 *
 * The preview is drawn from a local object URL, so the browser shows the image
 * without the page ever storing it anywhere. The server behaves the same way.
 */

const LABEL_COLORS = {
  normal: "var(--series-1)",
  pneumonia: "var(--series-2)",
};

export default function PredictPanel({ modelRound }) {
  const inputRef = useRef(null);
  const [dragging, setDragging] = useState(false);
  const [previewUrl, setPreviewUrl] = useState(null);
  const [result, setResult] = useState(null);
  const [error, setError] = useState(null);
  const [busy, setBusy] = useState(false);

  const submit = async (file) => {
    if (!file) return;
    setError(null);
    setResult(null);
    setBusy(true);

    if (previewUrl) URL.revokeObjectURL(previewUrl);
    setPreviewUrl(URL.createObjectURL(file));

    try {
      setResult(await predict(file));
    } catch (caught) {
      setError(caught.message);
    } finally {
      setBusy(false);
    }
  };

  const onDrop = (event) => {
    event.preventDefault();
    setDragging(false);
    submit(event.dataTransfer.files?.[0]);
  };

  return (
    <div className="card">
      <h2>Try the federated model</h2>
      <p className="subtitle">
        {modelRound
          ? `Serving the global model from round ${modelRound}.`
          : "Waiting for a trained model."}
      </p>

      <div
        className={`dropzone ${dragging ? "active" : ""}`}
        onClick={() => inputRef.current?.click()}
        onDragOver={(event) => {
```

```
        event.preventDefault();
        setDragging(true);
      }}
      onDragLeave={() => setDragging(false)}
      onDrop={onDrop}
      role="button"
      tabIndex={0}
      onKeyDown={(event) => {
        if (event.key === "Enter" || event.key === " ") inputRef.current?.click();
      }}
    >
      {busy ? "Classifying..." : "Drop a chest X-ray here, or click to choose a file"}
      <input
        ref={inputRef}
        type="file"
        accept="image/*"
        hidden
        onChange={(event) => submit(event.target.files?.[0])}
      />
    </div>

    {error ? <div className="error" style={{ marginTop: 14 }}>{error}</div> : null}

    {result ? (
      <div className="preview">
        {previewUrl ? <img src={previewUrl} alt="Uploaded chest X-ray" /> : null}
        <div className="result" style={{ flex: 1 }}>
          <div className="verdict" style={{ color: LABEL_COLORS[result.label] }}>
            {result.label}
          </div>
          <div style={{ color: "var(--text-secondary)", fontSize: 13 }}>
            {(result.confidence * 100).toFixed(1)}% confidence
          </div>

          {Object.entries(result.probabilities).map(([label, probability]) => (
            <div className="prob-row" key={label}>
              <span style={{ width: 78, color: "var(--text-secondary)" }}>{label}</span>
              <span className="track">
                <span
                  className="fill"
                  style={{ width: `${probability * 100}%`, background: LABEL_COLORS[label] }}
                />
              </span>
              <span style={{ width: 46, textAlign: "right", fontVariantNumeric: "tabular-nums" }}>
                {(probability * 100).toFixed(1)}%
              </span>
            </div>
          ))}

          <p className="mono" style={{ marginTop: 12 }}>
            audit hash {result.request_hash?.slice(0, 32)}...
          </p>
          <p style={{ color: "var(--text-muted)", fontSize: 12, margin: 0 }}>
            The image was decoded, classified and discarded. Only this hash was recorded.
          </p>
        </div>
      </div>
    ) : null}

    <div className="callout">
      A teaching demonstration on a public research dataset. Nothing here is a medical device and
      no output should inform a clinical decision.
    </div>
```

```
        </div>
      );
    }

```

dashboard/src/components/PrivacyPanel.jsx

```
/**
 * What the platform actually guarantees, stated without overclaiming.
 *
 * A dashboard that prints "differentially private" beside a green tick and stops
 * there is worse than one with no privacy panel at all. This one shows the
 * mechanism, the parameters, the resulting epsilon, and says plainly when that
 * epsilon is too large to mean much.
 */

function Row({ term, children }) {
  return (
    <>
      <dt>{term}</dt>
      <dd>{children}</dd>
    </>
  );
}

export default function PrivacyPanel({ model }) {
  const privacy = model?.privacy;

  if (!privacy) {
    return (
      <div className="card">
        <h2>Privacy</h2>
        <p className="subtitle">No model has been published yet.</p>
      </div>
    );
  }

  const disabled = privacy.mechanism === "none";
  const epsilon = privacy.epsilon;
  // Below about 10 the budget is in the range research papers treat as
  // meaningful. Above it, the noise is real but the formal guarantee is weak.
  const meaningful = typeof epsilon === "number" && epsilon <= 10;

  return (
    <div className="card">
      <h2>Privacy</h2>
      <p className="subtitle">Applied by the aggregator, before any checkpoint is written.</p>

      <span className={`pill ${disabled ? "warn" : meaningful ? "good" : "warn"}`}>
        <span className="dot" />
        {disabled ? "differential privacy off" : meaningful ? "budget is meaningful" : "noise applied, budget is loos
      </span>

      <dl className="hospital" style={{ display: "grid", gridTemplateColumns: "auto auto", gap: "4px 12px", marginTop
        <Row term="Mechanism">{disabled ? "none" : "clip then Gaussian noise"}</Row>
        <Row term="Clipping norm">{privacy.clipping_norm}</Row>
        <Row term="Noise multiplier">{privacy.noise_multiplier}</Row>
        <Row term="Rounds">{privacy.rounds}</Row>
        <Row term="Delta">{privacy.delta}</Row>
        <Row term="Epsilon">
          {typeof epsilon === "number" ? epsilon.toFixed(1) : "unbounded"}
        </Row>
      </dl>

      {!disabled && !meaningful ? (
        <div className="callout">
          This epsilon is large. With three participants all taking part in every round there is no
```

```
              subsampling amplification, so a noise level low enough to keep accuracy usable cannot buy
              a tight budget. The clipping and noise still blunt the memorisation of any single
              patient, but the formal guarantee is weak and should be reported as such. A real
              deployment fixes this with more participating sites, not with a bigger number here.
            </div>
          ) : null}

          {disabled ? (
            <div className="callout">
              Differential privacy is switched off for this run. Weight updates leave each hospital
              unclipped and unnoised, which is the correct configuration for measuring the accuracy
              ceiling and the wrong one for anything else.
            </div>
          ) : null}

          <details>
            <summary>What protects the data even without a tight epsilon</summary>
            <ul style={{ marginTop: 8, paddingLeft: 20, color: "var(--text-secondary)", fontSize: 13 }}>
              <li>Images never leave the hospital container. Only weight arrays cross the network.</li>
              <li>Each hospital reads one bucket, its own. There is no pooled image store to breach.</li>
              <li>The training channel is encrypted and the hospitals pin the aggregator certificate.</li>
              <li>Uploaded X-rays are classified in memory; the audit trail keeps a hash, not the image.</li>
            </ul>
          </details>
        </div>
      );
    }

```

scripts/partition_data.py

```
"""Download PneumoniaMNIST and split it into private per-hospital shards.

Run this once on a laptop. It produces:

    data/hospital_a/shard.npz     private to hospital A
    data/hospital_b/shard.npz     private to hospital B
    data/hospital_c/shard.npz     private to hospital C
    data/server/test.npz          the aggregator's held-out test set
    data/partition_summary.json   what the dashboard shows about the split

The default split is non-IID: a Dirichlet draw gives each hospital a different
class balance and a different volume, which is the situation federated learning
actually exists for. Pass ``--iid`` to produce the easy comparison case for the
report.
"""

from __future__ import annotations

import argparse
import json
import os
from typing import Dict, List

import numpy as np

HOSPITAL_IDS = ["a", "b", "c"]
CLASS_NAMES = ["normal", "pneumonia"]


def load_medmnist(size: int) -> Dict[str, tuple]:
    """Fetch the PneumoniaMNIST splits as raw NumPy arrays."""
    try:
        from medmnist import PneumoniaMNIST
    except ImportError as error:  # pragma: no cover - dependency guidance
        raise SystemExit(
            "medmnist is not installed. Run: pip install -r requirements-dev.txt"
        ) from error

    splits = {}
    for split in ("train", "val", "test"):
        dataset = PneumoniaMNIST(split=split, download=True, size=size)
        images = np.asarray(dataset.imgs)
        labels = np.asarray(dataset.labels).reshape(-1).astype(np.int64)
        if images.ndim == 4 and images.shape[-1] == 1:
            images = images[..., 0]
        splits[split] = (images.astype(np.uint8), labels)
        print(f"  {split:5s}: {images.shape[0]:5d} images, shape {images.shape[1:]}")
    return splits


def dirichlet_split(
    labels: np.ndarray, num_parts: int, alpha: float, rng: np.random.Generator
) -> List[np.ndarray]:
    """Assign every sample to a hospital, class by class, using Dirichlet weights.

    A small alpha means highly skewed hospitals. alpha=0.5 gives a realistic
    picture: one site sees mostly pneumonia, another is dominated by normals.
    """
    parts: List[List[int]] = [[] for _ in range(num_parts)]
    for class_index in range(len(CLASS_NAMES)):
```

```
            indices = np.where(labels == class_index)[0]
            rng.shuffle(indices)
            proportions = rng.dirichlet(np.repeat(alpha, num_parts))
            cut_points = (np.cumsum(proportions) * len(indices)).astype(int)[:-1]
            for part_index, chunk in enumerate(np.split(indices, cut_points)):
                parts[part_index].extend(chunk.tolist())

        result = []
        for part in parts:
            array = np.array(sorted(part), dtype=np.int64)
            rng.shuffle(array)
            result.append(array)
        return result


    def iid_split(
        labels: np.ndarray, num_parts: int, rng: np.random.Generator
    ) -> List[np.ndarray]:
        """Equal-sized shards with the same class balance as the full dataset."""
        indices = rng.permutation(len(labels))
        return [np.array(sorted(chunk)) for chunk in np.array_split(indices, num_parts)]


    def describe(labels: np.ndarray) -> Dict[str, object]:
        counts = np.bincount(labels, minlength=len(CLASS_NAMES))
        total = int(counts.sum())
        return {
            "examples": total,
            "class_counts": {name: int(count) for name, count in zip(CLASS_NAMES, counts)},
            "pneumonia_ratio": round(float(counts[1]) / max(total, 1), 4),
        }


    def main() -> int:
        parser = argparse.ArgumentParser(description="Partition PneumoniaMNIST across hospitals")
        parser.add_argument("--output", default="data", help="Output directory")
        parser.add_argument("--size", type=int, default=64, choices=[28, 64, 128], help="Image size")
        parser.add_argument("--alpha", type=float, default=0.5, help="Dirichlet concentration")
        parser.add_argument("--iid", action="store_true", help="Produce an IID split instead")
        parser.add_argument("--seed", type=int, default=42)
        args = parser.parse_args()

        rng = np.random.default_rng(args.seed)

        print(f"Downloading PneumoniaMNIST at {args.size}x{args.size} ...")
        splits = load_medmnist(args.size)

        # Hospitals get the training split. The official validation split is folded
        # into the server's test set so the centralised curve is measured on enough
        # patients to be stable.
        train_images, train_labels = splits["train"]
        test_images = np.concatenate([splits["test"][0], splits["val"][0]])
        test_labels = np.concatenate([splits["test"][1], splits["val"][1]])

        if args.iid:
            partitions = iid_split(train_labels, len(HOSPITAL_IDS), rng)
            strategy = "iid"
        else:
            partitions = dirichlet_split(train_labels, len(HOSPITAL_IDS), args.alpha, rng)
            strategy = f"dirichlet(alpha={args.alpha})"

        summary: Dict[str, object] = {
            "dataset": f"PneumoniaMNIST-{args.size}",
```

```
            "strategy": strategy,
            "seed": args.seed,
            "classes": CLASS_NAMES,
            "hospitals": [],
            "server_test": describe(test_labels),
        }

        for hospital_id, indices in zip(HOSPITAL_IDS, partitions):
            directory = os.path.join(args.output, f"hospital_{hospital_id}")
            os.makedirs(directory, exist_ok=True)
            path = os.path.join(directory, "shard.npz")
            np.savez_compressed(
                path, images=train_images[indices], labels=train_labels[indices]
            )
            stats = describe(train_labels[indices])
            stats["hospital_id"] = hospital_id
            stats["path"] = path
            stats["size_mb"] = round(os.path.getsize(path) / 1e6, 2)
            summary["hospitals"].append(stats)
            print(
                f"  hospital {hospital_id}: {stats['examples']:5d} images, "
                f"{stats['pneumonia_ratio'] * 100:5.1f}% pneumonia, {stats['size_mb']} MB"
            )

        server_directory = os.path.join(args.output, "server")
        os.makedirs(server_directory, exist_ok=True)
        np.savez_compressed(
            os.path.join(server_directory, "test.npz"), images=test_images, labels=test_labels
        )
        print(f"  server test set: {len(test_labels)} images")

        # Sanity check: the shards must be disjoint, or the privacy story is a lie.
        all_indices = np.concatenate(partitions)
        assert len(np.unique(all_indices)) == len(all_indices), "hospital shards overlap"
        print(f"  disjointness verified across {len(all_indices)} training images")

        summary_path = os.path.join(args.output, "partition_summary.json")
        with open(summary_path, "w", encoding="utf-8") as handle:
            json.dump(summary, handle, indent=2)
        print(f"Wrote {summary_path}")
        return 0


    if __name__ == "__main__":
        raise SystemExit(main())

```

scripts/run_local.py

```
    """Run the whole federation as local processes, without Docker.

    This is the fastest inner loop while developing: no image builds, and every
    process logs to the same terminal. ``docker compose up`` exercises the real
    container images and is what the cloud deployment mirrors.

        python scripts/run_local.py --rounds 5
    """

    from __future__ import annotations

    import argparse
    import os
    import subprocess
    import sys
    import time
    from typing import List

    REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    HOSPITAL_IDS = ["a", "b", "c"]


    def base_environment(args: argparse.Namespace) -> dict:
        environment = os.environ.copy()
        environment.update(
            {
                "PYTHONPATH": os.pathsep.join(
                    [os.path.join(REPO_ROOT, "common"), os.path.join(REPO_ROOT, "fl_server")]
                ),
                "PYTHONUNBUFFERED": "1",
                "STORAGE_BACKEND": "local",
                "LOCAL_ROOT": args.artifacts,
                "NUM_ROUNDS": str(args.rounds),
                "NUM_HOSPITALS": str(len(HOSPITAL_IDS)),
                "LOCAL_EPOCHS": str(args.local_epochs),
                "BATCH_SIZE": str(args.batch_size),
                "LEARNING_RATE": str(args.learning_rate),
                "DP_ENABLED": "true" if args.dp else "false",
                "DP_NOISE_MULTIPLIER": str(args.noise_multiplier),
                "DP_CLIP_NORM": str(args.clip_norm),
                "TLS_ENABLED": "true" if args.tls else "false",
                "TLS_CA_PATH": os.path.join(REPO_ROOT, "certs", "ca.crt"),
                "TLS_CERT_PATH": os.path.join(REPO_ROOT, "certs", "server.crt"),
                "TLS_KEY_PATH": os.path.join(REPO_ROOT, "certs", "server.key"),
                "RUN_ID": args.run_id,
                # With TLS on, the hostname must match a Subject Alternative Name in
                # the aggregator certificate, so dial "localhost" rather than the IP.
                "SERVER_ADDRESS": "localhost:8080" if args.tls else "127.0.0.1:8080",
                "BIND_ADDRESS": "0.0.0.0:8080",
            }
        )
        return environment


    def main() -> int:
        parser = argparse.ArgumentParser(description="Run FedMed locally as plain processes")
        parser.add_argument("--rounds", type=int, default=10)
        parser.add_argument("--local-epochs", type=int, default=1)
        parser.add_argument("--batch-size", type=int, default=64)
        parser.add_argument("--learning-rate", type=float, default=1e-3)
```

```
        parser.add_argument("--dp", action="store_true", help="Enable differential privacy")
        parser.add_argument("--noise-multiplier", type=float, default=0.05)
        parser.add_argument("--clip-norm", type=float, default=1.0)
        parser.add_argument(
            "--tls",
            action="store_true",
            help="Encrypt the gRPC channel using the certificates in ./certs",
        )
        parser.add_argument("--run-id", default="local-run")
        parser.add_argument("--data", default=os.path.join(REPO_ROOT, "data"))
        parser.add_argument(
            "--artifacts",
            default=os.environ.get("LOCAL_ROOT") or os.path.join(REPO_ROOT, "artifacts"),
            help="Where checkpoints and metrics are written. The sweep gives each "
                 "configuration its own directory so runs cannot overwrite each other.",
        )
        args = parser.parse_args()

        environment = base_environment(args)
        processes: List[subprocess.Popen] = []

        server_environment = dict(environment)
        server_environment["TEST_SHARD"] = os.path.join(args.data, "server", "test.npz")
        print("starting aggregation server ...")
        server = subprocess.Popen(
            [sys.executable, os.path.join(REPO_ROOT, "fl_server", "server.py")],
            env=server_environment,
            cwd=REPO_ROOT,
        )
        processes.append(server)

        time.sleep(3)  # let the gRPC listener bind before the hospitals dial in

        for hospital_id in HOSPITAL_IDS:
            hospital_environment = dict(environment)
            hospital_environment["HOSPITAL_ID"] = hospital_id
            hospital_environment["SHARD_PATH"] = os.path.join(
                args.data, f"hospital_{hospital_id}", "shard.npz"
            )
            print(f"starting hospital {hospital_id} ...")
            processes.append(
                subprocess.Popen(
                    [sys.executable, os.path.join(REPO_ROOT, "hospital_node", "client.py")],
                    env=hospital_environment,
                    cwd=REPO_ROOT,
                )
            )

        try:
            exit_code = server.wait()
        except KeyboardInterrupt:
            exit_code = 130
        finally:
            for process in processes:
                if process.poll() is None:
                    process.terminate()
            for process in processes:
                try:
                    process.wait(timeout=15)
                except subprocess.TimeoutExpired:
                    process.kill()

        print(f"server exited with code {exit_code}")
```

```
        print(f"artifacts in {environment['LOCAL_ROOT']}")
        return exit_code


if __name__ == "__main__":
    raise SystemExit(main())

```

scripts/run_experiments.py

```
"""Run the comparison sweep that the report's evaluation section is built from.

Four questions, answered by measurement rather than assertion:

  1. What does the model reach when all three datasets are pooled? (the ceiling)
  2. What does federated training reach on the same data, split three ways?
  3. How much does the non-IID split cost, compared with an even one?
  4. What does each level of differential-privacy noise cost in accuracy?

    python scripts/run_experiments.py            # the full sweep, about ten minutes
    python scripts/run_experiments.py --rounds 5 # quicker, for a dry run

Outputs artifacts/experiments.json, artifacts/experiments.md (paste straight into
the report) and two PNG figures.
"""

from __future__ import annotations

import argparse
import json
import os
import subprocess
import sys
import time
from typing import Any, Dict, List

REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))


def run(command: List[str], label: str) -> None:
    print(f"\n=== {label} ===")
    print("  " + " ".join(command[1:]))
    result = subprocess.run(command, cwd=REPO_ROOT)
    if result.returncode != 0:
        raise SystemExit(f"{label} failed with exit code {result.returncode}")


def ensure_iid_partition(seed: int) -> str:
    """Build the even split used for the IID comparison, if it is not there yet."""
    directory = os.path.join(REPO_ROOT, "data_iid")
    if os.path.exists(os.path.join(directory, "partition_summary.json")):
        print("reusing the existing IID partition in data_iid/")
        return directory

    run(
        [
            sys.executable,
            os.path.join("scripts", "partition_data.py"),
            "--output", "data_iid",
            "--iid",
            "--seed", str(seed),
        ],
        "building an IID partition for comparison",
    )
    return directory


def federated_run(
    name: str,
    data_dir: str,
```

```
        rounds: int,
        dp: bool,
        noise: float,
        clip: float,
    ) -> Dict[str, Any]:
        """One complete federation, isolated in its own artifacts directory."""
        output_root = os.path.join(REPO_ROOT, "artifacts", "experiments", name)
        os.makedirs(output_root, exist_ok=True)

        command = [
            sys.executable,
            os.path.join("scripts", "run_local.py"),
            "--rounds", str(rounds),
            "--run-id", name,
            "--data", data_dir,
            "--artifacts", output_root,
        ]
        if dp:
            command += ["--dp", "--noise-multiplier", str(noise), "--clip-norm", str(clip)]

        started = time.time()
        print(f"\n=== {name} ===")
        result = subprocess.run(command, cwd=REPO_ROOT)
        if result.returncode != 0:
            raise SystemExit(f"run {name} failed with exit code {result.returncode}")

        with open(os.path.join(output_root, "global", "summary.json"), encoding="utf-8") as handle:
            summary = json.load(handle)

        return {
            "name": name,
            "mode": "federated",
            "split": "iid" if "iid" in name else "non-iid",
            "dp": dp,
            "noise_multiplier": noise if dp else 0.0,
            "clip_norm": clip if dp else None,
            "rounds": rounds,
            "final_accuracy": summary["final_accuracy"],
            "best_accuracy": summary["best_accuracy"],
            "epsilon": summary["privacy"].get("epsilon"),
            "wall_seconds": round(time.time() - started, 1),
            "history": [
                {"round": entry["round"], "accuracy": entry["accuracy"], "loss": entry["loss"]}
                for entry in summary["history"]
            ],
        }


    def centralized(rounds: int) -> Dict[str, Any]:
        """The ceiling: one model, all three shards pooled.

        Given the same number of passes over the data as the federation gets, so the
        comparison is about where the data lives and not about compute budget.
        """
        output = os.path.join("artifacts", "experiments", "centralized.json")
        run(
            [
                sys.executable,
                os.path.join("scripts", "train_centralized.py"),
                "--epochs", str(rounds),
                "--output", output,
            ],
            "centralised baseline",
```

```
        )
        with open(os.path.join(REPO_ROOT, output), encoding="utf-8") as handle:
            summary = json.load(handle)

        return {
            "name": "centralized",
            "mode": "centralized",
            "split": "pooled",
            "dp": False,
            "noise_multiplier": 0.0,
            "clip_norm": None,
            "rounds": rounds,
            "final_accuracy": summary["final_accuracy"],
            "best_accuracy": summary["best_accuracy"],
            "epsilon": None,
            "wall_seconds": summary["wall_seconds"],
            "history": [
                {"round": entry["epoch"], "accuracy": entry["test_accuracy"], "loss": entry["test_loss"]}
                for entry in summary["history"]
            ],
        }


    DESCRIPTIONS = {
        "centralized": "All data pooled on one server",
        "fed-iid": "Federated, even split across three sites",
        "fed-noniid": "Federated, realistic uneven split",
        "fed-noniid-dp-005": "Federated non-IID, light privacy noise",
        "fed-noniid-dp-02": "Federated non-IID, moderate privacy noise",
        "fed-noniid-dp-05": "Federated non-IID, heavy privacy noise",
    }


    def write_markdown(results: List[Dict[str, Any]], path: str) -> None:
        ceiling = next(
            (entry["final_accuracy"] for entry in results if entry["mode"] == "centralized"), None
        )

        lines = [
            "# FedMed evaluation",
            "",
            f"Generated {time.strftime('%Y-%m-%d %H:%M')}. PneumoniaMNIST at 64 pixels, three sites,",
            f"{results[0]['rounds']} rounds or epochs, identical model and learning rate throughout.",
            "",
            "| Configuration | Final accuracy | Best round | vs pooled | Epsilon | Seconds |",
            "|---|---|---|---|---|---|",
        ]

        for entry in results:
            # Signed as accuracy relative to the ceiling, so a cost reads as a
            # negative number rather than as a confusing positive "gap".
            gap = (
                "baseline"
                if entry["mode"] == "centralized"
                else f"{(entry['final_accuracy'] - ceiling) * 100:+.1f} pts"
                if ceiling is not None
                else "n/a"
            )
            epsilon = f"{entry['epsilon']:.0f}" if entry["epsilon"] is not None else "none"
            lines.append(
                f"| {DESCRIPTIONS.get(entry['name'], entry['name'])} "
                f"| {entry['final_accuracy'] * 100:.1f}% "
                f"| {entry['best_accuracy'] * 100:.1f}% "
```

```
                f"| {gap} "
                f"| {epsilon} "
                f"| {entry['wall_seconds']:.0f} |"
            )

        lines += [
            "",
            "## Reading the table",
            "",
            "The gap between the pooled baseline and federated non-IID training is the price of",
            "keeping the images apart. The gap between the IID and non-IID federated rows is the",
            "price of the hospitals being genuinely different from one another, which is the part",
            "a synthetic even split would hide.",
            "",
            "The epsilon column is where the honesty lives. Noise levels light enough to preserve",
            "accuracy produce budgets far too large to claim as a formal guarantee, because three",
            "sites all participating in every round give no subsampling amplification to collect.",
            "The mitigation is more participating hospitals, not a smaller number in this column.",
            "",
            "Final and best accuracy differ by several points in some rows. Federated averaging over",
            "three unequal sites is genuinely noisy round to round, so quote both columns rather than",
            "picking whichever one flatters the configuration.",
            "",
        ]

        with open(path, "w", encoding="utf-8") as handle:
            handle.write("\n".join(lines))


    def write_figures(results: List[Dict[str, Any]], directory: str) -> None:
        try:
            import matplotlib

            matplotlib.use("Agg")
            import matplotlib.pyplot as plt
        except ImportError:
            print("matplotlib is not installed, skipping the figures")
            return

        # Figure 1: convergence.
        figure, axes = plt.subplots(figsize=(7.5, 4.2))
        for entry in results:
            if not entry["history"]:
                continue
            axes.plot(
                [point["round"] for point in entry["history"]],
                [point["accuracy"] * 100 for point in entry["history"]],
                marker="o",
                markersize=3.5,
                linewidth=1.8,
                label=DESCRIPTIONS.get(entry["name"], entry["name"]),
            )
        axes.set_xlabel("round (federated) or epoch (centralised)")
        axes.set_ylabel("test accuracy (%)")
        axes.set_title("Convergence under each configuration")
        axes.grid(True, linewidth=0.5, alpha=0.35)
        axes.legend(fontsize=8, loc="lower right")
        figure.tight_layout()
        figure.savefig(os.path.join(directory, "convergence.png"), dpi=160)
        plt.close(figure)

        # Figure 2: the privacy and accuracy trade, which is the interesting one.
        noisy = [entry for entry in results if entry["dp"]]
```

```
        clean = next((entry for entry in results if entry["name"] == "fed-noniid"), None)
        if noisy:
            points = ([(0.0, clean["final_accuracy"])] if clean else []) + [
                (entry["noise_multiplier"], entry["final_accuracy"]) for entry in noisy
            ]
            points.sort()

            figure, axes = plt.subplots(figsize=(6.4, 4.0))
            axes.plot(
                [point[0] for point in points],
                [point[1] * 100 for point in points],
                marker="o",
                linewidth=1.8,
            )
            for entry in noisy:
                if entry["epsilon"] is not None:
                    axes.annotate(
                        f"e={entry['epsilon']:.0f}",
                        (entry["noise_multiplier"], entry["final_accuracy"] * 100),
                        textcoords="offset points",
                        xytext=(6, 6),
                        fontsize=8,
                    )
            axes.set_xlabel("noise multiplier")
            axes.set_ylabel("test accuracy (%)")
            axes.set_title("What each unit of privacy noise costs in accuracy")
            axes.grid(True, linewidth=0.5, alpha=0.35)
            figure.tight_layout()
            figure.savefig(os.path.join(directory, "privacy_tradeoff.png"), dpi=160)
            plt.close(figure)

        print(f"figures written to {directory}")


    def main() -> int:
        parser = argparse.ArgumentParser(description="Run the FedMed evaluation sweep")
        parser.add_argument("--rounds", type=int, default=10)
        parser.add_argument("--clip-norm", type=float, default=1.0)
        parser.add_argument("--seed", type=int, default=42)
        parser.add_argument(
            "--skip-iid",
            action="store_true",
            help="Skip the IID comparison, which needs a second partition",
        )
        args = parser.parse_args()

        output_directory = os.path.join(REPO_ROOT, "artifacts", "experiments")
        os.makedirs(output_directory, exist_ok=True)

        data_dir = os.path.join(REPO_ROOT, "data")
        results: List[Dict[str, Any]] = [centralized(args.rounds)]

        if not args.skip_iid:
            iid_dir = ensure_iid_partition(args.seed)
            results.append(
                federated_run("fed-iid", iid_dir, args.rounds, False, 0.0, args.clip_norm)
            )

        results.append(
            federated_run("fed-noniid", data_dir, args.rounds, False, 0.0, args.clip_norm)
        )
        for noise, label in ((0.05, "005"), (0.2, "02"), (0.5, "05")):
            results.append(
```

```
                federated_run(
                    f"fed-noniid-dp-{label}", data_dir, args.rounds, True, noise, args.clip_norm
                )
            )

        json_path = os.path.join(output_directory, "experiments.json")
        with open(json_path, "w", encoding="utf-8") as handle:
            json.dump({"rounds": args.rounds, "results": results}, handle, indent=2)

        markdown_path = os.path.join(REPO_ROOT, "artifacts", "experiments.md")
        write_markdown(results, markdown_path)
        write_figures(results, output_directory)

        print("\n" + "=" * 70)
        with open(markdown_path, encoding="utf-8") as handle:
            print(handle.read())
        print(f"raw results: {json_path}")
        return 0


    if __name__ == "__main__":
        raise SystemExit(main())

```

scripts/train_centralized.py

```
    """Train one model on all three shards pooled together.

    This is the number every federated result is judged against. It is what you
    would get if the three hospitals were allowed to hand their images to a single
    server, so the gap between this and the federated accuracy is exactly the price
    the platform pays for privacy.

        python scripts/train_centralized.py --epochs 10
    """

    from __future__ import annotations

    import argparse
    import json
    import os
    import time

    import numpy as np

    from fedmed import data as data_utils
    from fedmed.model import build_model, count_parameters

    HOSPITAL_IDS = ["a", "b", "c"]


    def pool_shards(data_root: str) -> data_utils.Shard:
        """Concatenate every hospital's shard, which the real platform never does."""
        images = []
        labels = []
        for hospital_id in HOSPITAL_IDS:
            shard = data_utils.load_shard(
                os.path.join(data_root, f"hospital_{hospital_id}", "shard.npz")
            )
            images.append(shard.images)
            labels.append(shard.labels)
        return data_utils.Shard(
            np.concatenate(images), np.concatenate(labels), name="centralized"
        )


    def main() -> int:
        parser = argparse.ArgumentParser(description="Centralised baseline for FedMed")
        parser.add_argument("--data", default="data")
        parser.add_argument("--epochs", type=int, default=10)
        parser.add_argument("--batch-size", type=int, default=64)
        parser.add_argument("--learning-rate", type=float, default=1e-3)
        parser.add_argument("--seed", type=int, default=42)
        parser.add_argument("--output", default="artifacts/centralized_baseline.json")
        args = parser.parse_args()

        train_shard = pool_shards(args.data)
        test_shard = data_utils.load_shard(os.path.join(args.data, "server", "test.npz"))
        print(f"pooled training set: {len(train_shard)} images {train_shard.class_counts}")
        print(f"test set:            {len(test_shard)} images {test_shard.class_counts}")

        model = build_model(seed=args.seed)
        print(f"model parameters:    {count_parameters(model):,}")

        started_at = time.time()
        history = []
```

```
        for epoch in range(1, args.epochs + 1):
            stats = data_utils.train_local(
                model,
                train_shard,
                epochs=1,
                batch_size=args.batch_size,
                learning_rate=args.learning_rate,
            )
            results = data_utils.evaluate(model, test_shard)
            history.append(
                {
                    "epoch": epoch,
                    "train_loss": stats["train_loss"],
                    "test_accuracy": results["accuracy"],
                    "test_loss": results["loss"],
                }
            )
            print(
                f"epoch {epoch:2d} | train loss {stats['train_loss']:.4f} | "
                f"test accuracy {results['accuracy']:.4f}"
            )

        final = data_utils.evaluate(model, test_shard)
        summary = {
            "mode": "centralized",
            "epochs": args.epochs,
            "train_examples": len(train_shard),
            "wall_seconds": round(time.time() - started_at, 2),
            "final_accuracy": final["accuracy"],
            "final_loss": final["loss"],
            "recall_normal": final["recall_normal"],
            "recall_pneumonia": final["recall_pneumonia"],
            "best_accuracy": max(entry["test_accuracy"] for entry in history),
            "history": history,
        }

        os.makedirs(os.path.dirname(args.output) or ".", exist_ok=True)
        with open(args.output, "w", encoding="utf-8") as handle:
            json.dump(summary, handle, indent=2)

        print()
        print(f"centralised baseline accuracy: {final['accuracy']:.4f}")
        print(f"  recall normal    {final['recall_normal']:.4f}")
        print(f"  recall pneumonia {final['recall_pneumonia']:.4f}")
        print(f"written to {args.output}")
        return 0


    if __name__ == "__main__":
        raise SystemExit(main())

```

scripts/aws_common.sh

```
#!/usr/bin/env bash
# Shared helpers for the AWS scripts. Sourced, never executed directly.

set -euo pipefail

REPO_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
INFRA_DIR="${REPO_ROOT}/infra"
STATE_FILE="${REPO_ROOT}/.fedmed-run"

# Git Bash rewrites arguments that look like paths, which corrupts ECS ARNs and
# CloudWatch log group names such as /ecs/fedmed.
export MSYS_NO_PATHCONV=1
export MSYS2_ARG_CONV_EXCL='*'

log()  { printf '\033[1;36m==>\033[0m %s\n' "$*"; }
warn() { printf '\033[1;33mwarning:\033[0m %s\n' "$*" >&2; }
die()  { printf '\033[1;31merror:\033[0m %s\n' "$*" >&2; exit 1; }

require() {
  command -v "$1" >/dev/null 2>&1 || die "$1 is not on PATH. See docs/runbook.md for prerequisites."
}

# Read a single Terraform output. Fails loudly rather than returning an empty
# string, because an empty cluster name silently launches nothing.
tf_output() {
  local name="$1"
  local value
  value="$(terraform -chdir="${INFRA_DIR}" output -raw "${name}" 2>/dev/null || true)"
  [[ -n "${value}" ]] || die "Terraform output '${name}' is empty. Has 'terraform apply' finished?"
  printf '%s' "${value}"
}

# Same, but tolerates an empty result for genuinely optional outputs.
tf_output_optional() {
  terraform -chdir="${INFRA_DIR}" output -raw "$1" 2>/dev/null || true
}

# Turn the subnet_ids output into the bare comma-separated form the
# awsvpcConfiguration argument expects.
tf_subnets_csv() {
  terraform -chdir="${INFRA_DIR}" output -json subnet_ids \
    | tr -d '[]" \n\r\t'
}

# Build the --network-configuration argument. Public IPs are required because
# the tasks sit in public subnets with no NAT Gateway; inbound access is closed
# by the security groups regardless.
network_config() {
  local security_group="$1"
  printf 'awsvpcConfiguration={subnets=[%s],securityGroups=[%s],assignPublicIp=ENABLED}' \
    "$(tf_subnets_csv)" "${security_group}"
}

# Private IP of a running task, found through its elastic network interface.
task_private_ip() {
  local cluster="$1" task_arn="$2" eni

  eni="$(aws ecs describe-tasks --cluster "${cluster}" --tasks "${task_arn}" \
    --query "tasks[0].attachments[0].details[?name=='networkInterfaceId'].value" \
    --output text)"
```

```
  [[ -n "${eni}" && "${eni}" != "None" ]] || die "Could not find the network interface for task ${task_arn}"

  aws ec2 describe-network-interfaces --network-interface-ids "${eni}" \
    --query 'NetworkInterfaces[0].PrivateIpAddress' --output text
}

# Read newline-separated input into the named array. `mapfile` would be shorter
# but it needs bash 4, and macOS still ships bash 3.2 as /bin/bash.
read_lines_into() {
  local target="$1" line
  eval "${target}=()"
  while IFS= read -r line; do
    [[ -n "${line}" ]] || continue
    eval "${target}+=(\"\${line}\")"
  done
}

account_id() {
  aws sts get-caller-identity --query Account --output text
}

# Build a containerOverrides environment list from NAME=VALUE pairs.
# Emits JSON, so values containing spaces or quotes still survive.
env_overrides_json() {
  python_bin="$(command -v python3 || command -v python)"
  [[ -n "${python_bin}" ]] || die "python is required to build the task overrides"
  "${python_bin}" -c '
import json, sys
pairs = []
for item in sys.argv[1:]:
    name, _, value = item.partition("=")
    pairs.append({"name": name, "value": value})
print(json.dumps(pairs))
' "$@"
}

```

scripts/gen_certs.sh

```
#!/usr/bin/env bash
# Generate the self-signed certificate material for the Flower gRPC channel.
#
#     ./scripts/gen_certs.sh                      # local + Cloud Map names
#     ./scripts/gen_certs.sh 54.211.0.10          # also trust a task public IP
#
# Produces certs/ca.crt, certs/ca.key, certs/server.crt, certs/server.key.
# The hospitals pin ca.crt, so a man in the middle cannot present its own
# certificate for the aggregator even though nothing here is publicly trusted.
#
# Run it from the repository root. CERT_DIR must stay a relative path on
# Windows, for the reason explained just below.
set -euo pipefail

# Git Bash on Windows rewrites arguments that look like POSIX paths, which turns
# an OpenSSL subject such as /C=IN/O=... into C:/Program Files/Git/C=IN/O=...
# Disabling that translation is the only way to pass a subject through. It also
# means absolute POSIX paths are no longer translated, so keep CERT_DIR relative.
export MSYS_NO_PATHCONV=1
export MSYS2_ARG_CONV_EXCL='*'

EXTRA_SAN="${1:-}"
CERT_DIR="${CERT_DIR:-certs}"
DAYS="${DAYS:-365}"
SUBJECT_BASE="/C=IN/O=FedMed Teaching Federation"

mkdir -p "${CERT_DIR}"

# Subject Alternative Names must cover every address a hospital might dial:
# the compose service name, the Cloud Map record, and localhost for scripts.
SAN="DNS:fl-server,DNS:fl-server.fedmed.local,DNS:localhost,IP:127.0.0.1"
if [[ -n "${EXTRA_SAN}" ]]; then
  if [[ "${EXTRA_SAN}" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
    SAN="${SAN},IP:${EXTRA_SAN}"
  else
    SAN="${SAN},DNS:${EXTRA_SAN}"
  fi
fi
echo "Subject Alternative Names: ${SAN}"

echo "1/3 creating the certificate authority"
openssl req -x509 -newkey rsa:4096 -sha256 -days "${DAYS}" -nodes \
  -keyout "${CERT_DIR}/ca.key" -out "${CERT_DIR}/ca.crt" \
  -subj "${SUBJECT_BASE}/CN=FedMed Root CA" 2>/dev/null

echo "2/3 creating the aggregator key and signing request"
openssl req -newkey rsa:4096 -sha256 -nodes \
  -keyout "${CERT_DIR}/server.key" -out "${CERT_DIR}/server.csr" \
  -subj "${SUBJECT_BASE}/CN=fl-server" 2>/dev/null

# The extensions go in a real file rather than a <(process substitution). Native
# Windows OpenSSL cannot open the /dev/fd handle that substitution produces.
EXT_FILE="${CERT_DIR}/openssl-ext.cnf"
cat > "${EXT_FILE}" <<EXT
subjectAltName=${SAN}
basicConstraints=CA:FALSE
keyUsage=digitalSignature,keyEncipherment
extendedKeyUsage=serverAuth
EXT

```

```
    echo "3/3 signing the aggregator certificate"
    openssl x509 -req -in "${CERT_DIR}/server.csr" \
      -CA "${CERT_DIR}/ca.crt" -CAkey "${CERT_DIR}/ca.key" -CAcreateserial \
      -out "${CERT_DIR}/server.crt" -days "${DAYS}" -sha256 \
      -extfile "${EXT_FILE}" 2>/dev/null

    rm -f "${CERT_DIR}/server.csr" "${EXT_FILE}"
    chmod 600 "${CERT_DIR}"/*.key

    echo
    echo "Certificates written to ${CERT_DIR}/"
    openssl x509 -in "${CERT_DIR}/server.crt" -noout -subject -dates -ext subjectAltName

    cat <<'NOTE'

    Next steps:
      local   set TLS_ENABLED=true and mount ./certs into the containers at /certs
      aws     ./scripts/upload_certs.sh   (stores them in SSM Parameter Store)

    Never commit certs/. The .gitignore in this repo already excludes it.
    NOTE

```

scripts/push_images.sh

```
#!/usr/bin/env bash
# Build the three images and push them to ECR.
#
#     ./scripts/push_images.sh              # all three
#     ./scripts/push_images.sh api          # just one
#
# The images are built for linux/amd64 explicitly. On an Apple Silicon laptop the
# default build produces arm64 images, and a Fargate task definition pinned to
# X86_64 will then fail at startup with an exec format error that is genuinely
# hard to read.

source "$(dirname "${BASH_SOURCE[0]}")/aws_common.sh"

require aws
require docker
require terraform

TARGETS=("$@")
[[ ${#TARGETS[@]} -eq 0 ]] && TARGETS=(server hospital api)

REGION="$(tf_output region)"
ACCOUNT="$(account_id)"
REGISTRY="${ACCOUNT}.dkr.ecr.${REGION}.amazonaws.com"

log "signing in to ${REGISTRY}"
aws ecr get-login-password --region "${REGION}" \
  | docker login --username AWS --password-stdin "${REGISTRY}"

dockerfile_for() {
  case "$1" in
    server)   echo "fl_server/Dockerfile" ;;
    hospital) echo "hospital_node/Dockerfile" ;;
    api)      echo "inference_api/Dockerfile" ;;
    *)        die "unknown image '$1'. Choose from: server, hospital, api" ;;
  esac
}

# fl_server/Dockerfile bakes the held-out test set into the image, so that the
# aggregator needs no read permission on any hospital bucket. Without the file
# the build fails several minutes in, with a message that does not say why.
if [[ " ${TARGETS[*]} " == *" server "* && ! -f "${REPO_ROOT}/data/server/test.npz" ]]; then
  die "data/server/test.npz is missing. Run: python scripts/partition_data.py"
fi

for target in "${TARGETS[@]}"; do
  dockerfile="$(dockerfile_for "${target}")"
  repository="${REGISTRY}/fedmed-${target}"

  log "building fedmed-${target} from ${dockerfile}"
  docker build \
    --platform linux/amd64 \
    -f "${REPO_ROOT}/${dockerfile}" \
    -t "${repository}:latest" \
    "${REPO_ROOT}"

  log "pushing ${repository}:latest"
  docker push "${repository}:latest"
done

log "images pushed"
```

```
    echo
    echo "If you changed the inference API, publish the new image to the function:"
    echo "    aws lambda update-function-code --function-name fedmed-inference-api \\"
    echo "        --image-uri ${REGISTRY}/fedmed-api:latest"

```

scripts/start_run.sh

```
#!/usr/bin/env bash
# Launch one federated training run on Fargate.
#
#     ./scripts/start_run.sh                      # ten rounds, defaults from Terraform
#     ROUNDS=20 DP_NOISE=0.2 ./scripts/start_run.sh
#     ./scripts/start_run.sh --follow             # stream the aggregator log
#
# The sequence matters. The aggregator has to be listening and have a resolvable
# address before any hospital dials in, and a one-off Fargate task does not
# register itself in Cloud Map the way a service would. So the script starts the
# aggregator, waits for it, reads its private IP off the task's network
# interface, publishes that IP under fl-server.fedmed.local, and only then starts
# the hospitals.
#
# Everything exits on its own when the rounds finish. Nothing here creates an ECS
# service, so there is no process that will quietly bill overnight.

source "$(dirname "${BASH_SOURCE[0]}")/aws_common.sh"

require aws
require terraform

FOLLOW=false
[[ "${1:-}" == "--follow" ]] && FOLLOW=true

RUN_ID="${RUN_ID:-run-$(date +%Y%m%d-%H%M%S)}"
ROUNDS="${ROUNDS:-}"
LOCAL_EPOCHS="${LOCAL_EPOCHS:-}"
DP_ENABLED="${DP_ENABLED:-}"
DP_NOISE="${DP_NOISE:-}"
DP_CLIP="${DP_CLIP:-}"

CLUSTER="$(tf_output cluster_name)"
SERVER_TD="$(tf_output server_task_definition)"
HOSPITAL_TD="$(tf_output hospital_task_definition)"
SERVER_SG="$(tf_output server_security_group)"
HOSPITAL_SG="$(tf_output hospital_security_group)"
LOG_GROUP="$(tf_output log_group)"
REGION="$(tf_output region)"
SD_SERVICE_ID="$(tf_output_optional cloud_map_service_id)"
SERVER_DNS="$(tf_output_optional server_dns_name)"
TLS_ACTIVE="$(tf_output_optional tls_active)"
ACCOUNT="$(account_id)"
PROJECT="${PROJECT:-fedmed}"

# Optional per-run overrides. Anything left empty keeps the value baked into the
# task definition by Terraform, so a plain run needs no environment at all.
server_env=("RUN_ID=${RUN_ID}")
hospital_env=("RUN_ID=${RUN_ID}")
for pair in "NUM_ROUNDS=${ROUNDS}" "LOCAL_EPOCHS=${LOCAL_EPOCHS}" \
            "DP_ENABLED=${DP_ENABLED}" "DP_NOISE_MULTIPLIER=${DP_NOISE}" \
            "DP_CLIP_NORM=${DP_CLIP}"; do
  [[ "${pair#*=}" != "" ]] && server_env+=("${pair}")
done
for pair in "LOCAL_EPOCHS=${LOCAL_EPOCHS}"; do
  [[ "${pair#*=}" != "" ]] && hospital_env+=("${pair}")
done

log "run id ${RUN_ID}"
log "cluster ${CLUSTER} in ${REGION}"
```

```
[[ "${TLS_ACTIVE}" == "true" ]] && log "gRPC channel is TLS encrypted" || warn "gRPC channel is in plaintext (tls_ena

# ------------------------------------------------------------------------------
log "starting the aggregation server"
SERVER_TASK_ARN="$(aws ecs run-task \
  --cluster "${CLUSTER}" \
  --launch-type FARGATE \
  --task-definition "${SERVER_TD}" \
  --count 1 \
  --network-configuration "$(network_config "${SERVER_SG}")" \
  --overrides "{\"containerOverrides\":[{\"name\":\"fl-server\",\"environment\":$(env_overrides_json "${server_env[@]
  --started-by "${RUN_ID}" \
  --query 'tasks[0].taskArn' --output text)"

[[ -n "${SERVER_TASK_ARN}" && "${SERVER_TASK_ARN}" != "None" ]] || die "the aggregator task failed to launch"
log "aggregator task ${SERVER_TASK_ARN##*/}"

log "waiting for the aggregator to reach RUNNING (usually 30 to 60 seconds)"
aws ecs wait tasks-running --cluster "${CLUSTER}" --tasks "${SERVER_TASK_ARN}"

SERVER_IP="$(task_private_ip "${CLUSTER}" "${SERVER_TASK_ARN}")"
log "aggregator private IP ${SERVER_IP}"

# ------------------------------------------------------------------------------
if [[ -n "${SD_SERVICE_ID}" ]]; then
  log "registering ${SERVER_DNS} -> ${SERVER_IP} in Cloud Map"
  aws servicediscovery register-instance \
    --service-id "${SD_SERVICE_ID}" \
    --instance-id "fl-server" \
    --attributes "AWS_INSTANCE_IPV4=${SERVER_IP}" >/dev/null
  SERVER_ADDRESS="${SERVER_DNS}:8080"
  # The record has a ten second TTL but the resolver still needs a moment.
  sleep 10
else
  warn "Cloud Map is disabled, hospitals will dial the IP directly"
  [[ "${TLS_ACTIVE}" == "true" ]] && die "TLS needs the Cloud Map DNS name; set tls_enabled = false or enable_cloud_m
  SERVER_ADDRESS="${SERVER_IP}:8080"
fi

# ------------------------------------------------------------------------------
HOSPITAL_TASK_ARNS=()
for hospital in a b c; do
  bucket="${PROJECT}-${ACCOUNT}-data-${hospital}"
  log "starting hospital ${hospital} against ${bucket}"

  overrides=("${hospital_env[@]}" "HOSPITAL_ID=${hospital}" "DATA_BUCKET=${bucket}" "SERVER_ADDRESS=${SERVER_ADDRESS}
  arn="$(aws ecs run-task \
    --cluster "${CLUSTER}" \
    --launch-type FARGATE \
    --task-definition "${HOSPITAL_TD}" \
    --count 1 \
    --network-configuration "$(network_config "${HOSPITAL_SG}")" \
    --overrides "{\"containerOverrides\":[{\"name\":\"hospital\",\"environment\":$(env_overrides_json "${overrides[@]
    --started-by "${RUN_ID}" \
    --query 'tasks[0].taskArn' --output text)"
  [[ -n "${arn}" && "${arn}" != "None" ]] || die "hospital ${hospital} failed to launch"
  HOSPITAL_TASK_ARNS+=("${arn}")
done

# Remember what was started so stop_run.sh can clean up without guessing.
{
  echo "RUN_ID=${RUN_ID}"
  echo "CLUSTER=${CLUSTER}"
```

```
    echo "SERVER_TASK_ARN=${SERVER_TASK_ARN}"
    echo "HOSPITAL_TASK_ARNS=${HOSPITAL_TASK_ARNS[*]}"
    echo "SD_SERVICE_ID=${SD_SERVICE_ID}"
  } > "${STATE_FILE}"

  cat <<SUMMARY

    Run ${RUN_ID} is live.

    Watch it:
      aws logs tail ${LOG_GROUP} --follow --since 5m
      aws ecs list-tasks --cluster ${CLUSTER} --desired-status RUNNING

    Results land in:
      s3://$(tf_output models_bucket)/global/
      DynamoDB table $(tf_output rounds_table), partition key ${RUN_ID}

    Stop everything early:
      ./scripts/stop_run.sh

  SUMMARY

  if [[ "${FOLLOW}" == "true" ]]; then
    log "streaming logs, press Ctrl-C to detach (the run keeps going)"
    aws logs tail "${LOG_GROUP}" --follow --since 5m
  fi

```

scripts/stop_run.sh

```
#!/usr/bin/env bash
# Stop every FedMed task and release the Cloud Map registration.
#
#     ./scripts/stop_run.sh
#
# Run this as the last command of every lab session. An AWS Academy session
# expires after four hours, but a Fargate task started inside it keeps running
# and keeps billing after the console has logged you out. This script does not
# depend on the state file: it also finds and stops tasks left behind by an
# earlier session.

source "$(dirname "${BASH_SOURCE[0]}")/aws_common.sh"

require aws
require terraform

CLUSTER="$(tf_output cluster_name)"
SD_SERVICE_ID="$(tf_output_optional cloud_map_service_id)"

log "looking for running tasks in ${CLUSTER}"
mapfile -t TASKS < <(aws ecs list-tasks --cluster "${CLUSTER}" --desired-status RUNNING \
  --query 'taskArns[]' --output text | tr '\t' '\n' | sed '/^$/d')

# A task can also be stuck in PENDING while it pulls its image, which still bills.
mapfile -t PENDING < <(aws ecs list-tasks --cluster "${CLUSTER}" --desired-status PENDING \
  --query 'taskArns[]' --output text | tr '\t' '\n' | sed '/^$/d')
TASKS+=("${PENDING[@]}")

if [[ ${#TASKS[@]} -eq 0 ]]; then
  log "no running or pending tasks"
else
  for task in "${TASKS[@]}"; do
    [[ -n "${task}" ]] || continue
    log "stopping ${task##*/}"
    aws ecs stop-task --cluster "${CLUSTER}" --task "${task}" \
      --reason "stopped by stop_run.sh" >/dev/null
  done
fi

if [[ -n "${SD_SERVICE_ID}" ]]; then
  log "deregistering the aggregator from Cloud Map"
  # Not an error if nothing is registered; a run that already finished cleanly
  # leaves no instance behind.
  aws servicediscovery deregister-instance \
    --service-id "${SD_SERVICE_ID}" \
    --instance-id "fl-server" >/dev/null 2>&1 || log "nothing was registered"
fi

rm -f "${STATE_FILE}"

log "done. Confirm with:"
echo "    aws ecs list-tasks --cluster ${CLUSTER}"

```

scripts/deploy_web.sh

```
#!/usr/bin/env bash
# Build the dashboard against the deployed API and publish it to S3.
#
#     ./scripts/deploy_web.sh
#
# The API URL is baked in at build time as VITE_API_URL, so the published bundle
# is self-contained and the browser never has to be told where the backend lives.

source "$(dirname "${BASH_SOURCE[0]}")/aws_common.sh"

require aws
require terraform
require npm

API_URL="$(tf_output api_url)"
BUCKET="$(tf_output website_bucket)"
SITE_URL="$(tf_output website_url)"

log "API      ${API_URL}"
log "bucket   s3://${BUCKET}"

cd "${REPO_ROOT}/dashboard"

# The per-hospital class balance is a property of how the data was split, not
# something the API knows, so the manifest is copied into the bundle.
if [[ -f "${REPO_ROOT}/data/partition_summary.json" ]]; then
  cp "${REPO_ROOT}/data/partition_summary.json" public/partition_summary.json
  log "bundled the partition manifest"
else
  warn "data/partition_summary.json is missing; hospital cards will fall back to training metrics"
fi

# The centralised baseline is what the federated curve is compared against.
if [[ -f "${REPO_ROOT}/artifacts/centralized_baseline.json" ]]; then
  cp "${REPO_ROOT}/artifacts/centralized_baseline.json" public/centralized_baseline.json
  log "bundled the centralised baseline"
else
  warn "no centralised baseline found; run scripts/train_centralized.py to draw the comparison line"
fi

if [[ ! -d node_modules ]]; then
  log "installing dashboard dependencies"
  npm install
fi

log "building the dashboard"
VITE_API_URL="${API_URL}" npm run build

log "uploading"
# Hashed asset filenames can be cached hard; index.html must not be, or a
# redeploy during the demo will keep serving the previous bundle.
aws s3 sync dist "s3://${BUCKET}" \
  --delete \
  --exclude index.html \
  --cache-control "public,max-age=31536000,immutable" \
  --only-show-errors

aws s3 cp dist/index.html "s3://${BUCKET}/index.html" \
  --cache-control "no-cache,no-store,must-revalidate" \
  --content-type "text/html" \
```

```
    --only-show-errors

cat <<SUMMARY

  Dashboard published.

      ${SITE_URL}

  It is plain HTTP on an S3 website endpoint. That is acceptable here because the
  page holds no credentials and no patient data, and it is called out in
  docs/threat-model.md rather than left as a silent gap.

SUMMARY

```

scripts/upload_data.sh

```
#!/usr/bin/env bash
# Move each hospital's shard into that hospital's own bucket.
#
#     python scripts/partition_data.py     # first, to create the shards
#     ./scripts/upload_data.sh
#
# One shard, one bucket, no shared location. This is the only moment in the whole
# project where images travel anywhere, and after it every image sits in exactly
# one place that only its own hospital reads.

source "$(dirname "${BASH_SOURCE[0]}")/aws_common.sh"

require aws
require terraform

ACCOUNT="$(account_id)"
PROJECT="${PROJECT:-fedmed}"
DATA_DIR="${DATA_DIR:-${REPO_ROOT}/data}"

for hospital in a b c; do
  shard="${DATA_DIR}/hospital_${hospital}/shard.npz"
  bucket="${PROJECT}-${ACCOUNT}-data-${hospital}"

  [[ -f "${shard}" ]] || die "missing ${shard}. Run: python scripts/partition_data.py"

  size="$(du -h "${shard}" | cut -f1)"
  log "hospital ${hospital}: ${size} -> s3://${bucket}/shard.npz"
  aws s3 cp "${shard}" "s3://${bucket}/shard.npz" \
    --sse AES256 \
    --only-show-errors
done

log "verifying that each bucket holds exactly one shard"
for hospital in a b c; do
  bucket="${PROJECT}-${ACCOUNT}-data-${hospital}"
  aws s3 ls "s3://${bucket}/" --human-readable
done

cat <<'NOTE'

The aggregator has no reason to read any of these buckets and its image contains
no code path that does. The only dataset it opens is the held-out test split
baked into its own image.
NOTE

```

scripts/teardown.sh

```
#!/usr/bin/env bash
# Delete the entire stack.
#
#     ./scripts/teardown.sh
#
# Run this after the final demo. Nothing in the stack bills much at rest, but ECR
# storage and any forgotten task do add up over weeks, and an AWS Academy account
# that runs out of credit locks you out of your own project.

source "$(dirname "${BASH_SOURCE[0]}")/aws_common.sh"

require aws
require terraform

warn "This destroys every bucket, table, image and log group in the FedMed stack."
read -r -p "Type the word destroy to continue: " CONFIRMATION
[[ "${CONFIRMATION}" == "destroy" ]] || die "cancelled"

# Tasks hold on to network interfaces, which blocks security group deletion.
log "stopping anything that is still running"
"${REPO_ROOT}/scripts/stop_run.sh" || warn "stop_run.sh reported a problem, continuing"

log "waiting for the network interfaces to be released"
sleep 30

log "destroying the Terraform stack"
terraform -chdir="${INFRA_DIR}" destroy -auto-approve

log "stack destroyed. Confirm the bill has stopped in the Billing console."

```

tests/test_model_and_data.py

```
"""Tests for the model plumbing and the shard loader.

These cover the invariants the federation silently depends on: that a parameter
list survives a round trip through Flower's NumPy representation, that every
hospital builds the same architecture, and that a malformed shard is rejected at
load time rather than producing nonsense results twenty minutes into a run.
"""

from __future__ import annotations

import numpy as np
import pytest
import torch

from fedmed import data as data_utils
from fedmed.model import (
    CLASS_NAMES,
    IMAGE_SIZE,
    build_model,
    count_parameters,
    get_weights,
    set_weights,
)


def make_shard(count: int = 40, seed: int = 0) -> data_utils.Shard:
    rng = np.random.default_rng(seed)
    images = rng.integers(0, 256, size=(count, IMAGE_SIZE, IMAGE_SIZE), dtype=np.uint8)
    labels = rng.integers(0, len(CLASS_NAMES), size=count).astype(np.int64)
    return data_utils.Shard(images=images, labels=labels, name="synthetic")


def test_two_models_built_with_the_same_seed_are_identical():
    """Every hospital must start a run from the same weights.

    If they did not, the first FedAvg would average unrelated random networks and
    the run would spend several rounds recovering from the damage.
    """
    left = get_weights(build_model(seed=7))
    right = get_weights(build_model(seed=7))

    assert len(left) == len(right)
    for a, b in zip(left, right):
        np.testing.assert_array_equal(a, b)


def test_weights_survive_a_round_trip():
    model = build_model(seed=1)
    other = build_model(seed=2)
    original = get_weights(model)

    set_weights(other, original)

    for a, b in zip(original, get_weights(other)):
        np.testing.assert_allclose(a, b, rtol=1e-6, atol=1e-8)


def test_set_weights_rejects_the_wrong_number_of_arrays():
    model = build_model(seed=1)
    weights = get_weights(model)
```

```
    with pytest.raises(ValueError, match="parameter count mismatch"):
        set_weights(model, weights[:-1])


def test_the_parameter_list_carries_no_integer_buffers():
    """GroupNorm was chosen over BatchNorm partly for this.

    BatchNorm contributes an int64 num_batches_tracked buffer. Averaging integer
    counters across hospitals is meaningless, and it is the kind of bug that
    shows up as a slightly worse model rather than as an error.
    """
    for array in get_weights(build_model(seed=1)):
        assert np.issubdtype(array.dtype, np.floating)


def test_the_model_is_small_enough_for_a_one_vcpu_task():
    # Roughly a hundred kilobytes per round of network traffic per hospital.
    assert count_parameters(build_model()) < 100_000


def test_forward_pass_shape():
    model = build_model(seed=1)
    logits = model(torch.zeros(4, 1, IMAGE_SIZE, IMAGE_SIZE))
    assert logits.shape == (4, len(CLASS_NAMES))


def test_load_shard_round_trip(tmp_path):
    shard = make_shard(25)
    path = tmp_path / "shard.npz"
    np.savez_compressed(path, images=shard.images, labels=shard.labels)

    loaded = data_utils.load_shard(str(path))

    assert len(loaded) == 25
    np.testing.assert_array_equal(loaded.images, shard.images)
    assert sum(loaded.class_counts.values()) == 25


def test_load_shard_accepts_a_trailing_channel_axis(tmp_path):
    """MedMNIST hands back (N, H, W, 1); the loader has to squeeze it."""
    shard = make_shard(10)
    path = tmp_path / "shard.npz"
    np.savez_compressed(path, images=shard.images[..., None], labels=shard.labels)

    assert data_utils.load_shard(str(path)).images.shape == (10, IMAGE_SIZE, IMAGE_SIZE)


def test_load_shard_rejects_the_wrong_image_size(tmp_path):
    path = tmp_path / "shard.npz"
    np.savez_compressed(
        path, images=np.zeros((5, 28, 28), dtype=np.uint8), labels=np.zeros(5, dtype=np.int64)
    )

    with pytest.raises(ValueError, match="expected 64x64"):
        data_utils.load_shard(str(path))


def test_load_shard_rejects_a_label_count_mismatch(tmp_path):
    path = tmp_path / "shard.npz"
    np.savez_compressed(
        path,
        images=np.zeros((5, IMAGE_SIZE, IMAGE_SIZE), dtype=np.uint8),
```

```
            labels=np.zeros(4, dtype=np.int64),
        )

        with pytest.raises(ValueError, match="images but"):
            data_utils.load_shard(str(path))


    def test_training_reduces_loss_on_a_learnable_shard():
        """A shard the model can memorise; if this does not improve, training is broken."""
        rng = np.random.default_rng(3)
        labels = np.array([0, 1] * 30, dtype=np.int64)
        images = np.stack(
            [
                (rng.integers(0, 60, (IMAGE_SIZE, IMAGE_SIZE)) if label == 0
                 else rng.integers(195, 256, (IMAGE_SIZE, IMAGE_SIZE))).astype(np.uint8)
                for label in labels
            ]
        )
        shard = data_utils.Shard(images=images, labels=labels)

        model = build_model(seed=5)
        before = data_utils.evaluate(model, shard)["loss"]
        data_utils.train_local(model, shard, epochs=3, batch_size=16, learning_rate=1e-2)
        after = data_utils.evaluate(model, shard)["loss"]

        assert after < before


    def test_evaluate_reports_per_class_recall():
        results = data_utils.evaluate(build_model(seed=1), make_shard(30))

        for name in CLASS_NAMES:
            assert 0.0 <= results[f"recall_{name}"] <= 1.0
        assert 0.0 <= results["accuracy"] <= 1.0


    def test_preprocess_image_normalises_to_a_single_batch():
        tensor = data_utils.preprocess_image(
            np.full((IMAGE_SIZE, IMAGE_SIZE), 255, dtype=np.uint8)
        )

        assert tensor.shape == (1, 1, IMAGE_SIZE, IMAGE_SIZE)
        assert torch.allclose(tensor, torch.ones_like(tensor))

```

tests/test_inference_api.py

```
"""End-to-end tests for the public API.

These run the real FastAPI application against a checkpoint written to a
temporary directory, so they exercise the same code path the Lambda uses,
including the model cache and the image decoder.

The privacy assertions are the point of this file. A regression that started
persisting uploads would still pass every accuracy test, so it is checked here
explicitly.
"""

from __future__ import annotations

import importlib
import io
import os
import sys

import numpy as np
import pytest
from PIL import Image

from fedmed.model import IMAGE_SIZE, build_model, get_weights
from fedmed.storage import LocalStorage

REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
API_DIR = os.path.join(REPO_ROOT, "inference_api")


@pytest.fixture
def client(tmp_path, monkeypatch):
    """A test client backed by a freshly written checkpoint.

    The application module is imported after the environment is set, because it
    reads the storage configuration at import time.
    """
    storage = LocalStorage(str(tmp_path))
    storage.save_checkpoint(
        4,
        get_weights(build_model(seed=42)),
        {
            "run_id": "test-run",
            "round": 4,
            "accuracy": 0.88,
            "loss": 0.31,
            "privacy": {"mechanism": "none", "epsilon": None, "delta": 1e-5},
        },
    )

    monkeypatch.setenv("STORAGE_BACKEND", "local")
    monkeypatch.setenv("LOCAL_ROOT", str(tmp_path))
    monkeypatch.setenv("AUDIT_ENABLED", "false")
    monkeypatch.syspath_prepend(API_DIR)
    sys.modules.pop("app", None)

    from fastapi.testclient import TestClient

    app_module = importlib.import_module("app")
    with TestClient(app_module.app) as test_client:
        yield test_client, tmp_path
```

```
    sys.modules.pop("app", None)


def png_bytes(value: int = 200, size: int = IMAGE_SIZE) -> bytes:
    buffer = io.BytesIO()
    Image.fromarray(np.full((size, size), value, dtype=np.uint8)).save(buffer, format="PNG")
    return buffer.getvalue()


def test_health_reports_the_published_round(client):
    test_client, _ = client

    payload = test_client.get("/health").json()

    assert payload["status"] == "ok"
    assert payload["model_available"] is True
    assert payload["model_round"] == 4


def test_model_endpoint_describes_the_checkpoint(client):
    test_client, _ = client

    payload = test_client.get("/model").json()

    assert payload["round"] == 4
    assert payload["run_id"] == "test-run"
    assert payload["classes"] == ["normal", "pneumonia"]
    assert payload["privacy"]["mechanism"] == "none"


def test_predict_returns_a_calibrated_distribution(client):
    test_client, _ = client

    payload = test_client.post("/predict", files={"file": ("x.png", png_bytes(), "image/png")}).json()

    assert payload["label"] in {"normal", "pneumonia"}
    assert 0.0 <= payload["confidence"] <= 1.0
    assert sum(payload["probabilities"].values()) == pytest.approx(1.0, abs=1e-3)
    assert payload["model_round"] == 4


def test_predict_accepts_an_image_that_is_not_already_64_pixels(client):
    """A clinician uploading a full-resolution film should not have to resize it."""
    test_client, _ = client

    response = test_client.post(
        "/predict", files={"file": ("x.png", png_bytes(size=512), "image/png")}
    )

    assert response.status_code == 200


def test_predict_returns_a_stable_hash_for_identical_bytes(client):
    test_client, _ = client
    raw = png_bytes()

    first = test_client.post("/predict", files={"file": ("a.png", raw, "image/png")}).json()
    second = test_client.post("/predict", files={"file": ("b.png", raw, "image/png")}).json()

    assert first["request_hash"] == second["request_hash"]
    assert len(first["request_hash"]) == 64

```

```
124
125     def test_predict_never_writes_the_uploaded_image_anywhere(client):
126         """The privacy claim in the report, asserted rather than described.
127
128         The only file the API is allowed to create is nothing at all: the checkpoint
129         directory it read from must look exactly the same afterwards.
130         """
131         test_client, root = client
132         before = sorted(str(path.relative_to(root)) for path in root.rglob("*") if path.is_file())
133
134         test_client.post("/predict", files={"file": ("scan.png", png_bytes(), "image/png")})
135
136         after = sorted(str(path.relative_to(root)) for path in root.rglob("*") if path.is_file())
137         assert before == after
138
139
140     def test_predict_rejects_an_empty_upload(client):
141         test_client, _ = client
142
143         response = test_client.post("/predict", files={"file": ("empty.png", b"", "image/png")})
144
145         assert response.status_code == 400
146
147
148     def test_predict_rejects_something_that_is_not_an_image(client):
149         test_client, _ = client
150
151         response = test_client.post(
152             "/predict", files={"file": ("notes.txt", b"this is not a chest x-ray", "text/plain")}
153         )
154
155         assert response.status_code == 400
156         assert "Unreadable image" in response.json()["detail"]
157
158
159     def test_rounds_endpoint_reads_the_local_metrics_file(client, tmp_path):
160         test_client, root = client
161         import json
162
163         with open(os.path.join(str(root), "metrics.json"), "w", encoding="utf-8") as handle:
164             json.dump({"run_id": "test-run", "rounds": [{"round": 1, "accuracy": 0.7}]}, handle)
165
166         payload = test_client.get("/rounds").json()
167
168         assert payload["rounds"][0]["accuracy"] == 0.7
169
170
171     def test_rounds_endpoint_is_empty_before_the_first_run(client):
172         test_client, _ = client
173
174         assert test_client.get("/rounds").json()["rounds"] == []
175
```

tests/test_partitioning.py

```
"""Tests for the data split.

The central claim of the project is that no image is in two places at once. That
claim is only as good as the partitioner, so it is tested directly rather than
assumed.
"""

from __future__ import annotations

import importlib.util
import os

import numpy as np
import pytest

SCRIPT_PATH = os.path.join(
    os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "scripts", "partition_data.py"
)


def load_partition_module():
    """Import the script by path; scripts/ is not an installed package."""
    spec = importlib.util.spec_from_file_location("partition_data", SCRIPT_PATH)
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module


@pytest.fixture(scope="module")
def partition():
    return load_partition_module()


def make_labels(count: int = 900, pneumonia_ratio: float = 0.74) -> np.ndarray:
    """Labels with roughly the class balance of the real training split."""
    pneumonia = int(count * pneumonia_ratio)
    labels = np.array([1] * pneumonia + [0] * (count - pneumonia), dtype=np.int64)
    np.random.default_rng(0).shuffle(labels)
    return labels


def test_dirichlet_shards_are_disjoint(partition):
    labels = make_labels()
    parts = partition.dirichlet_split(labels, 3, 0.5, np.random.default_rng(42))

    combined = np.concatenate(parts)
    assert len(np.unique(combined)) == len(combined)


def test_dirichlet_shards_cover_every_sample(partition):
    labels = make_labels()
    parts = partition.dirichlet_split(labels, 3, 0.5, np.random.default_rng(42))

    combined = np.sort(np.concatenate(parts))
    np.testing.assert_array_equal(combined, np.arange(len(labels)))


def test_dirichlet_actually_produces_unequal_hospitals(partition):
    """A split that came out balanced would make the whole demo pointless.

```

```
        With alpha = 0.5 the sites should differ noticeably in class balance. The
        threshold is loose because the draw is random; it is here to catch a
        partitioner that has silently become uniform, not to pin an exact value.
        """
        labels = make_labels()
        parts = partition.dirichlet_split(labels, 3, 0.5, np.random.default_rng(42))

        ratios = [
            float(np.mean(labels[part] == 1)) for part in parts if len(part) > 0
        ]
        assert max(ratios) - min(ratios) > 0.1


    def test_iid_split_is_disjoint_and_balanced(partition):
        labels = make_labels()
        parts = partition.iid_split(labels, 3, np.random.default_rng(7))

        combined = np.concatenate(parts)
        assert len(np.unique(combined)) == len(labels)

        sizes = [len(part) for part in parts]
        assert max(sizes) - min(sizes) <= 1

        ratios = [float(np.mean(labels[part] == 1)) for part in parts]
        assert max(ratios) - min(ratios) < 0.1


    def test_describe_reports_counts_and_ratio(partition):
        labels = np.array([0, 0, 1, 1, 1, 1], dtype=np.int64)

        summary = partition.describe(labels)

        assert summary["examples"] == 6
        assert summary["class_counts"] == {"normal": 2, "pneumonia": 4}
        assert summary["pneumonia_ratio"] == pytest.approx(0.6667, abs=1e-4)


    def test_the_same_seed_reproduces_the_same_split(partition):
        """Reproducibility matters for the report: the numbers have to be defensible."""
        labels = make_labels()

        first = partition.dirichlet_split(labels, 3, 0.5, np.random.default_rng(11))
        second = partition.dirichlet_split(labels, 3, 0.5, np.random.default_rng(11))

        for left, right in zip(first, second):
            np.testing.assert_array_equal(left, right)

```

tests/test_privacy_and_storage.py

```
"""Tests for the privacy accountant and the checkpoint storage.

The accountant produces a number that goes straight into the report, so it is
checked against the properties an epsilon must have rather than against a value
copied out of a previous run.
"""

from __future__ import annotations

import json

import numpy as np
import pytest

from fedmed.metrics import LocalMetrics
from fedmed.privacy import build_report, compute_epsilon
from fedmed.storage import LocalStorage


# ---------------------------------------------------------------- privacy


def test_no_noise_means_no_finite_budget():
    assert compute_epsilon(0.0, 10) is None


def test_more_noise_buys_a_smaller_epsilon():
    quiet = compute_epsilon(0.4, 10)
    loud = compute_epsilon(1.2, 10)

    assert loud < quiet


def test_more_rounds_spend_more_budget():
    assert compute_epsilon(1.0, 20) > compute_epsilon(1.0, 10)


def test_subsampling_amplifies_privacy():
    """Selecting a fraction of clients each round should lower epsilon.

    This is the property the report leans on when explaining why three
    hospitals cannot reach a tight budget: with everyone participating every
    round there is no amplification to collect.
    """
    everyone = compute_epsilon(1.0, 10, sampling_rate=1.0)
    a_tenth = compute_epsilon(1.0, 10, sampling_rate=0.1)

    assert a_tenth < everyone


def test_epsilon_is_positive_and_finite():
    epsilon = compute_epsilon(0.8, 15, 1.0, 1e-5)

    assert epsilon > 0
    assert np.isfinite(epsilon)


def test_a_looser_delta_gives_a_smaller_epsilon():
    assert compute_epsilon(1.0, 10, delta=1e-3) < compute_epsilon(1.0, 10, delta=1e-7)
```

```

def test_disabled_report_records_the_mechanism_as_none():
    report = build_report(0.5, 1.0, 10, 3, enabled=False)

    assert report.mechanism == "none"
    assert report.epsilon is None
    assert report.noise_multiplier == 0.0


def test_report_serialises_for_the_dashboard():
    payload = build_report(0.5, 1.0, 10, 3).as_dict()

    assert json.loads(json.dumps(payload))["rounds"] == 10
    assert payload["sampling_rate"] == 1.0


def test_report_records_partial_participation():
    report = build_report(1.0, 1.0, 10, num_clients=10, clients_per_round=3)

    assert report.sampling_rate == pytest.approx(0.3)


# ---------------------------------------------------------------- storage


def weights():
    return [np.arange(6, dtype=np.float32).reshape(2, 3), np.ones(4, dtype=np.float32)]


def test_checkpoint_round_trip(tmp_path):
    storage = LocalStorage(str(tmp_path))
    original = weights()

    storage.save_checkpoint(3, original, {"run_id": "test", "round": 3, "accuracy": 0.9})
    restored, metadata = storage.load_latest()

    assert metadata["round"] == 3
    assert metadata["accuracy"] == 0.9
    for a, b in zip(original, restored):
        np.testing.assert_array_equal(a, b)


def test_each_round_is_kept_as_its_own_object(tmp_path):
    """Versioned checkpoints are the audit trail; overwriting would destroy it."""
    storage = LocalStorage(str(tmp_path))

    for round_number in (1, 2, 3):
        storage.save_checkpoint(round_number, weights(), {"round": round_number})

    for round_number in (1, 2, 3):
        assert (tmp_path / "global" / f"round_{round_number:03d}.npz").exists()


def test_latest_tracks_the_most_recent_round(tmp_path):
    storage = LocalStorage(str(tmp_path))

    storage.save_checkpoint(1, weights(), {"round": 1})
    storage.save_checkpoint(2, weights(), {"round": 2})

    assert storage.load_latest()[1]["round"] == 2


def test_write_and_read_json(tmp_path):
```

```
        storage = LocalStorage(str(tmp_path))

        storage.write_json("global/summary.json", {"final_accuracy": 0.88})
        payload = json.loads(storage.read_bytes("global/summary.json").decode("utf-8"))

        assert payload["final_accuracy"] == 0.88


    # ---------------------------------------------------------------- metrics


    def test_local_metrics_accumulates_rounds(tmp_path):
        path = tmp_path / "metrics.json"
        sink = LocalMetrics(str(path))

        sink.record_round("run-1", 1, {"accuracy": 0.7})
        sink.record_round("run-1", 2, {"accuracy": 0.8})

        payload = json.loads(path.read_text(encoding="utf-8"))
        assert [entry["round"] for entry in payload["rounds"]] == [1, 2]
        assert payload["rounds"][1]["accuracy"] == 0.8
        assert payload["run_id"] == "run-1"

```

infra/versions.tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.60"
    }
  }
}

provider "aws" {
  region = var.region

  default_tags {
    tags = {
      Project   = var.project
      ManagedBy = "terraform"
      Course    = var.course_tag
    }
  }
}

```

infra/variables.tf

```
variable "project" {
  description = "Name prefix for every resource this stack creates."
  type        = string
  default     = "fedmed"
}

variable "region" {
  description = "AWS region. Keep this at us-east-1 for AWS Academy Learner Lab accounts, which are pinned there."
  type        = string
  default     = "us-east-1"
}

variable "course_tag" {
  description = "Free-text tag so the whole stack is easy to find in Cost Explorer."
  type        = string
  default     = "cloud-computing-project"
}

variable "lab_role_arn" {
  description = <<-DESC
    Execution and task role for every compute resource.

    An AWS Academy Learner Lab cannot create IAM roles, so the stack reuses the
    pre-provisioned LabRole. Leave this empty and the stack builds the standard
    Learner Lab ARN from the current account id. On a normal account, point it at
    a role you created that can read the buckets and write to DynamoDB, ECR,
    CloudWatch Logs and SSM.
  DESC
  type        = string
  default     = ""
}

variable "hospital_ids" {
  description = "One private data bucket and one task is created per id."
  type        = list(string)
  default     = ["a", "b", "c"]
}

variable "task_cpu" {
  description = "Fargate CPU units per task. 1024 = 1 vCPU."
  type        = string
  default     = "1024"
}

variable "task_memory" {
  description = "Fargate memory in MiB. Must be a legal pairing with task_cpu."
  type        = string
  default     = "2048"
}

variable "num_rounds" {
  description = "Federated rounds per training run."
  type        = number
  default     = 10
}

variable "local_epochs" {
  description = "Local passes over a hospital's own data between two aggregations."
  type        = number
  default     = 1
```

```
}

variable "dp_enabled" {
  description = "Clip client updates and add Gaussian noise to the aggregate."
  type        = bool
  default     = true
}

variable "dp_noise_multiplier" {
  description = "Gaussian noise scale relative to the clipping norm. Higher means more privacy and lower accuracy."
  type        = number
  default     = 0.05
}

variable "dp_clip_norm" {
  description = "L2 norm each client update is clipped to before aggregation."
  type        = number
  default     = 1.0
}

variable "tls_enabled" {
  description = <<-DESC
    Encrypt the Flower gRPC channel.

    Requires that scripts/gen_certs.sh has been run, because the certificate
    files are read from certs_dir and stored in SSM Parameter Store. It also
    requires Cloud Map, since a hospital validates the aggregator's certificate
    against the DNS name and a Fargate task's IP changes on every run.
  DESC
  type        = bool
  default     = false
}

variable "certs_dir" {
  description = "Directory holding ca.crt, server.crt and server.key, relative to infra/."
  type        = string
  default     = "../certs"
}

variable "enable_cloud_map" {
  description = "Create a private DNS namespace so hospitals can reach the aggregator by name instead of by IP."
  type        = bool
  default     = true
}

variable "namespace_name" {
  description = "Private DNS namespace. Must match the name in the TLS certificate's SAN list."
  type        = string
  default     = "fedmed.local"
}

variable "create_lambda" {
  description = <<-DESC
    Create the inference Lambda and its API Gateway.

    Set this to false for the very first apply: the function needs an image that
    only exists after the ECR repositories have been created and pushed to. See
    docs/runbook.md for the two-phase order.
  DESC
  type        = bool
  default     = true
}

```

```
variable "api_image_tag" {
  description = "Tag of the inference API image in ECR."
  type        = string
  default     = "latest"
}

variable "log_retention_days" {
  description = "CloudWatch log retention. Short retention keeps the free tier comfortable."
  type        = number
  default     = 7
}

variable "enable_public_website" {
  description = "Serve the dashboard from an S3 static website. Turn off if the account blocks public buckets."
  type        = bool
  default     = true
}

```

infra/main.tf

```
# Shared lookups and naming.
#
# The stack deliberately uses the account's default VPC and its public subnets.
# Fargate tasks need a route to ECR, S3 and CloudWatch; the alternative to public
# subnets with public IPs is a NAT Gateway, which costs about thirty dollars a
# month and would consume a third of the course credit while sitting idle.
# Inbound access is closed by security group rules instead.

data "aws_caller_identity" "current" {}

data "aws_region" "current" {}

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

locals {
  account_id = data.aws_caller_identity.current.account_id

  # Bucket names are globally unique, so the account id is appended rather than
  # a random suffix. That keeps names stable across destroy and re-apply cycles,
  # which matters when a lab session expires mid-project.
  name_prefix = "${var.project}-${local.account_id}"

  # AWS Academy pre-creates LabRole with the permissions the lab allows.
  lab_role_arn = var.lab_role_arn != "" ? var.lab_role_arn : "arn:aws:iam::${local.account_id}:role/LabRole"

  server_dns_name = var.enable_cloud_map ? "fl-server.${var.namespace_name}" : ""

  # TLS needs a stable hostname for certificate validation, which only Cloud Map
  # provides here. Asking for TLS without Cloud Map is a configuration error and
  # is caught at plan time rather than at three in the morning before the demo.
  tls_active = var.tls_enabled && var.enable_cloud_map

  common_env = {
    NUM_ROUNDS           = tostring(var.num_rounds)
    NUM_HOSPITALS        = tostring(length(var.hospital_ids))
    LOCAL_EPOCHS         = tostring(var.local_epochs)
    STORAGE_BACKEND      = "s3"
    AWS_REGION           = var.region
    MODELS_BUCKET        = aws_s3_bucket.models.bucket
    ROUNDS_TABLE         = aws_dynamodb_table.rounds.name
    AUDIT_TABLE          = aws_dynamodb_table.audit.name
    DP_ENABLED           = var.dp_enabled ? "true" : "false"
    DP_NOISE_MULTIPLIER  = tostring(var.dp_noise_multiplier)
    DP_CLIP_NORM         = tostring(var.dp_clip_norm)
    TLS_ENABLED          = local.tls_active ? "true" : "false"
    PUSH_CLOUDWATCH      = "true"
    CLOUDWATCH_NAMESPACE = var.project
  }

  # Secret references are built by splatting over the count-guarded parameters
  # rather than by indexing them. A conditional such as
```

```
    # `tls_active ? aws_ssm_parameter.tls_ca[0].arn : ...` fails at plan time when
    # the count is zero, because Terraform still evaluates the index. Splatting an
    # empty list simply yields an empty list.
    server_tls_secrets = concat(
      [for arn in aws_ssm_parameter.tls_ca[*].arn : { name = "TLS_CA_PEM", valueFrom = arn }],
      [for arn in aws_ssm_parameter.tls_cert[*].arn : { name = "TLS_CERT_PEM", valueFrom = arn }],
      [for arn in aws_ssm_parameter.tls_key[*].arn : { name = "TLS_KEY_PEM", valueFrom = arn }],
    )

    hospital_tls_secrets = [
      for arn in aws_ssm_parameter.tls_ca[*].arn : { name = "TLS_CA_PEM", valueFrom = arn }
    ]

    # ECS rejects a container definition that carries an explicit null for
    # entryPoint or command, so the TLS bootstrap is merged in as extra keys only
    # when it is actually needed rather than being set to null when it is not.
    #
    # The bootstrap exists because the Python processes want certificate files
    # while ECS delivers SSM secrets as environment variables. A short shell prelude
    # writes them into the task's own ephemeral storage, which is destroyed with
    # the task, and then hands control to the application.
    server_tls_entrypoint = local.tls_active ? {
      entryPoint = ["/bin/sh", "-c"]
      command = [join(" && ", [
        "mkdir -p /tmp/certs",
        "printf '%s' \"$TLS_CA_PEM\" > /tmp/certs/ca.crt",
        "printf '%s' \"$TLS_CERT_PEM\" > /tmp/certs/server.crt",
        "printf '%s' \"$TLS_KEY_PEM\" > /tmp/certs/server.key",
        "export TLS_CA_PATH=/tmp/certs/ca.crt TLS_CERT_PATH=/tmp/certs/server.crt TLS_KEY_PATH=/tmp/certs/server.key",
        "exec python /app/server.py",
      ])]
    } : {}

    hospital_tls_entrypoint = local.tls_active ? {
      entryPoint = ["/bin/sh", "-c"]
      command = [join(" && ", [
        "mkdir -p /tmp/certs",
        "printf '%s' \"$TLS_CA_PEM\" > /tmp/certs/ca.crt",
        "export TLS_CA_PATH=/tmp/certs/ca.crt",
        "exec python /app/client.py",
      ])]
    } : {}
  }

  resource "terraform_data" "tls_precondition" {
    # A plan-time guard rail rather than a runtime surprise.
    lifecycle {
      precondition {
        condition     = !var.tls_enabled || var.enable_cloud_map
        error_message = "tls_enabled requires enable_cloud_map, because certificate validation needs the stable DNS nam
      }
    }
  }

```

infra/ecr.tf

```
# Container registries, one per component.
#
# These must exist before any image can be pushed, and the Lambda function
# cannot be created until its image is in place. That is why the runbook applies
# this file on its own first:
#
#     terraform apply -target=aws_ecr_repository.this

locals {
  ecr_repositories = ["server", "hospital", "api"]
}

resource "aws_ecr_repository" "this" {
  for_each = toset(local.ecr_repositories)

  name                 = "${var.project}-${each.key}"
  image_tag_mutability = "MUTABLE" # the demo repeatedly overwrites :latest
  force_delete         = true

  image_scanning_configuration {
    scan_on_push = true
  }

  encryption_configuration {
    encryption_type = "AES256"
  }

  tags = {
    Name = "${var.project} ${each.key} image"
  }
}

# Untagged layers accumulate every time :latest is overwritten. ECR storage is
# ten cents per gigabyte-month and a PyTorch image is not small, so expiring them
# keeps the bill in the noise.
resource "aws_ecr_lifecycle_policy" "expire_untagged" {
  for_each   = aws_ecr_repository.this
  repository = each.value.name

  policy = jsonencode({
    rules = [
      {
        rulePriority = 1
        description  = "Delete untagged images after one day"
        selection = {
          tagStatus   = "untagged"
          countType   = "sinceImagePushed"
          countUnit   = "days"
          countNumber = 1
        }
        action = { type = "expire" }
      },
      {
        rulePriority = 2
        description  = "Keep only the five most recent tagged images"
        selection = {
          tagStatus   = "any"
          countType   = "imageCountMoreThan"
          countNumber = 5
        }
```

```
            action = { type = "expire" }
          },
        ]
      })
    }

```

infra/ecs.tf

```
# The federation itself: one Fargate cluster, two task definitions, and the
# network rules that decide who may talk to whom.
#
# Nothing here runs on its own. Both task definitions are launched as one-off
# tasks by scripts/start_run.sh and exit when the run finishes, which is what
# keeps a training run to a few cents. There are deliberately no ECS services,
# because a service would restart the aggregator forever and bill for it.

resource "aws_ecs_cluster" "this" {
  name = var.project

  setting {
    name  = "containerInsights"
    value = "disabled" # Container Insights bills per metric and is not needed here
  }
}

resource "aws_cloudwatch_log_group" "ecs" {
  name              = "/ecs/${var.project}"
  retention_in_days = var.log_retention_days
}

# The aggregator accepts gRPC only from the hospital security group. It is not
# reachable from the internet even though the task sits in a public subnet with
# a public IP, because there is no rule that admits any other source.
resource "aws_security_group" "server" {
  name        = "${var.project}-server"
  description = "FedMed aggregation server: gRPC in from hospitals only"
  vpc_id      = data.aws_vpc.default.id

  egress {
    description = "Pull images from ECR and write to S3, DynamoDB and CloudWatch"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = { Name = "${var.project}-server" }
}

resource "aws_security_group" "hospital" {
  name        = "${var.project}-hospital"
  description = "FedMed hospital node: outbound only, nothing may dial in"
  vpc_id      = data.aws_vpc.default.id

  egress {
    description = "Reach the aggregator, ECR and the bucket owned by this hospital"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = { Name = "${var.project}-hospital" }
}

# Declared as a separate rule rather than inline, so the two groups can reference
# each other without a cycle.
resource "aws_security_group_rule" "server_from_hospitals" {
```

```
    type                     = "ingress"
    description              = "Flower gRPC from hospital nodes"
    from_port                = 8080
    to_port                  = 8080
    protocol                 = "tcp"
    security_group_id        = aws_security_group.server.id
    source_security_group_id = aws_security_group.hospital.id
  }

  # Cloud Map gives the aggregator a stable name, fl-server.fedmed.local, which is
  # what the TLS certificate is issued for. A one-off task does not register
  # itself, so scripts/start_run.sh registers the task's private IP against this
  # service once the task is running and deregisters it afterwards.
  resource "aws_service_discovery_private_dns_namespace" "this" {
    count = var.enable_cloud_map ? 1 : 0

    name        = var.namespace_name
    description = "Private namespace for the FedMed federation"
    vpc         = data.aws_vpc.default.id
  }

  resource "aws_service_discovery_service" "server" {
    count = var.enable_cloud_map ? 1 : 0

    name = "fl-server"

    dns_config {
      namespace_id = aws_service_discovery_private_dns_namespace.this[0].id

      dns_records {
        ttl  = 10 # short, because the address changes with every run
        type = "A"
      }

      routing_policy = "MULTIVALUE"
    }
  }

  resource "aws_ecs_task_definition" "server" {
    family                   = "${var.project}-server"
    requires_compatibilities = ["FARGATE"]
    network_mode             = "awsvpc"
    cpu                      = var.task_cpu
    memory                   = var.task_memory
    execution_role_arn       = local.lab_role_arn
    task_role_arn            = local.lab_role_arn

    runtime_platform {
      operating_system_family = "LINUX"
      cpu_architecture        = "X86_64"
    }

    container_definitions = jsonencode([
      merge({
        name      = "fl-server"
        image     = "${aws_ecr_repository.this["server"].repository_url}:latest"
        essential = true

        portMappings = [{
          containerPort = 8080
          protocol      = "tcp"
        }]

```

```
          environment = [
            for key, value in merge(local.common_env, {
              BIND_ADDRESS = "0.0.0.0:8080"
              RUN_ID       = "placeholder" # overridden per run by start_run.sh
            }) : { name = key, value = value }
          ]

          # Certificate material arrives as SSM secrets, so the private key never
          # appears in the task definition, in the image, or in terminal output.
          secrets = local.server_tls_secrets

          logConfiguration = {
            logDriver = "awslogs"
            options = {
              "awslogs-group"         = aws_cloudwatch_log_group.ecs.name
              "awslogs-region"        = var.region
              "awslogs-stream-prefix" = "server"
            }
          }
        }, local.server_tls_entrypoint)
      ])
    }

    # One definition serves all three hospitals. HOSPITAL_ID, DATA_BUCKET and
    # SERVER_ADDRESS are supplied as container overrides at launch, which is why
    # adding a fourth hospital needs no infrastructure change at all.
    resource "aws_ecs_task_definition" "hospital" {
      family                   = "${var.project}-hospital"
      requires_compatibilities = ["FARGATE"]
      network_mode             = "awsvpc"
      cpu                      = var.task_cpu
      memory                   = var.task_memory
      execution_role_arn       = local.lab_role_arn
      task_role_arn            = local.lab_role_arn

      runtime_platform {
        operating_system_family = "LINUX"
        cpu_architecture        = "X86_64"
      }

      container_definitions = jsonencode([
        merge({
          name      = "hospital"
          image     = "${aws_ecr_repository.this["hospital"].repository_url}:latest"
          essential = true

          environment = [
            for key, value in merge(local.common_env, {
              HOSPITAL_ID    = "placeholder"
              DATA_BUCKET    = "placeholder"
              SERVER_ADDRESS = local.server_dns_name != "" ? "${local.server_dns_name}:8080" : "placeholder:8080"
              SHARD_KEY      = "shard.npz"
              SHARD_PATH     = "/tmp/shard.npz"
            }) : { name = key, value = value }
          ]

          secrets = local.hospital_tls_secrets

          logConfiguration = {
            logDriver = "awslogs"
            options = {
              "awslogs-group"         = aws_cloudwatch_log_group.ecs.name
              "awslogs-region"        = var.region
```

```
              "awslogs-stream-prefix" = "hospital"
            }
          }
        }, local.hospital_tls_entrypoint)
      ])
    }

```

infra/storage.tf

```
# Buckets and tables.
#
# The important structural claim of the whole project lives here: each hospital
# gets its own bucket. There is no shared "raw images" bucket anywhere in the
# stack, so there is no location a curious operator could go to read the pooled
# dataset. The aggregator is given the models bucket and nothing else.

resource "aws_s3_bucket" "hospital_data" {
  for_each = toset(var.hospital_ids)
  bucket   = "${local.name_prefix}-data-${each.key}"

  # A teaching stack gets torn down and rebuilt often; without this a destroy
  # fails on any bucket that still holds a shard.
  force_destroy = true

  tags = {
    Name       = "Hospital ${upper(each.key)} private imaging data"
    DataClass  = "sensitive"
    HospitalId = each.key
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "hospital_data" {
  for_each = aws_s3_bucket.hospital_data
  bucket   = each.value.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "hospital_data" {
  for_each = aws_s3_bucket.hospital_data
  bucket   = each.value.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_versioning" "hospital_data" {
  for_each = aws_s3_bucket.hospital_data
  bucket   = each.value.id

  versioning_configuration {
    status = "Enabled"
  }
}

# Deny any request that is not TLS. Learner Lab accounts share one role across
# every task, so this policy is the enforceable half of the isolation story: it
# is attached to the bucket itself rather than to a caller.
resource "aws_s3_bucket_policy" "hospital_data_tls_only" {
  for_each = aws_s3_bucket.hospital_data
  bucket   = each.value.id

  policy = jsonencode({
```

```
      Version = "2012-10-17"
      Statement = [{
        Sid       = "DenyUnencryptedTransport"
        Effect    = "Deny"
        Principal = "*"
        Action    = "s3:*"
        Resource = [
          each.value.arn,
          "${each.value.arn}/*",
        ]
        Condition = {
          Bool = { "aws:SecureTransport" = "false" }
        }
      }]
    })

    depends_on = [aws_s3_bucket_public_access_block.hospital_data]
  }

  # Global model checkpoints. Versioning turns the bucket into an audit log: every
  # round is a distinct object, and a checkpoint cannot be silently replaced.
  resource "aws_s3_bucket" "models" {
    bucket        = "${local.name_prefix}-models"
    force_destroy = true

    tags = {
      Name      = "Federated global model checkpoints"
      DataClass = "derived"
    }
  }

  resource "aws_s3_bucket_versioning" "models" {
    bucket = aws_s3_bucket.models.id

    versioning_configuration {
      status = "Enabled"
    }
  }

  resource "aws_s3_bucket_server_side_encryption_configuration" "models" {
    bucket = aws_s3_bucket.models.id

    rule {
      apply_server_side_encryption_by_default {
        sse_algorithm = "AES256"
      }
    }
  }

  resource "aws_s3_bucket_public_access_block" "models" {
    bucket = aws_s3_bucket.models.id

    block_public_acls       = true
    block_public_policy     = true
    ignore_public_acls      = true
    restrict_public_buckets = true
  }

  # Old checkpoint versions pile up across demo runs and are never looked at again.
  resource "aws_s3_bucket_lifecycle_configuration" "models" {
    bucket = aws_s3_bucket.models.id

    rule {
```

```
    id     = "expire-old-versions"
    status = "Enabled"

    filter {}

    noncurrent_version_expiration {
      noncurrent_days = 14
    }
  }

  depends_on = [aws_s3_bucket_versioning.models]
}

# One item per federated round. run_id partitions the table so several demo runs
# coexist and the dashboard can ask for a specific one.
resource "aws_dynamodb_table" "rounds" {
  name         = "${var.project}-rounds"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "run_id"
  range_key    = "round"

  attribute {
    name = "run_id"
    type = "S"
  }

  attribute {
    name = "round"
    type = "N"
  }

  point_in_time_recovery {
    enabled = false # a teaching dataset; recovery would only add cost
  }

  tags = {
    Name = "Federated training round history"
  }
}

# Prediction audit trail. The partition key is a SHA-256 of the uploaded bytes,
# so the table proves that a given image was submitted without storing anything
# the image could be reconstructed from.
resource "aws_dynamodb_table" "audit" {
  name         = "${var.project}-audit"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "request_hash"
  range_key    = "timestamp"

  attribute {
    name = "request_hash"
    type = "S"
  }

  attribute {
    name = "timestamp"
    type = "N"
  }

  ttl {
    attribute_name = "expires_at"
    enabled        = true
  }
```

```
187
188       tags = {
189         Name = "Inference audit trail (hashes only)"
190       }
191     }
192
```

infra/lambda.tf

```
# The inference API: a Lambda container image behind an HTTP API Gateway.
#
# Lambda rather than a load-balanced service because the API is idle except
# during a demo. An Application Load Balancer costs about sixteen dollars a month
# whether or not anyone calls it; this combination costs nothing at rest and
# stays inside the free tier at demo volumes.
#
# The trade is cold starts. The image carries PyTorch, so the first request after
# an idle period takes several seconds while the runtime unpacks and the model
# loads from S3. Memory is set high partly for that reason: Lambda scales CPU
# with memory, so more memory makes the cold start noticeably shorter.

resource "aws_lambda_function" "api" {
  count = var.create_lambda ? 1 : 0

  function_name = "${var.project}-inference-api"
  role          = local.lab_role_arn
  package_type  = "Image"
  image_uri     = "${aws_ecr_repository.this["api"].repository_url}:${var.api_image_tag}"

  memory_size = 2048
  timeout     = 60

  ephemeral_storage {
    size = 512
  }

  environment {
    variables = {
      STORAGE_BACKEND     = "s3"
      MODELS_BUCKET       = aws_s3_bucket.models.bucket
      ROUNDS_TABLE        = aws_dynamodb_table.rounds.name
      AUDIT_TABLE         = aws_dynamodb_table.audit.name
      MODEL_CACHE_SECONDS = "60"
      AUDIT_ENABLED       = "true"
      CORS_ORIGINS        = "*"
      LOG_LEVEL           = "INFO"
    }
  }

  tags = { Name = "FedMed inference API" }
}

resource "aws_cloudwatch_log_group" "api" {
  count = var.create_lambda ? 1 : 0

  name              = "/aws/lambda/${var.project}-inference-api"
  retention_in_days = var.log_retention_days
}

resource "aws_apigatewayv2_api" "api" {
  count = var.create_lambda ? 1 : 0

  name          = "${var.project}-api"
  protocol_type = "HTTP"
  description   = "Public entry point for FedMed predictions and round history"

  # The dashboard is a static site on an S3 website endpoint, a different origin
  # from the API, so preflight has to be answered here as well as in FastAPI.
  cors_configuration {
```

```
    allow_origins = ["*"]
    allow_methods = ["GET", "POST", "OPTIONS"]
    allow_headers = ["content-type"]
    max_age       = 300
  }
}

resource "aws_apigatewayv2_integration" "api" {
  count = var.create_lambda ? 1 : 0

  api_id                 = aws_apigatewayv2_api.api[0].id
  integration_type       = "AWS_PROXY"
  integration_uri        = aws_lambda_function.api[0].invoke_arn
  payload_format_version = "2.0"
  timeout_milliseconds   = 30000
}

# One catch-all route. FastAPI already does the routing, so duplicating the path
# table in API Gateway would only create a second place to keep in sync.
resource "aws_apigatewayv2_route" "default" {
  count = var.create_lambda ? 1 : 0

  api_id    = aws_apigatewayv2_api.api[0].id
  route_key = "ANY /{proxy+}"
  target    = "integrations/${aws_apigatewayv2_integration.api[0].id}"
}

resource "aws_apigatewayv2_route" "root" {
  count = var.create_lambda ? 1 : 0

  api_id    = aws_apigatewayv2_api.api[0].id
  route_key = "ANY /"
  target    = "integrations/${aws_apigatewayv2_integration.api[0].id}"
}

resource "aws_apigatewayv2_stage" "default" {
  count = var.create_lambda ? 1 : 0

  api_id      = aws_apigatewayv2_api.api[0].id
  name        = "$default"
  auto_deploy = true

  # Access logs make it possible to show, during the demo, that a prediction
  # request carried no query string and therefore leaked nothing through the URL.
  access_log_settings {
    destination_arn = aws_cloudwatch_log_group.api_access[0].arn
    format = jsonencode({
      requestId      = "$context.requestId"
      httpMethod     = "$context.httpMethod"
      path           = "$context.path"
      status         = "$context.status"
      responseLength = "$context.responseLength"
      latencyMs      = "$context.responseLatency"
    })
  }

  default_route_settings {
    throttling_burst_limit = 20
    throttling_rate_limit  = 10
  }
}

resource "aws_cloudwatch_log_group" "api_access" {
```

```
  count = var.create_lambda ? 1 : 0

  name              = "/aws/apigateway/${var.project}"
  retention_in_days = var.log_retention_days
}

resource "aws_lambda_permission" "api_gateway" {
  count = var.create_lambda ? 1 : 0

  statement_id  = "AllowInvokeFromApiGateway"
  action        = "lambda:InvokeFunction"
  function_name = aws_lambda_function.api[0].function_name
  principal     = "apigateway.amazonaws.com"
  source_arn    = "${aws_apigatewayv2_api.api[0].execution_arn}/*/*"
}

```

infra/ssm.tf

```
# TLS material for the Flower channel, held in SSM Parameter Store.
#
# Run scripts/gen_certs.sh before applying with tls_enabled = true. The files are
# read from disk here and delivered to tasks as ECS secrets, so the private key
# is never baked into an image and never printed by a describe-task-definition.
#
# Caveat worth putting in the report: the key passes through Terraform state,
# which is a local file in this project. On a real deployment the state belongs
# in an encrypted S3 backend with a DynamoDB lock table, or the parameters get
# created out of band and referenced by ARN only.

resource "aws_ssm_parameter" "tls_ca" {
  count = local.tls_active ? 1 : 0

  name        = "/${var.project}/tls/ca"
  description = "FedMed self-signed root certificate, pinned by every hospital"
  type        = "SecureString"
  value       = file("${var.certs_dir}/ca.crt")

  tags = { Name = "${var.project} CA certificate" }
}

resource "aws_ssm_parameter" "tls_cert" {
  count = local.tls_active ? 1 : 0

  name        = "/${var.project}/tls/cert"
  description = "FedMed aggregator certificate for fl-server.${var.namespace_name}"
  type        = "SecureString"
  value       = file("${var.certs_dir}/server.crt")

  tags = { Name = "${var.project} server certificate" }
}

resource "aws_ssm_parameter" "tls_key" {
  count = local.tls_active ? 1 : 0

  name        = "/${var.project}/tls/key"
  description = "FedMed aggregator private key"
  type        = "SecureString"
  value       = file("${var.certs_dir}/server.key")

  tags = { Name = "${var.project} server private key" }
}

```

infra/web.tf

```
# The dashboard, served as an S3 static website.
#
# A CloudFront distribution would add HTTPS and a nicer domain, but it also adds
# fifteen minutes to every deploy while the distribution propagates, and the
# course credit does not benefit. The website endpoint is plain HTTP, which is
# acceptable here only because the dashboard carries no credentials and no
# patient data. Say so in the report rather than leaving it implicit.

resource "aws_s3_bucket" "web" {
  count = var.enable_public_website ? 1 : 0

  bucket        = "${local.name_prefix}-web"
  force_destroy = true

  tags = {
    Name      = "FedMed dashboard"
    DataClass = "public"
  }
}

resource "aws_s3_bucket_website_configuration" "web" {
  count = var.enable_public_website ? 1 : 0

  bucket = aws_s3_bucket.web[0].id

  index_document {
    suffix = "index.html"
  }

  # A single-page app has no server-side routes, so a missing key should render
  # the app rather than an XML error document.
  error_document {
    key = "index.html"
  }
}

resource "aws_s3_bucket_public_access_block" "web" {
  count = var.enable_public_website ? 1 : 0

  bucket = aws_s3_bucket.web[0].id

  # This is the one bucket in the stack that is meant to be readable by anyone.
  block_public_acls       = false
  block_public_policy     = false
  ignore_public_acls      = false
  restrict_public_buckets = false
}

resource "aws_s3_bucket_policy" "web_public_read" {
  count = var.enable_public_website ? 1 : 0

  bucket = aws_s3_bucket.web[0].id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Sid       = "PublicReadForWebsite"
      Effect    = "Allow"
      Principal = "*"
      Action    = "s3:GetObject"
```

```
            Resource  = "${aws_s3_bucket.web[0].arn}/*"
          }]
        })

        # Without this ordering the policy is rejected while the block is still on.
        depends_on = [aws_s3_bucket_public_access_block.web]
      }

```

infra/outputs.tf

```
# Everything the scripts and the dashboard need to find the deployed stack.
# scripts/start_run.sh reads these with `terraform output -json`.

output "region" {
  value = var.region
}

output "cluster_name" {
  value       = aws_ecs_cluster.this.name
  description = "ECS cluster that runs the aggregator and the hospitals."
}

output "server_task_definition" {
  value       = aws_ecs_task_definition.server.family
  description = "Task definition family for the aggregator."
}

output "hospital_task_definition" {
  value       = aws_ecs_task_definition.hospital.family
  description = "Task definition family shared by every hospital."
}

output "subnet_ids" {
  value       = data.aws_subnets.default.ids
  description = "Subnets tasks are launched into."
}

output "server_security_group" {
  value = aws_security_group.server.id
}

output "hospital_security_group" {
  value = aws_security_group.hospital.id
}

output "hospital_buckets" {
  value       = { for id, bucket in aws_s3_bucket.hospital_data : id => bucket.bucket }
  description = "Private data bucket for each hospital, keyed by hospital id."
}

output "models_bucket" {
  value       = aws_s3_bucket.models.bucket
  description = "Versioned bucket holding every global checkpoint."
}

output "rounds_table" {
  value = aws_dynamodb_table.rounds.name
}

output "audit_table" {
  value = aws_dynamodb_table.audit.name
}

output "ecr_repository_urls" {
  value       = { for key, repo in aws_ecr_repository.this : key => repo.repository_url }
  description = "Push targets for the three images."
}

output "cloud_map_service_id" {
  # join over a splat yields "" when the resource was not created, and never
```

```
  # fails the way an index into an empty list does.
  value       = join("", aws_service_discovery_service.server[*].id)
  description = "Service the aggregator's IP is registered against at run time."
}

output "server_dns_name" {
  value       = local.server_dns_name
  description = "Name hospitals dial, and the name in the TLS certificate."
}

output "tls_active" {
  value       = local.tls_active
  description = "Whether the gRPC channel is encrypted in this deployment."
}

output "log_group" {
  value = aws_cloudwatch_log_group.ecs.name
}

output "api_url" {
  value       = join("", aws_apigatewayv2_api.api[*].api_endpoint)
  description = "Base URL of the inference API. Feed this to the dashboard build."
}

output "website_url" {
  value       = join("", [for endpoint in aws_s3_bucket_website_configuration.web[*].website_endpoint : "http://${end
  description = "Public dashboard URL."
}

output "website_bucket" {
  value = join("", aws_s3_bucket.web[*].bucket)
}

```

infra/terraform.tfvars.example

```
# Copy to terraform.tfvars and edit. Every value here has a working default in
# variables.tf, so the file is optional; it exists to make the knobs visible.

# AWS Academy Learner Lab accounts are pinned to us-east-1.
region  = "us-east-1"
project = "fedmed"

# Leave empty to use arn:aws:iam::<your-account>:role/LabRole, which is what a
# Learner Lab provides and what this stack expects. On a personal account,
# point this at a role that can read the buckets and write to DynamoDB, ECR,
# CloudWatch Logs and SSM.
lab_role_arn = ""

# --- the federated run -------------------------------------------------
num_rounds   = 10
local_epochs = 1

# --- privacy -----------------------------------------------------------
# 0.05 keeps accuracy usable and produces a very loose epsilon. Raise it to see
# the trade-off; scripts/run_experiments.py measures the whole curve.
dp_enabled          = true
dp_noise_multiplier = 0.05
dp_clip_norm        = 1.0

# --- transport ---------------------------------------------------------
# Run scripts/gen_certs.sh before turning this on. It needs Cloud Map, because
# the certificate is issued for fl-server.fedmed.local and a Fargate task's IP
# changes on every run.
tls_enabled      = false
enable_cloud_map = true

# --- first apply -------------------------------------------------------
# Set this to false for the very first apply, then push the images, then set it
# back to true. The function cannot be created before its image exists.
create_lambda = true

# --- cost --------------------------------------------------------------
task_cpu           = "1024" # 1 vCPU
task_memory        = "2048" # MiB
log_retention_days = 7

```

docs/architecture.md

```
# Architecture

## The problem

Three hospitals each hold chest X-rays. Any one of them has too few images to
train a good pneumonia classifier alone, and none of them may hand its images to
the others or to a central server. The platform's job is to produce one model
that has learned from all three datasets without any image ever leaving the site
that owns it.

## The shape of the system

```
                    ?????????????????????? AWS us-east-1 ??????????????????????
                    ?                                                          ?
 Hospital A ???     ?  ECS Fargate cluster "fedmed"                           ?
 own bucket   ?     ?                                                         ?
 own task     ????????? ?????????????????                                    ?
              ? TLS ?   ?  fl-server     ????? checkpoint per round ??? S3     ?
 Hospital B ??? gRPC?   ?  FedAvg        ????? round metrics ?????????? DynamoDB?
              ?     ?   ?  + clip/noise  ????? logs and metrics ??????? CloudWatch
 Hospital C ???     ?   ?????????????????                                     ?
                    ?        ? registered in Cloud Map as                     ?
                    ?          fl-server.fedmed.local                         ?
                    ?                                                         ?
                    ?  API Gateway ?? Lambda "inference-api" ?? reads S3, DynamoDB
                    ?        GET /health  GET /model  GET /rounds  POST /predict?
                    ?                                                         ?
                    ?  S3 static website ?? React dashboard                    ?
                    ??????????????????????????????????????????????????????????
```

Components

Component	Runs on	Responsibility
Hospital node	Fargate task, 1 vCPU / 2 GB	Reads one shard from its own bucket, trains locally, returns weights
Aggregation server	Fargate task, 1 vCPU / 2 GB	FedAvg across the three updates, applies clipping and noise, eva
Inference API	Lambda container image behind an HTTP API	Serves the current global model and the round history
Dashboard	S3 static website	Operator view: convergence, per-site distributions, privacy budget, live prediction
Storage	5 S3 buckets, 2 DynamoDB tables	Three private data buckets, one versioned model bucket, one website buc
Secrets	SSM Parameter Store	The self-signed CA, aggregator certificate and private key

How one training run works

1. `start_run.sh` launches the aggregator as a one-off Fargate task.
2. It waits for the task to reach RUNNING, reads its private IP from the task's
   network interface, and registers that IP in Cloud Map under
   `fl-server.fedmed.local`.
3. It launches three hospital tasks, each with a different `HOSPITAL_ID` and
   `DATA_BUCKET`, pointing at that DNS name.
4. Each hospital downloads its own shard into task-local ephemeral storage,
   holds back fifteen percent for validation, and connects over TLS.
5. For each of ten rounds: the server sends the current global weights, each
   hospital trains one local epoch, the server clips each update to a fixed L2
   norm, averages them weighted by sample count, adds Gaussian noise, evaluates
   the result on its own test set, and writes a checkpoint and a metrics row.
6. After the last round every task exits. Nothing keeps running, and nothing
   keeps billing.
```

```
## Decisions worth defending

**Fargate rather than EKS.** An EKS control plane costs roughly seventy-three
dollars a month before a single workload runs, which would consume most of a
hundred-dollar course credit while idle. Fargate bills per task-second, so the
entire four-task training run costs about twenty cents an hour and nothing at
rest. The orchestration concepts carry over: a task definition is a pod spec, the
ECS scheduler places tasks, and the awsvpc network mode gives every task its own
elastic network interface exactly as a pod gets its own address.

**One-off tasks rather than ECS services.** A service maintains a desired count,
so it would restart the aggregator forever after the run finished and bill
overnight. Training is a job, not a server, so it is launched as a job.

**Lambda rather than a load-balanced container for inference.** The API is idle
except during a demo. An Application Load Balancer costs about sixteen dollars a
month whether or not it serves a request. Lambda scales to zero and stays inside
the free tier at demo volumes. The cost is a cold start of several seconds while
the PyTorch runtime unpacks, which is why the function is given two gigabytes of
memory: Lambda scales CPU with memory, so the extra memory buys a faster start.

**Public subnets rather than a NAT Gateway.** Tasks need outbound access to ECR,
S3 and CloudWatch. A NAT Gateway is about thirty-two dollars a month plus data
charges. Public subnets with public IPs cost nothing, and the security groups
close inbound access: the aggregator accepts port 8080 only from the hospital
security group, and the hospitals accept nothing at all.

**Cloud Map rather than passing an IP.** TLS validation needs a stable hostname
and a Fargate task's IP changes on every run. A one-off task does not register
itself the way a service does, so the launch script registers the IP manually
against a Cloud Map service. Set `enable_cloud_map = false` if the lab account
blocks the service; the hospitals then dial the IP directly and TLS has to be
turned off, which the launch script refuses to do silently.

**GroupNorm rather than BatchNorm.** BatchNorm carries running mean and variance
buffers computed on each hospital's own class distribution. Averaging those
across sites whose pneumonia rates range from fifteen to ninety-six percent
degrades the model, and the integer `num_batches_tracked` buffer does not survive
a weighted average cleanly. GroupNorm is stateless, so the parameter list Flower
exchanges contains only learnable weights. A test asserts this.

**The test set ships inside the aggregator's image.** It is not private hospital
data, and baking it in means the aggregator needs no read permission on any data
bucket at all. That is what makes the isolation claim structural rather than a
matter of trusting the code.

## Scaling to a fourth hospital

Add an id to `hospital_ids` in Terraform, which creates the bucket, then add it
to the loop in `start_run.sh`. There is no new task definition, no new image and
no code change, because a hospital is distinguished only by two environment
variables. The aggregator's `NUM_HOSPITALS` has to match, since FedAvg waits for
every participant before closing a round.

## What is deliberately not here

- **Secure aggregation.** The server sees each hospital's individual update. A
  protocol such as SecAgg+ would let it see only the sum. Flower supports this;
  it was left out to keep the moving parts inspectable, and it is the first thing
  a follow-up should add.
- **Client-level differential privacy at the hospital.** Noise is added centrally,
  which protects against someone reading the published checkpoints but not
  against the aggregator itself.
```

```
124     - **Real DICOM ingestion.** The dataset arrives as de-identified NumPy arrays.
125       A production system would need a de-identification stage before any pixel data
126       was written anywhere.
127
128     See [threat-model.md](threat-model.md) for what each of these gaps actually means.
129
```

docs/threat-model.md

```
# Threat model

What this platform actually defends against, what it does not, and why. The
useful part of a privacy claim is the boundary, so the gaps are stated as plainly
as the mitigations.

## Assets

| Asset | Why it matters |
|---|---|
| Chest X-rays held by each hospital | The regulated data. Re-identifiable in principle, and the thing the whole desi
| Weight updates sent each round | Derived from patient data. A gradient can leak information about the examples that
| The global model | Can memorise training examples and is exposed publicly through the API |
| Prediction requests | An uploaded X-ray belongs to a patient, and the fact that a query was made is itself informat

## Adversaries

**A1. A curious operator with console access.** Can list buckets, read objects,
read logs. In a Learner Lab this is every member of the team.

**A2. A compromised or dishonest hospital node.** Runs the real protocol but
inspects everything that reaches it.

**A3. A network observer** between hospital and aggregator.

**A4. A member of the public** with the API and dashboard URLs.

**A5. An honest-but-curious aggregator.** Follows the protocol and studies the
individual updates it receives.

## What holds

### Images never traverse the network

Structural, not a matter of policy. The hospital process reads its shard into
memory and the only outbound calls it makes are Flower's, which carry a parameter
list and a metrics dictionary built explicitly in `fit` and `evaluate`. There is
no code path from pixels to the wire. Against A2 and A3.

### There is no pooled image store

Each hospital gets its own bucket. No location in the account holds more than one
site's images, so no single breach exposes the combined dataset. The aggregator
never gets a bucket name for hospital data, and its test set ships inside its own
image so it has no reason to ask for one. Against A1.

### The training channel is encrypted and pinned

TLS on the gRPC channel with a self-signed CA. The hospitals pin `ca.crt`, so an
impostor cannot present its own certificate for the aggregator even though
nothing here is publicly trusted. The certificate is issued for
`fl-server.fedmed.local`, which is why Cloud Map is a prerequisite rather than a
convenience. Against A3.

### Inbound access is closed

The aggregator's security group admits port 8080 only from the hospital security
group. The hospital group admits nothing. Both sit in public subnets with public
IPs, which is a cost decision, and neither is reachable from the internet.
Against A4.
```

```
### Uploaded images are never persisted

`/predict` decodes into memory, classifies, and discards. The audit table records
a SHA-256 of the bytes, a timestamp and the predicted label. That proves a given
image was submitted if someone later presents the same file, and cannot be
inverted into the image. A test asserts that a prediction leaves the filesystem
byte-identical. Against A1 and A4.

### Published checkpoints are noised

Each client update is clipped to a fixed L2 norm and Gaussian noise is added to
the aggregate before anything is written to S3. Someone who downloads every
checkpoint from the versioned bucket is reading noised parameters. Against A1.

## What does not hold

### The differential-privacy budget is loose, and saying otherwise would be false

The accountant reports an epsilon for the noise multiplier and round count in
use. At the default setting it is in the thousands, which is not a meaningful
formal guarantee.

The reason is structural rather than a tuning mistake. Differential privacy in
federated learning gets most of its strength from subsampling amplification:
selecting a small random fraction of clients each round means any individual site
participates rarely, and the accountant credits that. With three hospitals all
participating in every round the sampling rate is one and there is no
amplification to collect. Reaching a single-digit epsilon would need a noise
multiplier that, as `artifacts/experiments.md` measures, costs twenty to thirty
accuracy points and produces a model not worth serving.

The honest claim is therefore narrow. Clipping and noise measurably blunt how
much any single patient's record can influence the published weights, and the
platform reports its exact parameters and the resulting budget instead of
labelling itself private. The fix is more participating sites, which is the
regime the technique was designed for. It is not a bigger number in the epsilon
column.

The dashboard says this in the interface, not only in this file.

### The aggregator sees individual updates

Against A5, nothing here helps. The server receives each hospital's parameters
separately, and gradient-inversion research has reconstructed recognisable
training images from updates in some settings. Secure aggregation is the answer:
under SecAgg+ the server can decrypt only the sum, never any individual
contribution. Flower implements it. It was left out to keep the protocol
inspectable for a teaching project, and it is the single most valuable addition.

Two things reduce the exposure without removing it. Each hospital sends weights
after a full local epoch over hundreds or thousands of images rather than a
single-batch gradient, which is a much harder starting point for inversion. And
clipping bounds how far any one update can move.

### Per-hospital IAM isolation is described, not enforced

The intended design gives each hospital task a role that can read only its own
bucket. An AWS Academy Learner Lab cannot create IAM roles, so every task runs as
the shared `LabRole` and could in principle read another hospital's bucket. What
is actually enforced is bucket separation, a bucket policy denying non-TLS
access, and the absence of any code path that would do it.
```

```
This is a real gap and it should be named as such in the report. On an account
that permits IAM, three task roles with a bucket-scoped read policy each would
close it, and the Terraform change is roughly twenty lines.

### The dashboard is plain HTTP with no authentication

An S3 website endpoint does not support HTTPS, and there is no login. Anyone with
the URL sees the training metrics and can call `/predict`. Acceptable only
because the page holds no credentials and the dataset is public research data.
A real deployment needs CloudFront for TLS and an authenticated identity, which
also gives the audit trail a subject worth recording.

### The model itself can leak

Membership-inference attacks ask whether a particular record was in the training
set, and a model that generalises imperfectly can answer. The noise helps and
does not eliminate this. No membership-inference evaluation was run, so no claim
is made either way.

### Terraform state holds the private key

With `tls_enabled = true` the certificate and key are read from disk and stored
in SSM, and they pass through Terraform state, which is a local file in this
project. Fine for a course; a real deployment uses an encrypted S3 backend with
a lock table, or creates the parameters out of band and references them by ARN.

## Summary

| Threat | Status |
|---|---|
| Raw images cross the network | Prevented by construction |
| One breach exposes the pooled dataset | Prevented, no pooled store exists |
| Traffic interception or impersonation | Prevented by pinned TLS |
| Public reaches the training plane | Prevented by security groups |
| Uploaded scans persisted | Prevented, hash-only audit |
| Checkpoints reveal individuals | Reduced by clipping and noise, budget is loose |
| Aggregator inspects one site's update | **Not addressed.** Needs secure aggregation |
| One hospital reads another's bucket | **Not enforced** under Learner Lab IAM |
| Dashboard confidentiality | **Not addressed.** Plain HTTP, no auth |
| Membership inference on the model | **Not evaluated** |

```

docs/cost.md

````
# Cost

The project has a hundred dollars of AWS Academy credit and needs to survive
several weeks of development plus a live demo. Every architectural choice in
`architecture.md` had cost as one of its inputs, so this file records what those
choices were worth.

## What the stack costs at rest

Nothing is running between demos. No ECS services, no load balancer, no NAT
Gateway, no EC2 instance, no provisioned database.

| Resource | Idle cost |
|---|---|
| ECS cluster | Free. It is a namespace, not a machine |
| Task definitions | Free |
| Cloud Map namespace | About $0.10 a month |
| S3 buckets | Storage only, under 20 MB total |
| DynamoDB, on demand | Storage only, a few kilobytes |
| Lambda | Free unless invoked |
| API Gateway | Free unless invoked |
| ECR | $0.10 per GB-month, about $0.30 for three images |
| CloudWatch Logs | Free under 5 GB |

Roughly **fifty cents a month** while nobody is using it.

## What a training run costs

Four tasks at 1 vCPU and 2 GB, in us-east-1, at $0.04048 per vCPU-hour and
$0.004445 per GB-hour:

```
per task-hour = 0.04048 + (2 x 0.004445) = 0.04937
four tasks    = 0.1975 per hour
```

A ten-round run takes about four minutes including task startup, so roughly
**1.3 cents per run**. Twenty runs during development is about **26 cents**.

## What the whole project should cost

| Line item | Estimate |
|---|---|
| Fargate, around 20 development and demo runs | $0.30 |
| ECR storage, three images for six weeks | $0.50 |
| S3 and DynamoDB | under $0.20 |
| Lambda and API Gateway | free tier |
| CloudWatch Logs and custom metrics | under $1.00 |
| Data transfer | under $0.50 |
| **Total** | **under $3** |

Record the real figure from Cost Explorer in the report rather than quoting this
table. The estimate is the argument; the invoice is the evidence.

## What the alternatives would have cost

This is the part worth putting in the report, because it is where the design
decisions become quantitative.

| Choice made | Alternative | Alternative's monthly cost |
````

```
|---|---|---|
| Fargate one-off tasks | EKS cluster | $73 for the control plane alone, before any node |
| Fargate one-off tasks | Three t3.medium EC2 instances, always on | About $90 |
| Lambda behind API Gateway | ECS service behind an Application Load Balancer | About $16 for the balancer, plus the
| Public subnets | Private subnets with a NAT Gateway | About $32, plus $0.045 per GB |
| S3 website | CloudFront distribution | Small, but adds 15 minutes to every deploy |
| On-demand DynamoDB | Provisioned capacity | A few dollars, and pointless at this volume |

Choosing EKS with a NAT Gateway and a load balancer, which is the conventional
production answer, would have cost about **$120 a month**. The credit would have
been gone before the first demo.

## Guard rails

**Set a budget alert on day one.** Billing, then Budgets, then a $10 monthly
budget with an alert at fifty percent. A Learner Lab that runs out of credit locks
you out of your own project, mid-project.

**Check for stragglers at the end of every session:**

```bash
aws ecs list-tasks --cluster fedmed
```

Anything listed is still billing. `./scripts/stop_run.sh` clears it.

Tag everything. The Terraform provider applies `Project = fedmed` to every
resource, so Cost Explorer can be filtered to exactly this stack. Group by tag
rather than by service when you screenshot the bill for the report.

Log retention is seven days. CloudWatch charges for stored logs indefinitely
otherwise, and nobody reads a demo log from three weeks ago.

ECR expires untagged images after a day and keeps only the five most recent
tagged ones. Every `push_images.sh` overwrites `:latest` and orphans the previous
layers, which would otherwise accumulate a gigabyte at a time.

Where the money would actually go at scale

Worth a paragraph in the report, because the answer changes. At three hospitals
and 5,000 images, compute is free in practice and the bill is noise. At fifty
hospitals with full-resolution DICOM studies, the picture inverts: training tasks
need more memory and probably GPUs, per-round communication grows with model size
rather than dataset size, and the dominant cost becomes cross-region data
transfer between hospitals and the aggregator. That last one is an argument for
placing the aggregator in the region where most participants are, and for
compressing or quantising updates, neither of which matters at this scale.
```

docs/runbook.md

```
# Runbook

Everything needed to take this from a fresh clone to a working cloud demo, and
to shut it down again without leaking credit.

Read the two rules at the bottom before your first lab session.

---

## 0. Prerequisites

| Tool | Why | Check |
|---|---|---|
| Python 3.10 or newer | Everything | `python --version` |
| Docker Desktop | Builds the three images | `docker --version` |
| Node 18 or newer | Builds the dashboard | `node --version` |
| AWS CLI v2 | Every cloud script | `aws --version` |
| Terraform 1.5 or newer | The infrastructure | `terraform --version` |
| Git Bash, WSL or a Unix shell | The `.sh` scripts | |

On Windows, run the shell scripts from Git Bash. They already work around the
path rewriting that otherwise corrupts OpenSSL subjects and ECS ARNs.

Two Windows PowerShell habits worth knowing, because both produce errors that
look like the project is broken when it is not:

- `&&` does not chain commands in Windows PowerShell 5.1. `cd dashboard && npm run dev`
  fails with "the token '&&' is not a valid statement separator". Run the two
  commands on separate lines, or use `;` between them.
- `python` on its own may resolve to the Microsoft Store stub rather than this
  project's virtualenv. Call the interpreter directly as `.venv/Scripts/python.exe`.

---

## 1. Local first

Get it working on a laptop before spending a cent. This takes about ten minutes.

```bash
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements-dev.txt   # Windows
# source .venv/bin/activate && pip install -r requirements-dev.txt  # macOS, Linux

python scripts/partition_data.py
python -m pytest
python scripts/run_local.py --rounds 10 --dp
```

You should see three hospitals connect, ten rounds complete in about half a
minute, and accuracy climb into the high eighties. Checkpoints land in
`artifacts/global/` and metrics in `artifacts/metrics.json`.

Then the same thing in containers, which is what actually gets deployed:

```bash
docker compose up --build --abort-on-container-exit --exit-code-from fl-server
```

The two extra flags matter. A plain `docker compose up` stays attached forever,
because the inference API is a web server that never exits, so the terminal looks
```

````
frozen long after training has finished. These flags stop the whole stack the
moment the aggregator completes its rounds and hand the prompt back.

To keep the API running afterwards so the dashboard has something to talk to,
start it on its own instead:

```bash
docker compose up -d inference-api
```

and stop it with `docker compose down` when you are done.

And the numbers for the report:

```bash
python scripts/train_centralized.py --epochs 10
python scripts/run_experiments.py
```

That last one takes about ten minutes and writes `artifacts/experiments.md`
plus two figures.

---

## 2. Point the CLI at your lab account

An AWS Academy Learner Lab issues temporary credentials that expire when the
session does. From the lab page, open **AWS Details**, then **AWS CLI**, and copy
the block into `~/.aws/credentials`. It contains an `aws_session_token` as well
as the key pair; all three are needed.

```bash
aws sts get-caller-identity
```

This must succeed before anything below will work. When a script fails with an
expired-token error, come back here rather than debugging the script.

---

## 3. Create the registries first

The inference Lambda cannot be created before its image exists, and the image
cannot be pushed before the registry exists. So the first apply creates only the
registries.

```bash
terraform -chdir=infra init
terraform -chdir=infra apply -target=aws_ecr_repository.this
```

---

## 4. Build and push the images

```bash
./scripts/push_images.sh
```

Three images, roughly a gigabyte each because of PyTorch. The first push takes
ten to fifteen minutes on a normal connection; later pushes only send changed
layers and take under a minute.
````

The images are built for `linux/amd64` explicitly. On an Apple Silicon laptop the default build would produce arm64 images, and the Fargate task definition pinned to X86_64 would fail at startup with an unhelpful exec format error.

---

5. Create the rest of the stack

```bash
terraform -chdir=infra apply
```

About two minutes. It creates the ECS cluster, both task definitions, the security groups, the Cloud Map namespace, five buckets, two DynamoDB tables, the Lambda function, the API Gateway and the log groups.

```bash
terraform -chdir=infra output
```

Note `api_url` and `website_url`.

Optionally, turn on TLS

```bash
./scripts/gen_certs.sh
terraform -chdir=infra apply -var tls_enabled=true
```

The certificate is issued for `fl-server.fedmed.local`, so this needs `enable_cloud_map = true`, which is the default. Terraform refuses the combination at plan time rather than letting it fail during a demo.

---

6. Upload the shards

```bash
./scripts/upload_data.sh
```

One shard per bucket, about seventeen megabytes in total. This is the only moment in the project where image data moves anywhere.

---

7. Run a federation

```bash
./scripts/start_run.sh --follow
```

The script starts the aggregator, waits for it to reach RUNNING, registers its private IP in Cloud Map, then starts the three hospitals. Ten rounds take three to four minutes including task startup. Every task exits on its own.

Watch it from another terminal if you did not pass `--follow`:

```bash
aws logs tail /ecs/fedmed --follow --since 5m
aws ecs list-tasks --cluster fedmed --desired-status RUNNING
```

Per-run overrides, useful for showing the privacy trade-off live:

```bash
ROUNDS=20 DP_NOISE=0.5 ./scripts/start_run.sh
```

---

8. Publish the dashboard

```bash
./scripts/deploy_web.sh
```

It bakes the API URL into the bundle, copies in the partition manifest and the centralised baseline, builds, and syncs to the website bucket. Open the `website_url` output.

---

9. Stop everything

```bash
./scripts/stop_run.sh
```

Run this at the end of every session, even when the run finished on its own. It also catches tasks left behind by an earlier session.

After the final demo:

```bash
./scripts/teardown.sh
```

The two rules

Stop your tasks before you close the lab. A Learner Lab session expires after four hours, but a Fargate task started inside it keeps running and keeps billing after the console has logged you out. `stop_run.sh` is the last command of every session.

Never leave an ECS service with a desired count above zero. This project creates none, deliberately. If you add one, it will restart the aggregator forever.

When something goes wrong

`ExpiredToken` or `InvalidClientTokenId`. The lab session ended. Repeat step 2.

A task stops immediately with `CannotPullContainerError`. The image is not in ECR yet, or it was built for the wrong architecture. Re-run `./scripts/push_images.sh`.

Hospitals log `server not ready` and retry. Normal for the first ten to twenty seconds. If it continues past thirty attempts, the aggregator died. Check its log stream:

```bash
aws logs tail /ecs/fedmed --since 10m --filter-pattern server
```

The run hangs with the server waiting for hospitals. FedAvg waits for all three. Check whether a hospital task stopped, and read its stop reason:

```bash
aws ecs describe-tasks --cluster fedmed --tasks <task-arn> \
  --query 'tasks[0].{status:lastStatus,reason:stoppedReason}'
```

`/predict` returns 503. No checkpoint has been written yet. Run a federation first, or wait for round one to finish.

The first `/predict` after an idle period takes ten seconds. A Lambda cold start unpacking PyTorch. Call `/health` a moment before the demo to warm it.

Terraform cannot delete a security group. A task is still holding a network interface. Run `stop_run.sh`, wait a minute, and destroy again. `teardown.sh` already does this.

`terraform apply` fails creating IAM. It should not; the stack creates no roles. If it does, `lab_role_arn` is pointing at something that does not exist. Check `aws iam get-role --role-name LabRole`.

Docker Desktop will not start on Windows, with an error naming a `.sock` file. The dialog says something like:

```
starting services: initializing Secrets Engine: listening on
unix://C:/Users/<you>/AppData/Local/docker-secrets-engine/engine.sock:
remove ...: The file cannot be accessed by the system.
```

Docker Desktop was killed without shutting down cleanly and left orphaned Unix socket files behind. Windows cannot open, rename or delete them, so Docker cannot replace them on the next start, and neither can Explorer or `del`.

Do **not** click "Reset to factory defaults" in that dialog. It deletes every image and volume, which can be tens of gigabytes.

The fix is to rename the parent directories, which is a metadata operation that never touches the unreadable files. Quit Docker Desktop, then run this in PowerShell:

```powershell
Add-Type -TypeDefinition 'using System;using System.Runtime.InteropServices;public class R{[DllImport("kernel32.dll",
$s = Get-Date -Format 'yyyyMMddHHmmss'
foreach ($d in @("$env:LOCALAPPDATA\Docker
un", "$env:LOCALAPPDATA\docker-secrets-engine")) {
  if (Test-Path -LiteralPath $d) { "$d -> $([R]::Go($d, "$d.orphaned-$s"))" }
}
```

Then start Docker Desktop again. It recreates both directories cleanly. The renamed copies can be left where they are; they are empty of anything useful and Windows will not delete them either.

`Move-Item` and `Remove-Item` are not substitutes here. `Move-Item` silently falls back to copy-then-delete when a rename fails, which recreates the same broken files at the destination and leaves the originals in place.

```
313     **Cloud Map is not permitted in your lab.** Apply with
314     `-var enable_cloud_map=false -var tls_enabled=false`. Hospitals then dial the
315     aggregator's IP directly. Say so in the report; it is a real reduction in the
316     security posture, not a detail.
317
```

docs/demo-script.md

````
# Demo script

Eight minutes, three speakers. The structure is deliberate: show the problem
before the solution, and show the honest limitation before someone asks about it.

## Twenty minutes before

Do all of this before the room fills. Nothing here should happen live.

```bash
aws sts get-caller-identity          # credentials still valid
terraform -chdir=infra output        # note api_url and website_url
aws ecs list-tasks --cluster fedmed  # must be empty
curl -s $API_URL/health              # warms the Lambda, avoiding a cold start
```

Open four tabs: the dashboard, the ECS console on the cluster, the S3 console on
the models bucket, and a terminal. Leave a previous run's results loaded in the
dashboard so there is something on screen while the live run starts.

---

## 1. The problem (1 minute, speaker A)

Open the dashboard and scroll to the hospital cards.

> Three hospitals. Hospital A has three and a half thousand X-rays, seventy-four
> percent pneumonia. Hospital C has three hundred, fifteen percent pneumonia.
> None of them has enough data to train a good model alone, and none of them is
> allowed to send images to the others.
>
> Everything you see about these hospitals came from metadata. No image in any of
> those three collections will move during this demo.

Point at the class-balance bars. The visual contrast is the argument.

---

## 2. Start the run (1 minute, speaker B)

```bash
./scripts/start_run.sh
```

Switch to the ECS console while it runs.

> One command starts four Fargate tasks: an aggregator and three hospitals. The
> script starts the aggregator first, waits for it, reads its private IP off the
> task's network interface, and publishes it in Cloud Map, because a one-off task
> does not register itself the way a long-running service does. Then the
> hospitals connect by name over TLS.
>
> These are one-off tasks, not services. When the tenth round finishes, all four
> exit and the billing stops. A whole run costs about one and a half cents.

Show the four tasks appearing in the console.

---

## 3. Watch it learn (2 minutes, speaker A)
````

Back to the dashboard. It polls every five seconds, so the chart grows live.

> The blue line is the global model on a test set that belongs to no hospital.
> The other three are each site's accuracy on its own patients.
>
> Hospital B sits high the whole time. That is not a good model, it is a site
> whose data is ninety-six percent pneumonia, so predicting pneumonia every time
> scores well locally. Hospital C, the small site with mostly normal scans, is
> the hard one. Watch the blue line pull them together.
>
> The dashed line is what one model reaches when it is allowed to pool all three
> datasets. The gap between the dashed line and the blue line is the entire price
> of this architecture. Ten rounds, non-IID split, no privacy noise: about nine
> points. With light noise, about two.

If a round is slow, cut to the CloudWatch log and show a hospital's line:
`local training done: loss ... accuracy ... on 3419 images`.

---

4. Prove nothing leaked (1 minute, speaker B)

```bash
aws logs tail /ecs/fedmed --since 5m --filter-pattern hospital
```

> This is everything a hospital node said out loud for an entire run. Image
> counts, loss, accuracy, timings. Weight arrays went over the wire; pixels did
> not. The hospital process has no code path from an image to the network.
>
> And there is no bucket in this account holding more than one hospital's data.
> The aggregator's test set is baked into its own container image, so it needs no
> read permission on a hospital bucket at all.

Show the S3 console: three separate data buckets, and the versioned models bucket
with one object per round.

---

5. The limitation, stated first (1 minute, speaker C)

Point at the privacy panel. Do not skip this. It is the strongest part of the
project and it will be the first question if you leave it out.

> We clip every update and add Gaussian noise, and the dashboard reports the
> resulting epsilon. It is in the thousands, which is not a meaningful formal
> guarantee, and we would rather say so than print a green tick.
>
> The reason is structural. Differential privacy in federated learning gets its
> strength from sampling a small fraction of participants each round. With three
> hospitals all participating every round, there is no amplification to collect.
> Getting to a single-digit epsilon needs noise that costs us thirty accuracy
> points, and we measured that.
>
> The fix is fifty hospitals, not a smaller number on this screen.

If you have `artifacts/experiments.md` open, show the noise sweep row by row.

---

6. Use the model (1 minute, speaker C)

```
Drag an X-ray onto the dashboard's upload area.

> This is the federated model, served from Lambda, loaded from the checkpoint the
> run just wrote. The image was decoded in memory, classified, and discarded. The
> audit table holds a SHA-256 of the bytes and the predicted label. That proves a
> query happened without storing anything the image could be rebuilt from.

---

## 7. Shut it down (1 minute, speaker B)

```bash
./scripts/stop_run.sh
aws ecs list-tasks --cluster fedmed
```

> Empty. Nothing is running and nothing is billing.
>
> The alternative architecture, EKS with a NAT Gateway and a load balancer, would
> have cost about a hundred and twenty dollars a month sitting idle. This has cost
> us under three dollars for the entire project, and most of that was container
> registry storage.

Show the Cost Explorer view filtered to `Project = fedmed`.

Questions to expect

Why not just anonymise and pool the images? De-identification is not the same
as anonymisation for imaging data. Chest films carry burned-in text, and a face
can be reconstructed from head CT. More practically, hospitals are usually barred
from sharing by contract and jurisdiction regardless of the technique.

Why is federated accuracy lower? Two reasons, and we measured them
separately. Averaging models trained on different distributions is lossy, worth
about four points here even with an even split. The uneven split costs another
five.

What stops the aggregator inspecting one hospital's update? Nothing in this
build, and that is the largest gap. Secure aggregation, SecAgg+, lets the server
decrypt only the sum. Flower supports it; we left it out to keep the protocol
inspectable.

Is this Kubernetes? ECS, not Kubernetes, and for a cost reason we can defend.
An EKS control plane is seventy-three dollars a month before any workload. The
concepts map directly: a task definition is a pod spec, awsvpc gives each task
its own network interface.

How would you add a fourth hospital? One id in a Terraform list and one entry
in the launch loop. No new image, no new task definition, no code change.

Is this a medical device? No, and the dashboard says so on screen. It is a
teaching demonstration on a public research dataset.
```